

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ I



Bài giảng:

KỸ THUẬT LOGIC KHẢ TRÌNH PLC

Người biên soạn: Ths. Vũ Anh Đào

Hà Nội, tháng 12 năm 2014

LỜI NÓI ĐẦU

PLC (Programmable Logic Controller), là thiết bị điều khiển khả trình cho phép thực hiện linh hoạt các thuật toán điều khiển logic thông qua một ngôn ngữ lập trình. Người sử dụng có thể lập trình để thực hiện một loạt trình tự các sự kiện. Các sự kiện này được kích hoạt bởi tác nhân kích thích (lỗi vào) tác động vào PLC hoặc qua các hoạt động có trễ như thời gian hay các sự kiện được đếm. PLC dùng để thay thế các mạch rơ le trong thực tế. PLC hoạt động theo phương thức quét các trạng thái trên đầu ra và đầu vào. Khi có sự thay đổi ở đầu vào thì đầu ra sẽ thay đổi theo.

Ngôn ngữ lập trình của PLC có thể là Ladder hay Statement List. Hiện nay có nhiều hãng sản xuất ra PLC như Siemens, Allen-Bradley, Mitsubishi Electric, General Electric, Omron, Honeywell...

Bài giảng “Kỹ thuật logic khả trình PLC” gồm bốn chương:

Chương 1 giới thiệu tổng quan về PLC, đặc điểm của nó, cách phân loại PLC theo các tiêu chí khác nhau.

Chương 2 tập trung giới thiệu các họ PLC của hãng Siemens, Omron và Misubishi.

Chương 3 giới thiệu cấu tạo, cách khai báo hai thành phần quan trọng trong PLC là Timer và Counter.

Chương 4 giới thiệu ngôn ngữ lập trình cơ bản của PLC là Ladder và Statement List. Từ những kiến thức đã học trong chương 1, 2, 3 và 4, tác giả đưa ra một số ví dụ của PLC áp dụng trong toán học, điện tử viễn thông và điều khiển.

Tôi xin chân thành cảm ơn ban Lãnh đạo Học viện, cảm ơn các Lãnh đạo Khoa và các thầy cô giáo trong khoa Kỹ thuật điện tử I đã tạo mọi điều kiện tốt nhất cho tôi hoàn thành cuốn bài giảng này.

Rất mong nhận được sự góp ý của bạn đọc.

Tác giả

MỤC LỤC

LỜI NÓI ĐẦU	2
MỤC LỤC.....	3
DANH MỤC HÌNH VẼ.....	5
DANH MỤC BẢNG BIỂU	7
DANH MỤC TỪ VIẾT TẮT	8
CHƯƠNG 1. GIỚI THIỆU VỀ PLC	9
1.1. Giới thiệu chung.....	9
1.2. Lịch sử phát triển	13
1.3. Cấu tạo và hoạt động của PLC.....	14
1.3.1. Khối xử lý trung tâm (CPU – Central Processing Unit).	15
1.3.2. Các thiết bị I/O.....	21
1.4. Các thiết bị ngoại vi.....	24
1.4.1. Bộ lập trình chuyên dụng PG (Programmer).....	24
1.4.2. Máy tính cá nhân PC.....	25
1.4.3. Các thiết bị giao diện người – máy (HMI).....	25
1.4.4. Các thiết bị ngoại vi khác.....	26
1.5. Phân loại PLC.....	26
1.6. Các họ PLC thông dụng.....	28
1.6.1. Họ SIMATIC của SIEMENS (Đức).....	28
1.6.2. Họ SYSMAC của OMRON (Nhật).....	29
1.6.3. PLC của ALLEN BRADLEY (Mỹ).....	29
1.6.4. PLC của Misubishi	29
1.7. Kết luận	30
BÀI TẬP CHƯƠNG I	31
CHƯƠNG 2. CÁC HỌ PLC	32
2.1. PLC của hãng Siemens.....	32
2.1.1. Các module của PLC S7 -300.....	32
2.1.2. Kiểu dữ liệu trong PLC S7-300.....	33
2.1.3. Tổ chức bộ nhớ CPU.....	34
2.1.4. Vòng quét chương trình của PLC.	36
2.1.5. Cấu trúc chương trình.	38
2.1.6. Ngôn ngữ lập trình.....	40
2.2. PLC của hãng Omron.....	41
2.3. PLC của hãng Mitsubishi	45
2.3.1. PLC loại FX0.....	45
2.3.2. PLC loại FX0S	45
2.3.3. PLC loại FX1S	46
2.3.4. PLC loại FX1N.....	46
2.3.5. PLC loại FX2N.....	47

2.3.6. PLC loại FX2NC.....	48
2.3.7. AnS PLC	48
2.3.8. AnSH/QnAS PLC.....	49
2.3.9. QnA/Q4AR.....	49
2.3.10. Qn PLC	49
2.4. Kết luận	50
BÀI TẬP CHƯƠNG 2	51
CHƯƠNG 3. BỘ ĐỊNH THỜI VÀ BỘ ĐẾM.....	52
3.1. Bộ định thời.....	52
3.1.1. Cấu tạo	52
3.1.2. Khai báo sử dụng.....	54
3.2. Counter.....	62
3.2.1. Cấu tạo	62
3.2.2. Khai báo sử dụng.....	63
3.3. Kết luận	66
BÀI TẬP CHƯƠNG 3	68
CHƯƠNG 4. NGÔN NGỮ LẬP TRÌNH CHO PLC.....	71
4.1. Giới thiệu chung.....	71
4.2. Tập lệnh của S7-300	72
4.2.1. Cấu trúc lệnh	72
4.2.2. Các lệnh cơ bản.....	78
4.2.3 Các lệnh toán học.....	90
4.2.4. Lệnh logic tiếp điểm trên thanh ghi trạng thái.....	93
4.2.5. Các lệnh điều khiển chương trình	96
4.3. Ngôn ngữ Ladder (LAD)	101
4.3.1. Đặc điểm.....	101
4.3.2 Tập lệnh trong S7-300.....	101
4.4. Ứng dụng	110
4.4.1. Ứng dụng trong toán học	110
4.4.2 Ứng dụng trong điện tử viễn thông	111
4.4.3. Ứng dụng trong điều khiển	112
4.5 Kết luận	122
BÀI TẬP CHƯƠNG 4	123
TÀI LIỆU THAM KHẢO	125

DANH MỤC HÌNH VẼ

Hình 1.1. Cấu trúc bên trong của một PLC.....	9
Hình 1.2. Sơ đồ ứng dụng của PLC.....	11
Hình 1.3. Một số ví dụ về PLC cỡ nhỏ (a) và cỡ lớn (b)	11
Hình 1.4. Sơ đồ khối bộ điều khiển PLC	15
Hình 1.5. Sơ đồ khối của CPU	16
Hình 1.6. Giao tiếp RS232 ASCII của Misubishi(a) và ALLEN BRADLEY (b)	20
Hình 1.7. Hệ thống BUS I/O của PLC ALLEN BRADLEY	21
Hình 1.8. Biểu đồ phân loại mạng BUS I/O.....	22
Hình 1.9. Module I/O	23
Hình 1.10. Bộ lập trình cầm tay cho PLC cỡ nhỏ.....	24
Hình 1.11. Hệ thống điều khiển PLC sử dụng PC	25
Hình 1.12. Phân loại PLC theo số lượng đầu I/O	26
Hình 1.13. PLC của Siemens.....	28
Hình 1.14. PLC loại ZEN-10C của Omron.....	29
Hình 1.15. PLC họ Misubishi.....	30
Hình 2.1. Vòng quét CPU	36
Hình 2.2. Sơ đồ vòng quét thực hiện chương trình của PLC.....	37
Hình 2.3. Lập trình tuyến tính.....	38
Hình 2.4. Lập trình có cấu trúc.....	39
Hình 2.5. Mối liên hệ giữa ba phương pháp lập trình STL, FBD và LAD	41
Hình 2.6. Khối thực hiện chức năng trừ hai số nguyên 16 bit	41
Hình 2.7. PLC của Omron.....	43
Hình 2.8. PLC FX0S, FX0N và FX1S của hãng Misubishi.....	46
Hình 2.9. PLC FX1N, FX2N và FX2NC của hãng Misubishi.....	47
Hình 3.1. Timer	52
Hình 3.2. Cấu hình giá trị thời gian trễ đặt trước trong PLC S7-300.....	53
Hình 3.3. Nguyên tắc làm việc của Timer	54
Hình 3.4 Tạo khoảng thời gian trễ 2450 giây cho Timer	55
Hình 3.5. Khai báo Timer SP bằng ba ngôn ngữ FBD, LAD và STL.....	56
Hình 3.6. Timer SP	56
Hình 3.7. Khai báo Timer SE bằng ba ngôn ngữ FBD, LAD và STL.....	57
Hình 3.8. Timer SE.....	57
Hình 3.9. Khai báo Timer SD bằng ba ngôn ngữ FBD, LAD và STL.....	58
Hình 3.10. Timer SD.....	58
Hình 3.11. Khai báo Timer SS bằng ba ngôn ngữ FBD, LAD và STL.....	59
Hình 3.12. Timer SS.....	59
Hình 3.13. Khai báo Timer SF bằng ba ngôn ngữ FBD, LAD và STL.....	60
Hình 3.14. Timer SF.....	60
Hình 3.15. Hoạt động của rơ le và Timer.	61
Hình 3.16. Các chế độ hoạt động của Timer: On Delay (a), Interval (b), Off Delay (c) và Flicker (d)	61

Hình 3.17. Phương thức hoạt động của Timer: khởi động bằng nguồn (a) và khởi động bằng tín hiệu (b).....	62
Hình 3.18. Counter.....	62
Hình 3.19. Khai báo Counter tiến bằng ba ngôn ngữ FBD, LAD và STL.....	64
Hình 3.20. Counter tiến theo sườn lên.....	65
Hình 3.21. Khai báo Counter lùi bằng ba ngôn ngữ FBD, LAD và STL	65
Hình 3.22. Counter đếm tiến, lùi theo sườn lên	66
Hình 3.23. Khai báo Counter tiến/lùi bằng ba ngôn ngữ FBD, LAD và STL.....	66
Hình 4.1. Minh hoạ lệnh FP	82
Hình 4.2. Cấu tạo thanh ghi ACCU trong S7-300	83
Hình 4.3. Minh hoạ lệnh CAW	85
Hình 4.4. Minh hoạ lệnh CAD.....	86
Hình 4.5. Lệnh rẽ nhánh theo bit trạng thái (a) nhảy xuôi và (b) nhảy ngược.	98
Hình 4.6. Nguyên tắc làm việc của lệnh LOOP	100
Hình 4.7. Chương trình khởi động động cơ ba pha.....	101
Hình 4.8. Mạch sử dụng tiếp điểm thường mở	102
Hình 4.9. Mạch sử dụng tiếp điểm thường đóng.....	102
Hình 4.10. Mạch sử dụng tiếp điểm ra.....	103
Hình 4.11. Hàm XOR.....	103
Hình 4.12. Mạch sử dụng hàm XOR.....	103
Hình 4.13. Mạch sử dụng hàm NOT	104
Hình 4.14. Mạch điện sử dụng hàm SET(S).....	104
Hình 4.15. Mạch điện sử dụng lệnh Reset (R).....	105
Hình 4.16. Mạch điện sử dụng tiếp điểm phát hiện sườn lên	105
Hình 4.17. Mạch sử dụng tiếp điểm phát hiện sườn xuống.....	106
Hình 4.18. Timer TON	106
Hình 4.19. Mạch sử dụng Timer để đóng ngắt động cơ.....	107
Hình 4.20. Counter CU.....	108
Hình 4.21. Mạch sử dụng Counter CU	108
Hình 4.22. Counter CUD.....	109
Hình 4.23. Mạch sử dụng Counter CUD	109
Hình 4.24. Cách thức hoạt động của đèn tín hiệu giao thông.....	111
Hình 4.25. Giá trị PV cho Timer	111
Hình 4.26. Mô hình hệ thống tháo/rót nhiên liệu.....	113
Hình 4.27. Lưu đồ thuật toán.....	113
Hình 4.28. Giảm đồ thời gian của hệ thống tháo rót nhiên liệu.....	114
Hình 4.29. Mô hình hệ thống đếm sản phẩm.....	117
Hình 4.30. Bảng gán địa chỉ cho các biến.....	117

DANH MỤC BẢNG BIỂU

Bảng 1.1. So sánh PLC với các cách điều khiển thông thường.....	12
Bảng 2.1. Vùng địa chỉ trong PLC S7-300	34
Bảng 2.2. các loại PLC CPM2A của Omron	42
Bảng 2.3. Các chế độ đèn báo trong PLC của hãng Omron	44
Bảng 3.1. Giá trị tương ứng độ phân giải trong Timer của S7-300	53
Bảng 4.1. Giá trị của CC0, CC1 khi thực hiện lệnh toán học	76
Bảng 4.2. Giá trị của CC0, CC1 khi thực hiện lệnh toán học với số nguyên bị tràn ô nhớ..	76
Bảng 4.3. Giá trị của CC0, CC1 khi thực hiện lệnh toán học với số thực bị tràn ô nhớ	77
Bảng 4.4. Giá trị của CC0, CC1 khi thực hiện lệnh dịch chuyển	77
Bảng 4.5. Giá trị của CC0, CC1 khi thực hiện lệnh logic trong ACCU	77
Bảng 4.6 Các dạng hợp lệ trong thanh ghi ACCU	83
Bảng 4.7. Quy tắc thay đổi của CC0 và CC1 với nhóm lệnh số nguyên 16 bits	86
Bảng 4.8. Quy tắc thay đổi của CC0 và CC1 với nhóm lệnh số nguyên 32 bits	87
Bảng 4.9. Quy tắc thay đổi của CC0 và CC1 với nhóm lệnh số thực	89
Bảng 4.10. Quy tắc thay đổi của CC0 và CC1 với các lệnh toán học	90

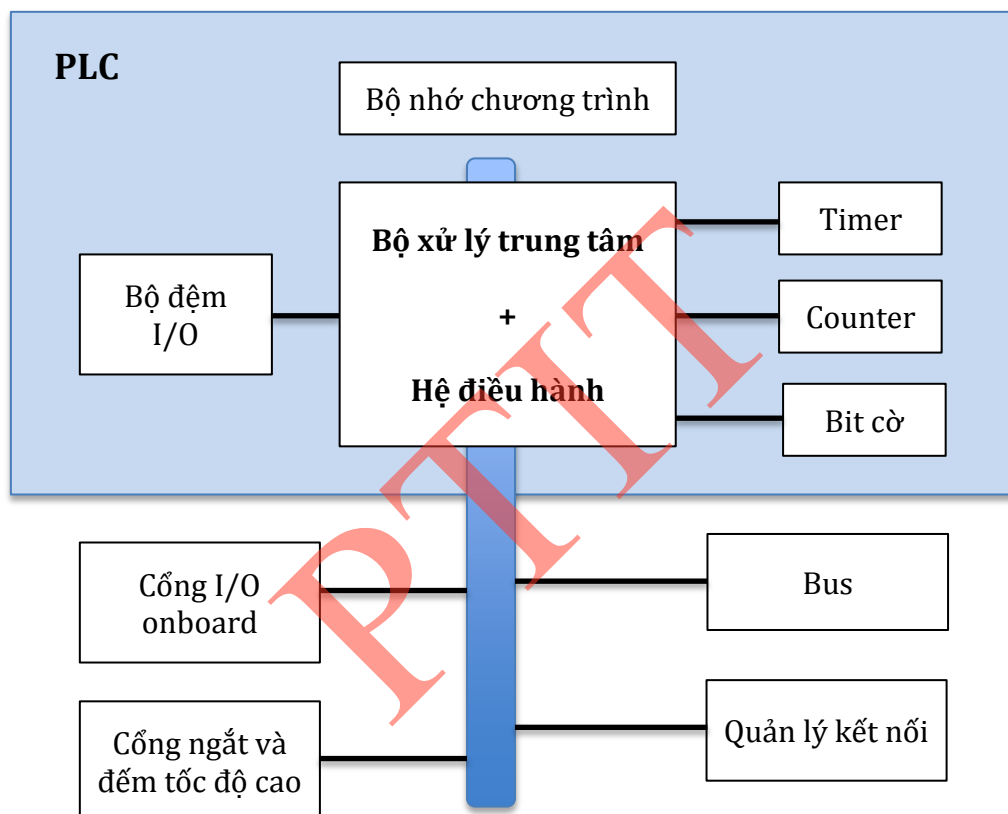
DANH MỤC TỪ VIẾT TẮT

KÝ HIỆU	TIẾNG ANH	TIẾNG VIỆT
AD	Alternating Current	Dòng điện xoay chiều
CMOS	Complementary Metal Oxide Semiconductor	chất bán dẫn oxit kim loại bổ sung
CPU	Central Processing Unit	Khối xử lý trung tâm
CRT	Cathode Ray Tube	Ông phóng tia cathodỐ
DC	Direct Current	Dòng điện một chiều
DCS	Distributed Control System	Hệ điều khiển phân tán
EPROM	Erasable Programmable ROM	ROM lập trình được và xoá được bằng tia cực tím
EEPROM	Electrically Erasable PROM	ROM lập trình được và xoá được bằng điện
FBD	Function Block Diagram	Ngôn ngữ lập trình dạng biểu đồ khối chức năng
HMI	Human – Machine Interface	Giao diện người – máy
I/O	Input/Output	Vào/ra
LAD	Ladder Logic	Ngôn ngữ lập trình hình thang
LCD	Liquid Crystal Display	Màn hình tinh thể lỏng
PC	Personal Computer	Máy tính cá nhân
PG	ProGrammer	Bộ lập trình
PLC	Programmable Logic Controller	Bộ điều khiển logic khả trình
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
ROM	Read Only Memory	Bộ nhớ chỉ đọc
SCADA	Supervisory Control And Data Acquisition	Hệ thống điều khiển giám sát và thu thập dữ liệu
STL	Statement List	Ngôn ngữ lập trình liệt kê câu lệnh

CHƯƠNG 1. GIỚI THIỆU VỀ PLC

1.1. Giới thiệu chung

Bộ điều khiển logic khả trình (Programmable Logic Controller) thực hiện linh hoạt các thuật toán điều khiển số thông qua một ngôn ngữ lập trình, thay vì phải thực hiện thuật toán đó bằng mạch số. Như vậy, PLC là một bộ điều khiển gọn, nhẹ và dễ trao đổi thông tin với môi trường bên ngoài (với các PLC khác hoặc máy tính). Toàn bộ chương trình điều khiển được lưu trữ trong bộ nhớ của PLC dưới dạng các khối chương trình và được thực hiện theo chu kỳ của vòng quét (scan).



Hình 1.1. Cấu trúc bên trong của một PLC

Cũng như các thiết bị lập trình khác, hệ thống lập trình cơ bản của PLC bao gồm hai phần là khối xử lý trung tâm (CPU) và hệ thống giao tiếp vào/ra (I/O) như sơ đồ khối trong hình 1.1.

- Khối xử lý trung tâm: Là một vi xử lý điều khiển tất cả các hoạt động của PLC như thực hiện chương trình, xử lý I/O và truyền thông với các thiết bị bên ngoài.

- Bộ nhớ: Có nhiều các bộ nhớ khác nhau dùng để chứa chương trình hệ thống là một phần mềm điều khiển các hoạt động của hệ thống, sơ đồ LAD, trị số của Timer, Counter được chứa trong vùng nhớ ứng dụng, tùy theo yêu cầu của người dùng có thể chọn các bộ nhớ khác nhau:

1. Bộ nhớ ROM: là loại bộ nhớ không thay đổi được, bộ nhớ này chỉ nạp được một lần nên ít được sử dụng phổ biến như các loại bộ nhớ khác.

2. Bộ nhớ RAM: là loại bộ nhớ có thể thay đổi được và dùng để chứa các chương trình ứng dụng cũng như dữ liệu, dữ liệu chứa trong RAM sẽ bị mất khi mất điện. Tuy nhiên, điều này có thể khắc phục bằng cách dùng pin.

3. Bộ nhớ EPROM: Giống như ROM, nguồn nuôi cho EPROM không cần dùng pin, tuy nhiên nội dung chứa trong nó có thể xóa bằng cách chiếu tia cực tím vào một cửa sổ nhỏ trên EPROM và sau đó nạp lại nội dung bằng máy nạp.

4. Bộ nhớ EEPROM: kết hợp hai ưu điểm của RAM và EPROM, loại này có thể xóa và nạp bằng tín hiệu điện. Tuy nhiên số lần nạp cũng có giới hạn.

Một PLC có đầy đủ các chức năng như: Counter, Timer, các thanh ghi (registers) và tập lệnh cho phép thực hiện các yêu cầu điều khiển phức tạp khác nhau. Hoạt động của PLC hoàn toàn phụ thuộc vào chương trình nằm trong bộ nhớ, nó luôn cập nhật tín hiệu lối vào, xử lý tín hiệu để điều khiển lối ra.

Để thực hiện được một chương trình điều khiển, PLC phải có tính năng như một máy tính, nghĩa là phải có một bộ vi xử lý (CPU), một hệ điều hành, bộ nhớ để lưu chương trình điều khiển, dữ liệu và tất nhiên phải có các cổng I/O để giao tiếp được với đối tượng điều khiển và để trao đổi thông tin với môi trường xung quanh. Bên cạnh đó, nhằm phục vụ bài toán điều khiển số, PLC còn phải có thêm một số khối chức năng đặc biệt khác như bộ đếm (Counter), bộ định thời (Timer) ... và những khối hàm chuyên dụng.

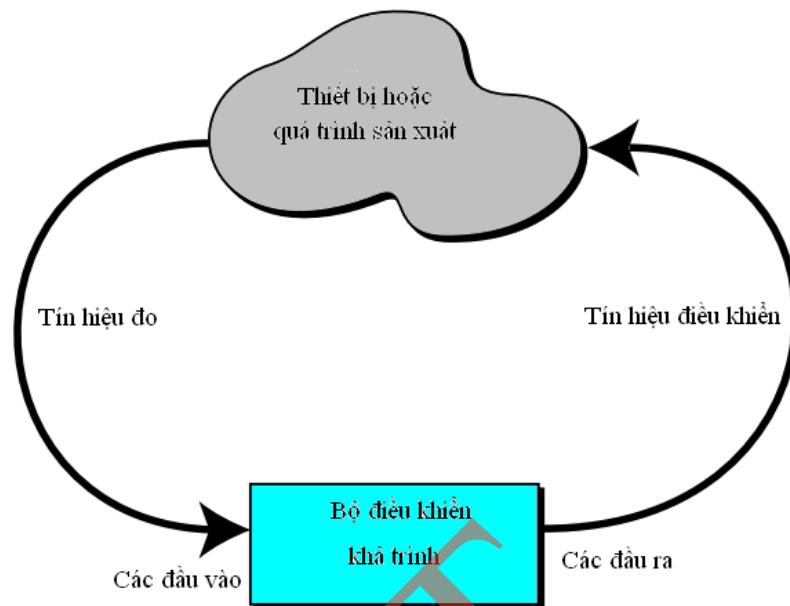
PLC là thiết bị điện tử bán dẫn, thực hiện các hàm điều khiển logic bằng chương trình, thay cho các mạch logic kiểu rơ le. Hoạt động của PLC dựa trên nguyên tắc quét vòng. PLC đọc tín hiệu logic từ các cổng vào (phím bấm, tín hiệu ra của các cảm biến...), thực hiện hàm điều khiển và gửi đến cổng ra để điều khiển các cơ cấu chấp hành (rơ le, đèn, van...).

Về bản chất, PLC là hệ vi xử lý được thiết kế tương tự máy tính số với ngôn ngữ lập trình riêng gắn gũi với người sử dụng và được áp dụng trong các bài toán điều khiển logic. Thành phần trung tâm là bộ vi xử lý (thực hiện các phép tính số học và logic) cùng bộ nhớ, các cổng I/O...

Về ứng dụng, PLC là thiết bị đặt tại dây chuyền sản xuất, tích hợp với các thành phần của hệ thống điều khiển để điều khiển trực tiếp các quá trình công nghệ. PLC thường làm việc trong môi trường khắc nghiệt (nhiệt độ cao, độ ẩm lớn, thời gian hoạt động liên tục...) nên nó thường được thiết kế với tiêu chuẩn đặc biệt về độ bền, tính module hóa cao, ngôn ngữ lập trình thân thiện với người sử dụng. Hình 1.2 minh họa một ứng dụng của PLC.

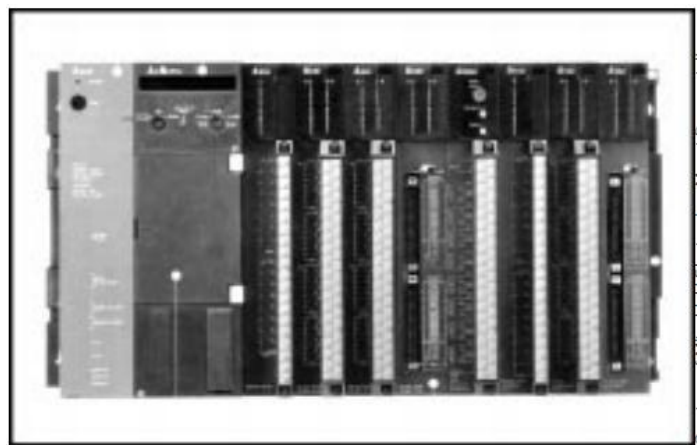
Về chức năng, PLC là thiết bị điều khiển ở mức trường. Ban đầu, PLC được dùng để điều khiển các đại lượng logic. Ngày nay, cùng với các yêu cầu ngày càng cao thì PLC được dùng như các thiết bị tính toán và điều khiển các quá trình. Sự chênh lệch

giữa PLC và máy tính công nghiệp ngày càng được thu hẹp. PLC hiện đại được trang bị thêm tính năng xử lý thông tin, quản lý dữ liệu và mở rộng các chức năng ngắt. Ngoài chức năng điều khiển, PLC còn là khâu thu thập và xử lý dữ liệu trong các hệ thống SCADA và là một nút trong hệ điều khiển phân tán (DCS).



Hình 1.2. Sơ đồ ứng dụng của PLC

Nghiên cứu ứng dụng của PLC trong các hệ thống điều khiển bao gồm hai vấn đề song song và có liên hệ chặt chẽ với nhau là phần cứng và phần mềm. Phần cứng là các thiết bị cấu thành hệ thống gồm nguồn cung cấp, CPU, module I/O, các thiết bị trợ giúp... Các thiết bị được lắp ghép với nhau tạo thành cấu hình vật lý của hệ thống. Phần mềm bao gồm hệ điều hành và phần mềm ứng dụng. Hệ điều hành do nhà sản xuất cung cấp và được cài sẵn trong bộ nhớ của PLC. Chương trình ứng dụng do người sử dụng lập trình bằng ngôn ngữ của PLC để thực hiện một thuật toán điều khiển xác định. Một chương trình ứng dụng chỉ được thiết lập trên cơ sở một cấu hình vật lý cụ thể. Ngược lại, hệ thống chỉ có thể thực hiện đúng thuật toán điều khiển nếu chương trình được thiết kế phù hợp với cấu hình của nó.



Hình 1.3. Một số ví dụ về PLC cỡ nhỏ (a) và cỡ lớn (b)

* Ưu điểm của PLC

Sự ra đời của PLC đã làm thay đổi hẳn hệ thống điều khiển cũng như các quan điểm về thiết kế chúng. PLC có nhiều ưu điểm như:

- Giảm 80% số lượng dây nối.
- Công suất tiêu thụ của PLC rất thấp .
- Có chức năng tự chuẩn đoán do đó giúp cho công tác sửa chữa được nhanh chóng và dễ dàng.
- Chức năng điều khiển thay đổi dễ dàng bằng thiết bị lập trình (máy tính, màn hình) mà không cần thay đổi phần cứng nếu không có yêu cầu thêm bớt các thiết bị vào, ra.
- Số lượng rơ le và Timer ít hơn nhiều so với hệ điều khiển cổ điển.
- Số lượng tiếp điểm trong chương trình sử dụng không hạn chế.

Bảng 1.1. So sánh PLC với các cách điều khiển thông thường.

Chỉ tiêu so sánh	Rơ le	Mạch số	Máy tính	PLC
Giá thành	Khá thấp	Thấp	Cao	Cao
Kích thước vật lý	Lớn	Rất gọn	Khá gọn	Rất gọn
Tốc độ điều khiển	Chậm	Rất nhanh	Khá nhanh	Nhanh
Khả năng chống nhiễu	Rất tốt	Tốt	Khá tốt	Tốt
Lắp đặt	Mất thời gian thiết kế và lắp đặt	Mất thời gian thiết kế	Lập trình phức tạp và tốn thời gian	Lập trình và lắp đặt đơn giản
Khả năng điều khiển các tác vụ phức tạp	Không có	Có	Có	Có
Thay đổi, nâng cấp và điều khiển	Rất khó	Khó	Khá đơn giản	Rất đơn giản
Công tác bảo trì	Kém	Kém	Kém	Tốt

- Thời gian hoàn thành một chu trình điều khiển rất nhanh (vài ms) dẫn đến tăng cao tốc độ sản xuất .
- Chương trình điều khiển có thể in ra giấy chỉ trong vài phút giúp thuận tiện cho vấn đề bảo trì và sửa chữa hệ thống.
- Lập trình dễ dàng, ngôn ngữ lập trình dễ học.

- Gọn nhẹ, dễ dàng bảo quản, sửa chữa.
- Dung lượng bộ nhớ lớn để có thể chứa được những chương trình phức tạp.
- Hoàn toàn tin cậy trong môi trường công nghiệp.
- Giao tiếp được với các thiết bị thông minh khác như máy tính, nối mạng, các module mở rộng.

- Độ tin cậy cao, kích thước nhỏ.

Bảng 1.1 so sánh bộ điều khiển PLC so với một số cách điều khiển khác để cho thấy PLC với những ưu điểm về phần cứng và phần mềm có thể đáp ứng được hầu hết các yêu cầu chỉ tiêu kỹ thuật. Mặt khác, PLC có khả năng kết nối mạng và kết nối với các thiết bị ngoại vi rất cao giúp cho việc điều khiển được dễ dàng.

*** Ứng dụng**

- Hệ thống nâng vận chuyển.
- Dây chuyền đóng gói.
- Các robot lắp ráp sản phẩm.
- Điều khiển bơm.
- Dây chuyền xử lý hoá học.
- Công nghệ sản xuất giấy.
- Dây chuyền sản xuất thuỷ tinh.
- Sản xuất xi măng.
- Công nghệ chế biến thực phẩm.
- Dây chuyền chế tạo linh kiện bán dẫn.
- Dây chuyền lắp ráp tivi.
- Điều khiển hệ thống đèn giao thông.
- Quản lý tự động bãi đậu xe.
- Hệ thống báo động.
- Dây chuyền may công nghiệp.
- Điều khiển thang máy.
- Dây chuyền sản xuất xe ô tô.
- Sản xuất vi mạch.
- Kiểm tra quá trình sản xuất

1.2. Lịch sử phát triển

Ngày nay tự động hóa ngày càng đóng vai trò quan trọng đời sống và công nghiệp, tự động hóa đã phát triển đến trình độ cao nhờ những tiến bộ của lý thuyết điều khiển tự động, tiến bộ của ngành điện tử, tin học... Chính vì vậy mà nhiều hệ thống điều

khiển ra đời, nhưng phát triển mạnh và có khả năng ứng dụng rộng là bộ điều khiển lập trình PLC.

Bộ điều khiển lập trình đầu tiên (Programmable controller) đã được những nhà thiết kế cho ra đời năm 1968 (Công ty General Motor - Mỹ), với các chỉ tiêu kỹ thuật nhằm đáp ứng các yêu cầu điều khiển:

- + Dễ lập trình và thay đổi chương trình.
- + Cấu trúc dạng module mở rộng, dễ bảo trì và sửa chữa.
- + Đảm bảo độ tin cậy trong môi trường sản xuất.

Tuy nhiên, hệ thống còn khá đơn giản và cồng kềnh, người sử dụng gặp nhiều khó khăn trong việc vận hành và lập trình hệ thống. Vì vậy các nhà thiết kế từng bước cải tiến hệ thống đơn giản, gọn nhẹ, dễ vận hành. Để đơn giản hóa việc lập trình, hệ thống điều khiển lập trình cầm tay (Programmable controller Handle) đầu tiên được ra đời vào năm 1969. Điều này đã tạo ra sự phát triển thật sự cho kỹ thuật lập trình. Trong giai đoạn này, các hệ thống điều khiển lập trình (PLC) chỉ đơn giản nhằm thay thế hệ thống rơ le và dây nối trong hệ thống điều khiển cổ. Qua quá trình vận hành, các nhà thiết kế đã từng bước tạo ra được một phương pháp lập trình mới cho hệ thống, đó là dùng giản đồ hình thang.

Sự phát triển của hệ thống phần cứng từ năm 1975 cho đến nay đã làm cho hệ thống PLC phát triển mạnh mẽ hơn với các chức năng mở rộng :

- + Số lượng lối vào, lối ra nhiều hơn và có khả năng điều khiển các lối vào, lối ra từ xa bằng kỹ thuật truyền thông.
- + Bộ nhớ lớn hơn.
- + Nhiều loại module chuyên dụng hơn.

Trong những đầu thập niên 1970, với sự phát triển của phần mềm, bộ lập trình PLC không chỉ thực hiện các lệnh logic đơn giản mà còn có thêm các lệnh về định thời, đếm sự kiện, các lệnh về xử lý toán học, xử lý dữ liệu, xử lý xung, xử lý thời gian thực..

Ngoài ra, các nhà thiết kế còn tạo ra kỹ thuật kết nối các hệ thống PLC riêng lẻ thành một hệ thống PLC chung, tăng khả năng của từng hệ thống riêng lẻ. Tốc độ của hệ thống được cải thiện, chu kỳ quét nhanh hơn. Bên cạnh đó, PLC được chế tạo có thể giao tiếp với các thiết bị ngoại, nhờ vậy mà khả năng ứng dụng của PLC được mở rộng hơn.

1.3. Cấu tạo và hoạt động của PLC

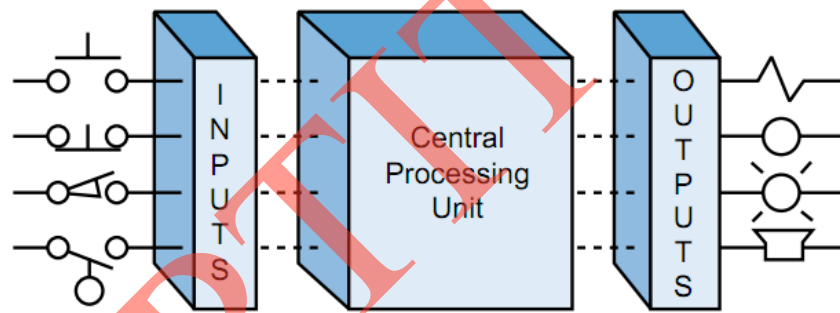
PLC là một thiết bị cho phép thực hiện các thuật toán điều khiển số thông qua một ngôn ngữ lập trình. Toàn bộ chương trình điều khiển được lưu nhớ trong bộ nhớ của PLC. Điều này có thể nói PLC giống như một máy tính, nghĩa là có bộ vi xử lý, một bộ điều hành, bộ nhớ để lưu chương trình điều khiển, dữ liệu và các cổng ra vào để

giao tiếp với các đối tượng điều khiển...Như vậy có thể thấy cấu trúc cơ bản của một PLC bao giờ cũng gồm các thành phần sau :

- + Module nguồn
- + Module xử lý tín hiệu
- + Module vào
- + Module ra
- + Module nhớ
- + Thiết bị lập trình

Sơ đồ khối của một bộ PLC cơ bản được biểu diễn ở hình 1.4. Ngoài các module chính này, các PLC còn có các module phụ trợ như module kết nối mạng, module truyền thông, module ghép nối các module chức năng để xử lý tín hiệu như module kết nối với các cảm biến nhiệt, module điều khiển động cơ bước, module kết nối với encoder, module đếm xung vào...

Các thành phần cơ bản của PLC gồm khối xử lý trung tâm (CPU – Central Processing Unit), và các module I/O, như minh họa trong hình 1.4.



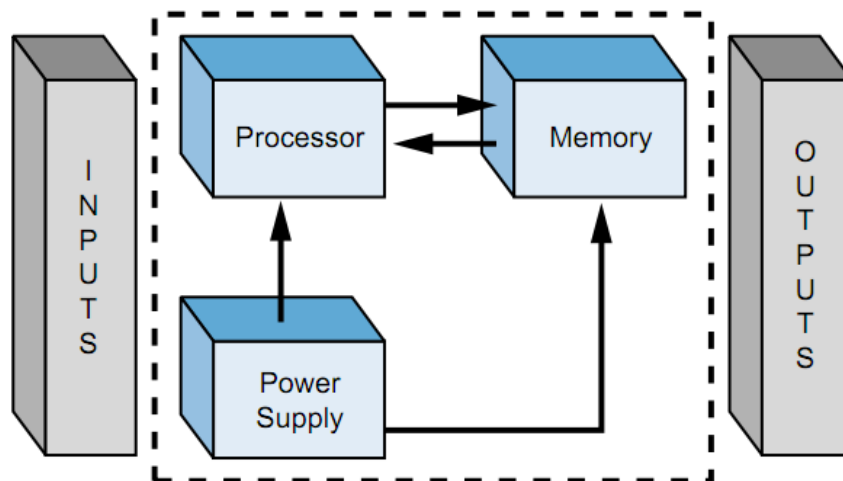
Hình 1.4. Sơ đồ khối bộ điều khiển PLC

1.3.1. Khối xử lý trung tâm (CPU – Central Processing Unit).

Đây là thành phần quan trọng nhất của PLC, nó là bộ não của PLC với hạt nhân là bộ vi xử lý, quyết định tính chất và khả năng của PLC (tốc độ xử lý, khả năng quản lý I/O...), bộ nhớ và nguồn cung cấp. CPU thực hiện chương trình trong bộ nhớ chương trình, đưa ra các quyết định và trao đổi thông tin với bên ngoài thông qua các cổng I/O.

*** Bộ vi xử lý (Processor)**

Là hạt nhân của CPU, quyết định các tính năng của CPU như tốc độ xử lý, khả năng quản lý bộ nhớ I/O. Bộ vi xử lý sử dụng có các loại 8 bit, 16 bit, 32 bit... Một số họ PLC sử dụng bộ vi xử lý tương tự như ở máy tính PC. CPU có thể có một hoặc nhiều bộ vi xử lý thực hiện các nhiệm vụ khác nhau. Lợi ích của việc sử dụng nhiều bộ vi xử lý là tăng tốc độ xử lý của PLC. Ví dụ như bộ vi xử lý thực hiện điều khiển, truyền thông, tính toán, xử lý tín hiệu phức tạp, thực hiện chức năng định thời, chức năng ngắt...



Hình 1.5. Sơ đồ khối của CPU

*** Bộ nhớ (Memory)**

Là thiết bị để lưu thông tin (chương trình, dữ liệu, tham số và cấu hình hệ thống). Việc tổ chức và quản lý bộ nhớ do hệ điều hành đảm nhiệm.

Theo tính chất, bộ nhớ được chia ra làm hai loại là bộ nhớ duy trì (Non-Volatile) và bộ nhớ không duy trì (Volatile).

+ Bộ nhớ không duy trì: nội dung sẽ bị mất khi mất nguồn nuôi. Ưu điểm của nó là khả năng ghi/đọc tốc độ cao. Trong trường hợp cần thiết, có thể dùng nguồn ổn định (pin) làm nguồn nuôi cho bộ nhớ này. Nó được sử dụng làm bộ nhớ chính cho PLC để lưu giữ chương trình và dữ liệu đang thực hiện. Việc truy nhập bộ nhớ (đọc/ghi) có thể thực hiện thông qua chương trình ứng dụng.

+ Bộ nhớ duy trì: có thể lưu giữ thông tin khi mất nguồn nuôi, tuy nhiên, tốc độ ghi thông tin vào bộ nhớ không cao và phải sử dụng thiết bị hoặc chương trình đặc biệt. Nó được sử dụng để lưu giữ các thông tin ít thay đổi, dữ liệu cần được bảo vệ khi mất nguồn (ví dụ như tham số đặt, các trạng thái đặc biệt và cấu hình của hệ thống, hệ điều hành...).

Ghi thông tin vào bộ nhớ được gọi là viết (write) và nhận thông tin từ bộ nhớ gọi là đọc (read). Việc đọc/ghi bộ nhớ được gọi là truy cập bộ nhớ. Thông tin lưu trong bộ nhớ dưới dạng số nhị phân (mỗi ký hiệu gọi là một bit thông tin).

Về tổ chức, bộ nhớ gồm các ô nhớ được sắp xếp liên tiếp nhau về mặt logic, nó chứa đựng thông tin về nội dung và địa chỉ ô nhớ trong bộ nhớ.

+ Nội dung ô nhớ chứa một từ nhị phân N bit (N là kích thước ô nhớ).

+ Địa chỉ ô nhớ là vị trí ô nhớ trong bộ nhớ. Hệ thống quản lý bộ nhớ bằng địa chỉ. Để xác định địa chỉ ô nhớ, người ta dùng địa chỉ vật lý hay địa chỉ quy ước. Hai ô nhớ liên tiếp nhau có địa chỉ hơn kém nhau 1. Địa chỉ vật lý là vị trí ô nhớ so với địa chỉ gốc của bộ nhớ (thường quy ước là 0), được biểu diễn dưới dạng số hexa. Địa chỉ quy ước do hệ điều hành quy định, người sử dụng truy cập đến địa chỉ quy ước thông qua hệ

điều hành. Địa chỉ của ô nhớ được xác định trên bus địa chỉ (Address bus). Bus địa chỉ N bit có thể địa chỉ hóa 2^N ô nhớ, gọi là không gian nhớ. Vì vậy, bus địa chỉ quy định dung lượng bộ nhớ và số điểm I/O của PLC. Việc đọc/ghi nội dung ô nhớ thông qua bus dữ liệu (data bus). Thông thường, kích thước bus dữ liệu bằng kích thước ô nhớ. Việc điều khiển quá trình truy cập bộ nhớ được thực hiện thông qua các tín hiệu điều khiển trên bus điều khiển (control bus). Về tổ chức, bộ nhớ được chia thành các vùng, mỗi vùng lưu giữ một kiểu thông tin nhất định (vùng nhớ mã chương trình gọi là bộ nhớ chương trình, vùng nhớ dữ liệu gọi là bộ nhớ dữ liệu...).

Đánh giá bộ nhớ theo dung lượng và tốc độ truy cập của bộ nhớ. Dung lượng bộ nhớ là lượng thông tin mà bộ nhớ có khả năng lưu giữ (được tính theo bit, byte, kbit, kbyte...). Tuy nhiên, đánh giá dung lượng của PLC thông qua dung lượng bộ nhớ chương trình (tính bằng K lệnh) và dung lượng bộ nhớ dữ liệu (tính bằng K từ - word).

1 k lệnh = 1024 lệnh cơ bản (k-step) của PLC

1 k từ = 1024 từ dữ liệu (k-word)

Tốc độ truy cập của bộ nhớ được đánh giá thông qua thời gian truy cập. Nếu dung lượng bộ nhớ đặc trưng cho khả năng của PLC giải các bài toán phức tạp với chương trình lớn và dữ liệu nhiều thì tốc độ truy cập đóng vai trò quan trọng. Trên thực tế, bộ nhớ trong của CPU chỉ có dung lượng nhất định, nhỏ hơn không gian nhớ để đảm bảo tính kinh tế và tính thực tiễn. Trong trường hợp yêu cầu dung lượng bộ nhớ lớn hơn dung lượng bộ nhớ trong thì ta có thể trang bị thêm thẻ nhớ mở rộng với dung lượng tùy chọn.

Bộ nhớ của PLC được xây dựng dựa trên hai loại bộ nhớ là bộ nhớ cố định (ROM, EPROM, EEPROM, FLASH ROM) và bộ nhớ đọc/ghi. Bộ nhớ cố định là bộ nhớ duy trì, nội dung của nó không mất khi mất nguồn nuôi, vì vậy nó được dùng để lưu chương trình hay dữ liệu không thay đổi. Loại EEPROM, FLASH ROM được sử dụng làm một phần của bộ nhớ trong của CPU để lưu dữ liệu, thông tin, trạng thái của hệ đang hoạt động khi mất nguồn cung cấp. Bộ nhớ đọc/ghi (RAM) là bộ nhớ không duy trì, nội dung của nó bị mất khi mất nguồn cấp, có thể dễ dàng ghi/đọc với tốc độ cao. RAM (SRAM và DRAM) được sử dụng như bộ nhớ hoạt động của PLC để lưu chương trình và dữ liệu đang thực hiện. Để duy trì thông tin trong RAM khi mất nguồn cấp thì nó phải sử dụng nguồn nuôi cố định là pin. Một số họ PLC dùng tụ điện dung lượng lớn để làm nguồn nuôi cho bộ nhớ CMOS RAM.

* Nguồn cung cấp (Power Supply)

Nguồn cung cấp đóng vai trò chính trong toàn bộ hoạt động của hệ thống, nó đảm bảo tính toàn vẹn và tin cậy cho hệ thống. Nguồn cung cấp không chỉ cấp nguồn DC cho các thành phần của hệ thống (bộ xử lý, bộ nhớ, giao diện I/O) mà còn giám sát, điều chỉnh điện áp cung cấp và cảnh báo các CPU nếu có lỗi.

+ Nguồn cấp đầu vào: thông thường PLC cấp nguồn từ nguồn AC, tuy nhiên, một số PLC chấp nhận cả nguồn DC. Trong khi hầu hết các PLC chấp nhận nguồn 120V

AC hoặc 220V AC thì các bộ điều khiển lại yêu cầu điện áp 24V DC. Điện công nghiệp thường có độ biến động về điện áp và tần số nên các PLC phải chịu được mức dao động từ 10% – 15% so với điện áp yêu cầu. Ví dụ, PLC sử dụng điện áp 120V AC, mức chịu đựng là 10% thì điện áp của nó sẽ duy trì ở mức từ 108V AC đến 132V AC. Khi điện áp cao hơn/thấp hơn mức chịu đựng trong một khoảng thời gian nào đó (thường thì khoảng 3 chu kỳ AC) thì hầu hết các nguồn cấp sẽ phát ra lệnh tắt để xử lý, điều này có thể làm cho một số quy trình công nghệ bị ảnh hưởng. Trong trường hợp đó, PLC phải chuyển sang sử dụng nguồn dự phòng để có thể duy trì điện áp (pin, tụ điện dung lượng lớn...).

+ Nguồn cấp đầu ra: Nguồn cung cấp tạo ra điện áp DC cho các mạch logic của CPU và module I/O. Nguồn cấp tạo ra dòng lớn nhất có thể đối với mức điện áp đã cho (ví dụ như 10 A ở điện áp 5 V). Bảng mạch CPU tạo nguồn cung cấp với các mức điện áp khác nhau (+5V, -5V, +12V, -12V). Khối nguồn cung cấp nguồn một chiều cho các khối được lắp vào bảng mạch BUS. Việc lựa chọn công suất khối nguồn phụ thuộc và cấu hình của hệ thống. Trong hầu hết các trường hợp, nguồn cung cấp này không phù hợp với các thiết bị trường (nên chúng được lấy từ khối nguồn riêng bên ngoài).

Ngoài ba thành phần chính trên thì trên khối CPU còn có thêm một số thành phần khác như:

* LED trạng thái

Đây là cờ trạng thái của hệ thống mà PLC lập trong quá trình hoạt động. Một số LED quan trọng là RUN, STOP, FAULT, COMM, LOW BATTERY...

RUN LED: báo PLC đang ở chế độ thực hiện chương trình trong bộ nhớ của PLC.

STOP LED: báo PLC đang ở trạng thái dừng, không thực hiện chương trình, các trạng thái ở đầu ra đều bằng 0.

FAULT: báo hệ thống có lỗi (việc xác định lỗi cụ thể phải dùng chương trình). Nếu lỗi nhẹ (non – fault) thì chương trình vẫn tiếp tục thực hiện (RUN LED vẫn sáng). Nếu lỗi nặng (faultal) thì dừng chương trình, STOP LED sáng.

COMM LED: báo việc kết nối giữa PLC và thiết bị ngoại vi đang được thực hiện.

LOW BATTERY: báo nguồn pin đã yếu, cần thay thế. Việc thay pin cho PLC phải được thực hiện khi nguồn cung cấp cho PLC vẫn được duy trì.

Chuyển mạch đặt chế độ hoạt động của PLC (DIP SWITCH)

Có N chuyển mạch ở dạng DIP SWITCH. Mỗi chuyển mạch có hai trạng thái nên sẽ có 2N chế độ hoạt động. Mỗi lần khởi động, CPU đọc trạng thái của DIP SWITCH và thiết lập chế độ hoạt động cho PLC. Việc thiết lập chế độ hoạt động cho PLC có thể thực hiện bằng phần mềm, một số được thực hiện theo DIP SWITCH theo cách sau:

+ Mỗi lần khởi động, chương trình được nạp từ bộ nhớ ngoài vào bộ nhớ trong của PLC.

- + Thiết lập chế độ sử dụng bộ nhớ ngoài.
 - + Đặt điều kiện cho phép/không cho phép hoạt động của một số ngắt
- Chuyển mạch chọn chế độ làm việc của PLC (MODE SWITCH)
- Có ba vị trí tương ứng với ba chế độ làm việc: RUN, MONITOR và STOP.

Chế độ RUN:

- + Đặt PLC vào chế độ hoạt động, thực hiện chương trình trong bộ nhớ.
- + Không cho phép sửa, thay đổi chương trình và dữ liệu trong bộ nhớ.
- + Không cho phép thay đổi chế độ hoạt động của PLC bằng thiết bị lập trình hoặc từ màn hình giao diện.

Chế độ MONITOR (còn gọi là chế độ REM – Remote):

- + Đặt PLC vào chế độ điều khiển từ xa. Các chế độ có thể chọn từ xa là REMOTE RUN, REMOTE PROGRAM, REMOTE TEST.
- + Cho phép theo dõi, thay đổi dữ liệu và chương trình trong bộ nhớ.
- + Cho phép thay đổi chế độ hoạt động của PLC bằng thiết bị lập trình, màn hình giao diện.

Chế độ STOP (còn gọi là chế độ PROGRAM):

- + Đặt PLC vào chế độ dừng.
- + Cho phép nạp chương trình vào PLC từ thiết bị lập trình và cho phép trực tiếp soạn thảo chương trình trong PLC.
- + Không cho phép thay đổi chế độ hoạt động của PLC bằng thiết bị lập trình hoặc từ màn hình giao diện.

Ở một số họ PLC, MODE SWITCH chỉ có hai chế độ là RUN và STOP. Trong trường hợp này, STOP có cả chức năng REM và PROGRAM.

*** Pin (Battery)**

Được dùng làm nguồn cung cấp cố định cho bộ nhớ duy trì. Vai trò của pin là duy trì nội dung một phần bộ nhớ của PLC khi ngắt nguồn cấp. Đó là:

- + Chương trình trong bộ nhớ RAM.
- + Dữ liệu của chương trình như giá trị đặt, giá trị các Counter, kết quả tính toán...
- + Các trạng thái đặc biệt khi thực hiện chương trình. Các trạng thái này cần phải được duy trì khi nguồn cấp mất đột ngột, là điều kiện để thực hiện chương trình khi nguồn được cung cấp lại (các trạng thái báo lỗi, trạng thái chỉ hướng, chiều chuyển động của thiết bị, các vị trí của cơ cấu chấp hành...).
- + Các tham số thiết lập cấu hình của hệ thống (chọn chế độ hoạt động của các cổng truyền thông nối tiếp, đặt chế độ làm việc đặc biệt của PLC theo yêu cầu của người sử dụng...).
- + Thực hiện thời gian thực của hệ thống.

Vì pin có vai trò quan trọng và thời gian sống của nó lại có hạn nên khi hệ sắp hết pin, cờ trạng thái LOW BATTERY sẽ sáng để báo hiệu cần phải thay pin mới. Việc thay pin phải được tiến hành trong thời gian có nguồn cấp.

Một số PLC cỡ nhỏ sử dụng tụ có dung lượng lớn làm pin. Trong điều kiện tiêu chuẩn, thời gian duy trì nội dung bộ nhớ có thể kéo dài vài chục ngày sau khi ngắt nguồn.

Để kéo dài tuổi thọ của pin, một số PLC đời mới còn dùng bộ nhớ EEPROM làm bộ nhớ duy trì (khi ngắt nguồn, nội dung trong EEPROM không bị mất). Tuy nhiên, việc ghi và xóa nội dung trong EEPROM phải được thực hiện bằng chương trình và thiết bị riêng được tích hợp trong PLC.

*** Khe cắm thẻ nhớ (Rack)**

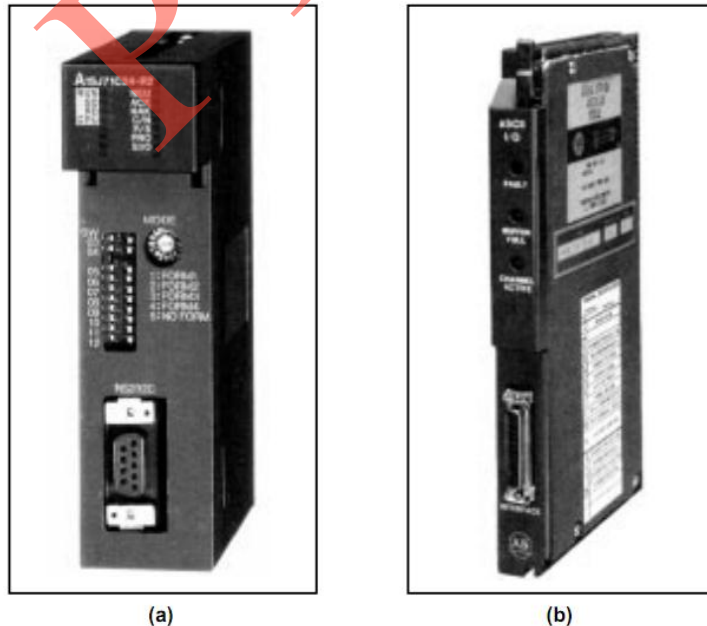
Dùng để lắp thẻ nhớ bên ngoài, đó là:

- + Thẻ nhớ RAM/EEPROM dùng để mở rộng bộ nhớ hoạt động cho PLC.
- + Thẻ nhớ ngoài EPROM/EEPROM lưu giữ chương trình điều khiển đã hoàn thiện. Chương trình sẽ được nạp vào bộ nhớ trong một cách tự động khi khởi động máy hoặc qua một vài thao tác đơn giản.

Thẻ nhớ ngoài có các loại 8KB, 16 KB, 32 KB...

*** Cổng giao tiếp song song:**

CPU thực hiện trao đổi dữ liệu với các module I/O bằng phương pháp song song thông qua hệ thống BUS. Cổng giao tiếp song song được thực hiện bằng cách mạch ghép nối giữa CPU và hệ thống BUS.



Hình 1.6. Giao tiếp RS232 ASCII của Misubishi(a) và ALLEN BRADLEY (b)

*** Cổng giao tiếp nối tiếp:**

CPU thực hiện trao đổi thông tin với các thiết bị ngoại vi thông qua cổng nối tiếp bằng phương pháp không đồng bộ. Các thiết bị ngoại vi gồm có bộ lập trình PG

(Programmer), máy tính PC, giao diện người máy HMI (Human Machine Interface), bảng vận hành OP (Operator) và các thiết bị ngoại vi khác. Trên khối CPU có một hoặc nhiều cổng nối tiếp và mỗi họ PLC chọn một chuẩn cổng nối tiếp cụ thể làm cổng chuẩn của thiết bị (Siemens chọn RS485, OMRON chọn RS422...). Nếu trên khối CPU chỉ có một cổng nối tiếp thì đó là cổng chuẩn của hãng. Nếu trên CPU có hai cổng thì cổng thứ hai thường là cổng RS232... Các PLC thế hệ mới sử dụng cổng USB. Thực hiện ghép nối giữa các cổng nối tiếp có chuẩn khác nhau qua bộ chuyển đổi (bộ chuyển đổi RS485/RS232, bộ chuyển đổi RS422/RS232...).

1.3.2. Các thiết bị I/O

Các thiết bị I/O thực hiện ghép nối giữa CPU và các thiết bị ngoại vi, nó bao gồm BUS module và các module I/O riêng biệt.

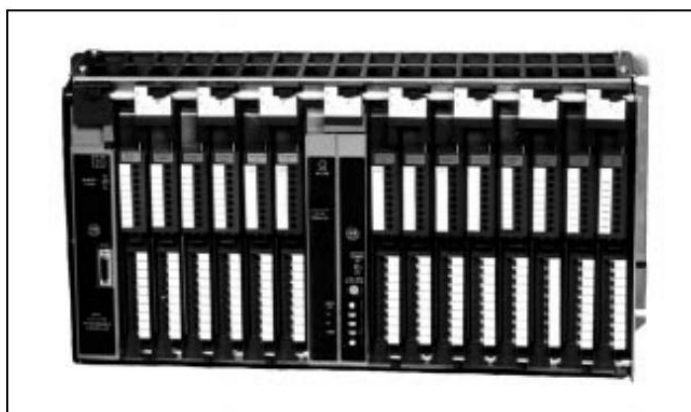
1.3.2.1. BUS module và hệ thống BUS

BUS module dùng để ghép nối giữa CPU và các module I/O.

+ Về vật lý, BUS module là khung kim loại vững chắc để đỡ bảng mạch BUS, trên đó là BUS hệ thống và các khe cắm cho các module chức năng. BUS hệ thống gồm có BUS dữ liệu (Data BUS), BUS địa chỉ (Address BUS), BUS điều khiển (Control BUS) và BUS nguồn (Power Supply BUS). BUS module có nhiều kích cỡ tùy thuộc vào số khe cắm (4, 7, 10, 16 khe). Trên BUS module có các đầu nối (Terminal) để nối với các thiết bị bên ngoài. Cấu hình cụ thể của PLC có thể dùng một hoặc nhiều BUS module tùy thuộc vào số các module chức năng. Khi đó, các BUS module được nối với nhau bằng đầu nối (BUS connector), tạo thành thống BUS (BUS system). BUS module chứa CPU gọi là BUS module chính, có địa chỉ 0 (các BUS module khác gọi là BUS module mở rộng, có địa chỉ là 1, 2, 3...).

Khi lập trình cấu hình của hệ điều khiển, người ta phải khai báo các thành phần của hệ thống BUS gồm số lượng, loại module, địa chỉ...

+ Về logic, BUS module được tổ chức ở dạng đơn vị logic (Logic Rack) để hệ thống quản lý các module I/O được gắn trên BUS module. Mỗi đơn vị logic có 128 đầu vào và 128 đầu ra.

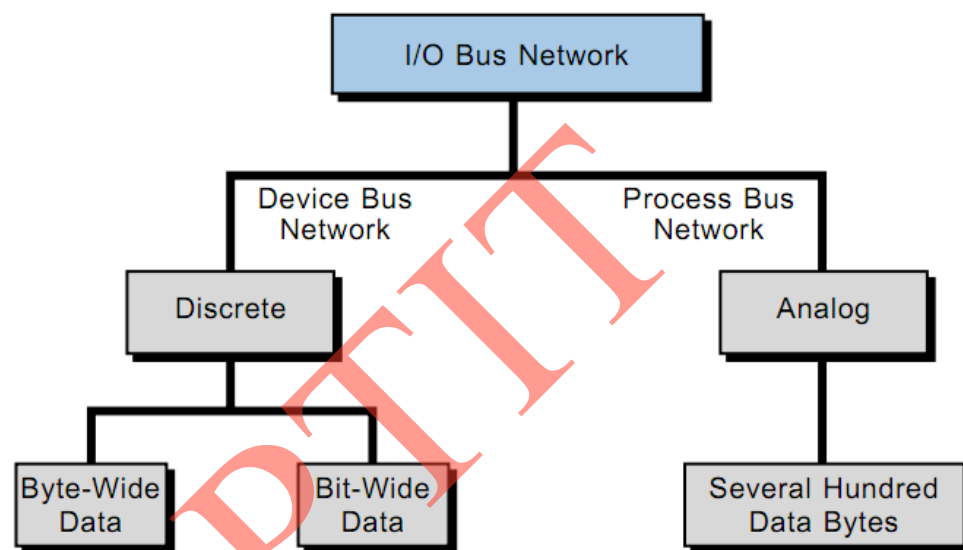


Hình 1.7. Hệ thống BUS I/O của PLC ALLEN BRADLEY

Tùy thuộc vào CPU, mỗi đơn vị logic gồm một số từ (word) dữ liệu trong vùng ảnh đầu vào và vùng ảnh đầu ra. Ví dụ, với CPU 8 bit, mỗi đơn vị logic gồm 16 byte vùng ảnh đầu vào và 16 byte vùng ảnh đầu ra. Với CPU 16 bit, mỗi đơn vị logic có 8 word ảnh đầu vào và 8 word ảnh đầu ra. Trên mỗi BUS module có thể chứa một hoặc nhiều đơn vị logic và ngược lại, một đơn vị logic có thể gồm một vài BUS module, tùy thuộc vào loại module I/O được sử dụng.

BUS module và hệ thống bus của các họ PLC hiện đại và các PLC cỡ nhỏ có thể được tích hợp trong các module chức năng. Khi đó, việc ghép nối các module chức năng được thực hiện thông qua các đầu nối (jack) trên module và thanh đỡ (rail).

Trong một hệ thống điều khiển, người ta thường sử dụng mạng BUS I/O để quản lý. Nó được chia ra làm hai loại chính là mạng BUS thiết bị và mạng BUS quá trình. Hình 1.8 là biểu đồ phân loại của một mạng BUS I/O trong hệ thống điều khiển.



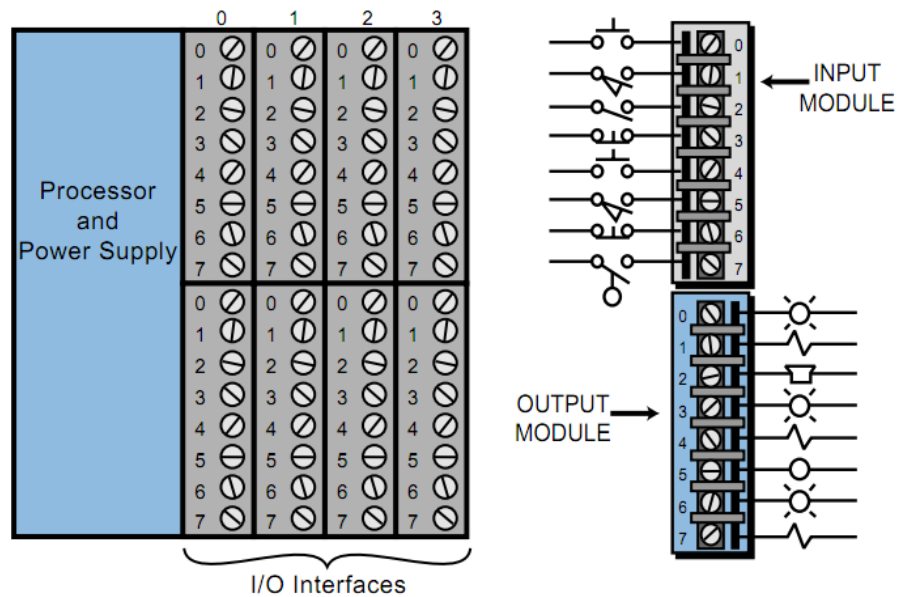
Hình 1.8. Biểu đồ phân loại mạng BUS I/O

1.3.2.2. Module I/O

Module I/O là các cổng ghép nối PLC với thiết bị bên ngoài (thiết bị trường – Field Device). Cổng I/O có nhiệm vụ chuyển đổi thích ứng giữa nguồn tín hiệu vào PLC. Module vào nhận tín hiệu từ thiết bị vào (phím bấm, công tắc vận hành, cảm biến...) và chuyển đổi thành dữ liệu. Module ra có nhiệm vụ ghép nối PLC với các thiết bị ra, chuyển đổi dữ liệu thành tín hiệu điều khiển các cơ cấu chấp hành (rơ le, van, đèn...).

Trên thực tế, các cổng I/O có hai loại là loại cố định (fixed) và loại module hóa (modular). Loại cố định được sử dụng cho các PLC cỡ nhỏ, cổng I/O gắn cố định vào CPU. Ưu điểm của loại này là giá thành thấp. Tuy nhiên, muốn mở rộng cổng I/O thì phải trang bị thêm khối mở rộng tương ứng. Loại module hóa được sử dụng trong đa số các trường hợp, và là cấu trúc tiêu chuẩn của PLC. Các module I/O có thể tháo lắp, thay đổi vị trí trên khe cắm/rãnh một cách dễ dàng. Cấu trúc kiểu này (gồm các đầu

nối) tạo thành bảng mạch BUS (Backplane), trên đó có thể lắp các khối nguồn, CPU I/O, module mở rộng... và thực hiện trao đổi thông tin với nhau.



Hình 1.9. Module I/O

Thiết bị/phần tử logic nối với PLC và gửi tín hiệu đến module vào được gọi là thiết bị/phần tử vào. Tín hiệu mà thiết bị vào gửi đến PLC gọi là tín hiệu vào. Hầu hết các họ PLC có tín hiệu vào logic với mức thấp (0V) và mức cao (24 V) đối với một chiều; mức thấp (0V) và mức cao (110 ÷ 220V) đối với xoay chiều. PLC nhận tín hiệu vào qua các đầu nối có vị trí xác định trên module vào, gọi là các điểm đầu vào.

Trạng thái tín hiệu tại mỗi đầu nối được PLC đọc và lưu giữ trong một vùng nhớ dữ liệu, gọi là vùng ảnh đầu vào (gồm các thanh ghi ảnh đầu vào). Mỗi đầu nối tương đương một bit trong thanh ghi ảnh đầu vào. Địa chỉ của bit là địa chỉ của đầu nối tương ứng. Khi tín hiệu vào ở mức cao thì bit tương ứng có giá trị bằng 1, ngược lại thì có giá trị bằng 0.

PLC thực hiện lần lượt từng lệnh của chương trình trong bộ nhớ. Kết quả thực hiện chương trình là các dữ liệu hoặc các quyết định gửi ra bên ngoài. Dữ liệu trong bộ nhớ dùng để lưu giữ hoặc để làm điều kiện thực hiện lệnh tiếp theo. Các quyết định gửi ra bên ngoài ở dạng dữ liệu ra, được gửi đến một vùng nhớ dữ liệu gọi là vùng ảnh đầu ra (gồm các thanh ghi ảnh đầu ra).

Thiết bị nối với PLC và nhận tín hiệu điều khiển từ module ra gọi là thiết bị ra hoặc cơ cấu chấp hành. PLC gửi tín hiệu điều khiển thiết bị ra thông qua các đầu nối có vị trí xác định trên module ra, được gọi là điểm đầu ra.

Mỗi điểm đầu ra tương ứng với một bit trong thanh ghi ảnh đầu ra. Địa chỉ của bit là địa chỉ của điểm đầu ra tương ứng. Giá trị của bit ảnh đầu ra quyết định trạng thái của thiết bị ra. Thiết bị ra logic có hai trạng thái là tích cực (active) và không tích cực (inactive). Trạng thái tích cực/không tích cực của thiết bị do tính chất của thiết bị và do người sử dụng quy định.

Ở trạng thái hoạt động (RUN MODE), PLC thực hiện đọc trạng thái tín hiệu vào, ghi vào vùng ảnh đầu vào, thiết lập trạng thái ảnh đầu ra trên cơ sở thực hiện chương trình. Quá trình này được lặp lại liên tục, gọi là quét vòng. Đây chính là nguyên tắc hoạt động của PLC.

Do nguồn tín hiệu vào các thiết bị ra rất đa dạng về chủng loại nên các module I/O có rất nhiều loại, ví dụ như loại rời rạc, tương tự hoặc loại đặc biệt (thực hiện một chức năng đặc biệt nào đó). Các module I/O được chế tạo theo chuẩn và ghép nối với CPU qua các khe cắm trên BUS module.

Việc trao đổi dữ liệu giữa CPU và các module I/O bằng phương pháp song song nhờ thao tác đọc/ghi. Mỗi lần trao đổi (đọc/ghi) một từ dữ liệu (8 hoặc 16 bit). Hệ thống quản lý các đầu I/O theo địa chỉ. Địa chỉ này được xác định trên cơ sở vị trí của khe cắm, kiểm module I/O.

Các thiết bị I/O ghép nối CPU với các thiết bị ngoại vi, nó bao gồm BUS module và các module I/O riêng biệt.

1.4. Các thiết bị ngoại vi

Các thiết bị ngoại vi trao đổi thông tin với CPU qua cổng nối tiếp.

1.4.1. Bộ lập trình chuyên dụng PG (Programmer)



Hình 1.10. Bộ lập trình cầm tay cho PLC cỡ nhỏ

Là thiết bị dùng để lập trình cho PLC, trao đổi dữ liệu với PLC và điều khiển hoạt động của PLC (soạn thảo chương trình, nạp chương trình vào PLC, theo dõi hoạt động của chương trình, phát hiện và sửa lỗi...). Bộ lập trình là một máy tính chuyên dụng được cài phần mềm lập trình và trang bị các công cụ giao tiếp với PLC. Bộ lập trình có các loại sau:

+ Bộ lập trình cầm tay: là bộ lập trình gọn nhẹ, tiện lợi cho người sử dụng khi cài đặt, thay đổi tham số, lệnh chương trình của PLC đang hoạt động.

Nhược điểm của bộ lập trình này là màn hình hiển thị LCD nhỏ, HMI không thuận tiện, thông tin hiển thị ở dạng ký tự và thao tác phức tạp.

+ Bộ lập trình chuyên dụng có màn hình CRT, tương tự như laptop. Thông tin hiển thị trên màn hình lớn, ở dạng ký tự hoặc đồ họa nhằm tăng tính trực quan và dễ giao tiếp. Ưu điểm của bộ lập trình này là bộ nhớ lớn, tốc độ xử lý nhanh nên nó có nhiều chức năng điều khiển, giao tiếp và xử lý thông tin từ PLC tương tự như PC.

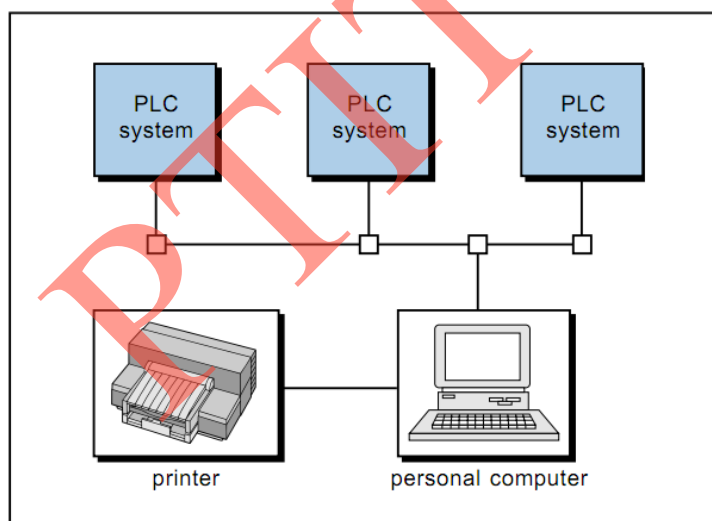
1.4.2. Máy tính cá nhân PC

PC là thiết bị ngoại vi được sử dụng rộng rãi do nó có ưu điểm về tốc độ xử lý, dung lượng bộ nhớ lớn và giá thành rẻ. PC được sử dụng với các chức năng:

+ PC được cài phần mềm lập trình, trở thành bộ lập trình cho PLC rất thuận tiện vì sử dụng hệ điều hành và các công cụ phần mềm thông dụng.

+ PC đóng vai trò như trạm thu thập dữ liệu từ PLC, xử lý thông tin, tham gia vào các hệ thống điều khiển, vận hành và quản lý hệ thống.

+ PC được cài đặt chương trình mô phỏng PLC đóng vai trò như PLC ảo, phục vụ cho việc đào tạo và thiết kế hệ thống với chi phí rẻ.



Hình 1.11. Hệ thống điều khiển PLC sử dụng PC

1.4.3. Các thiết bị giao diện người – máy (HMI).

Các thiết bị giao diện người – máy nối với PLC được gọi là các trạm vận hành hoặc giao diện vận hành hệ thống. Các thiết bị này là một máy tính chuyên dụng với màn hình LCD đen trắng hoặc màu. Người sử dụng có thể tạo lập các giao diện để vận hành và theo dõi hoạt động của hệ thống như start, stop, thay đổi tham số chương trình, hiển thị giá trị, trạng thái các biến, quản lý lỗi, theo dõi và thực hiện các thao tác cần thiết tác động lên hệ thống...

Các nhà sản xuất PLC thường cung cấp các thiết bị giao diện phù hợp với chuẩn của hãng. Ví dụ, Siemens có các giao diện OP, hãng AB có giao diện PanelView,

Omron có giao diện NT... Tuy nhiên, có một số hãng chuyên sản xuất các giao diện và phần mềm có kết nối với hầu hết các họ PLC (PROFACE của hãng DIGITAL).

Việc kết nối các PLC với nhau tạo thành mạng PLC để thực hiện điều khiển một quá trình công nghệ cũng được thực hiện thông qua cổng nối tiếp. Các PLC trong mạng thực hiện trao đổi dữ liệu trong quá trình thực hiện chương trình thông qua một vùng nhớ có chung địa chỉ.

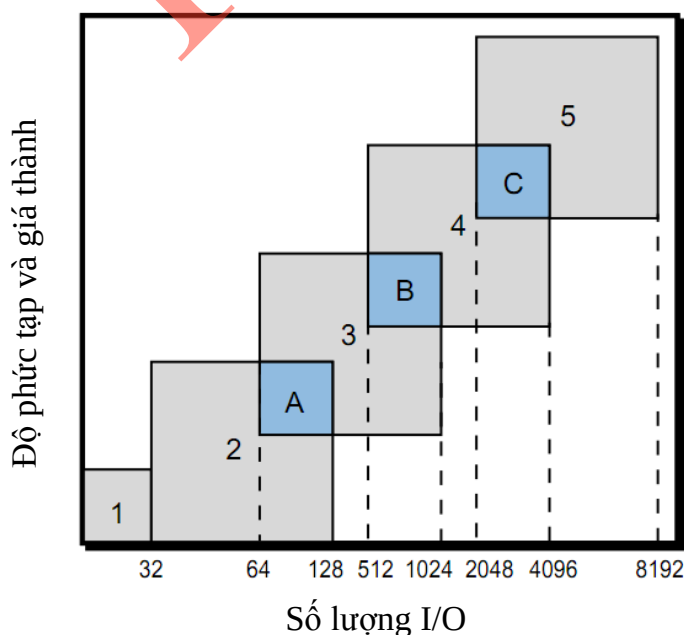
1.4.4. Các thiết bị ngoại vi khác.

Các thiết bị ngoại vi khác có thể kết nối với PLC qua cổng nối tiếp để thực hiện các ứng dụng cụ thể như đọc mã vạch, bộ xử lý video, các thiết bị kiểm soát...

Quá trình trao đổi thông tin qua cổng nối tiếp được thực hiện theo hai phương thức là HOST-LINK (chủ - tớ) và LINK-LINK.

1.5. Phân loại PLC

PLC có rất nhiều chủng loại, do nhiều nhà sản xuất cung cấp. Một số được sản xuất và tích hợp hệ thống sử dụng PLC do chính họ chế tạo, nó là một thành phần cấu thành hệ thống và được sử dụng trong phạm vi hẹp. Một số nhà sản xuất cung cấp PLC như là một sản phẩm đa dụng cho người thiết kế và tích hợp hệ thống. Nhà sản xuất cung cấp thiết bị, phần mềm, hỗ trợ kỹ thuật và đào tạo để người sử dụng có điều kiện ứng dụng các sản phẩm này vào hệ thống của mình. Một số hãng sản xuất PLC điển hình là Siemens (Đức), Allen-Bradley, GE-Fanuc (Mỹ), OMRON, Mitsubishi, Toshiba (Nhật)... Do PLC được sử dụng rộng rãi, từ các bài toán đơn giản đến phức tạp, nên PLC được chế tạo dưới nhiều loại khác nhau phù hợp với từng yêu cầu thực tế.



Hình 1.12. Phân loại PLC theo số lượng đầu I/O

Hình 1.12 minh họa một cách phân loại PLC, không có ranh giới cụ thể giữa hai loại gần nhau nhưng cách phân loại này thực tế lại rất hợp lý. Theo đó, PLC được chia ra làm năm loại là PLC siêu nhỏ (micro), nhỏ (small), vừa (medium), lớn (large) và rất lớn (very large).

- PLC loại siêu nhỏ: dùng cho các ứng dụng lên đến 32 thiết bị I/O.
- PLC loại nhỏ: dùng cho các ứng dụng có từ 32 đến 128 thiết bị I/O.
- PLC loại vừa: dùng cho các ứng dụng có từ 64 đến 1024 thiết bị I/O.
- PLC loại lớn: dùng cho các ứng dụng có từ 512 đến 4096 thiết bị I/O.
- PLC loại vừa: dùng cho các ứng dụng có từ 2048 đến 8192 thiết bị I/O.

A, B, C trong hình 1.12 là các vùng bị chồng chéo nhau, nó phản ánh tính kế thừa và làm tăng khả năng lựa chọn khi sử dụng PLC.

Việc phân loại sản phẩm có thể dựa theo rất nhiều tiêu chí, với mỗi tiêu chí khác nhau thì lại có cách phân loại khác nhau. Để tăng tính đơn giản cho người sử dụng, hiện nay người ta thường phân PLC làm ba loại là nhỏ, vừa và lớn.

Phân loại PLC dựa trên khả năng (tốc độ xử lý, dung lượng bộ nhớ, số lượng đầu I/O) được chia thành các loại là nhỏ, vừa và lớn.

- PLC loại nhỏ có nhiều tên gọi khác nhau tùy thuộc vào từng hãng (small, micro), có dung lượng bộ nhớ dưới 2Kbyte, quản lý số điểm I/O dưới 128, được sử dụng trong các ứng dụng đơn giản, yêu cầu ít điểm I/O.
- PLC loại vừa (medium) có bộ nhớ lên đến 32Kbyte, quản lý số điểm vào ra lên đến 2048. Cấu hình của hệ có thể sử dụng các module I/O, thực hiện điều khiển quá trình và xử lý thông tin.
- PLC loại lớn (large) là thiết bị phức tạp nhất, có bộ nhớ lên đến 2Mbyte và 16000 điểm I/O. PLC loại này có ứng dụng không hạn chế, từ điều khiển một quá trình công nghệ đến điều khiển một phân xưởng, nhà máy.

Dựa vào khả năng và kiểu dáng chế tạo thì PLC được phân loại thành PLC cỡ nhỏ, vừa và lớn.

- Các PLC cỡ nhỏ thường được chế tạo ở dạng cố định (compact, fixed). Với loại này, nguồn cung cấp, CPU và một số điểm I/O được chế tạo trên cùng một khối không thể tách rời. Ưu điểm của PLC loại này là giá thành thấp, nhỏ gọn, thích hợp cho các ứng dụng nhỏ. Số các điểm I/O trên PLC theo tỉ lệ 3:2 (3 vào, 2 ra). Khi cần thiết có thể sử dụng các module mở rộng (nhưng ít được sử dụng). Nhược điểm chính là tính mềm dẻo không cao, tốc độ xử lý chậm, bộ nhớ nhỏ, hạn chế số điểm I/O.
- Các PLC loại vừa và lớn được chế tạo ở dạng các module riêng biệt, có thể tháo lắp dễ dàng (modular). Các module cơ bản là nguồn, CPU, điểm I/O... Đây là cấu trúc tiêu chuẩn của PLC, đảm bảo cho PLC được sử dụng một cách mềm dẻo và người sử dụng có nhiều cơ hội lựa chọn cho cấu hình của mình. Các module được lắp vào các khe cắm (slot) trên bảng mạch bus.

Ứng dụng của PLC được chia thành ba nhóm chính là đơn nhiệm (single), đa nhiệm (multitask) và quản lý điều khiển (control manegment).

- Ứng dụng đơn nhiệm: chỉ sử dụng một PLC để điều khiển một quá trình kỹ thuật. Đó là một khối điều khiển độc lập, không có trao đổi thông tin với máy tính hoặc các PLC khác. Cấu hình của hệ có thể dùng PLC loại nhỏ, vừa hoặc lớn.
- Ứng dụng đa nhiệm: thường sử dụng PLC cỡ vừa để điều khiển một công đoạn của dây chuyền sản xuất hoặc điều khiển một vài quá trình kỹ thuật với số lượng điểm I/O thích hợp.
- Ứng dụng quản lý điều khiển: sử dụng PLC để điều khiển một số quá trình kỹ thuật khác nhau. PLC được sử dụng thường là cỡ lớn, với cấu hình của hệ là một mạng LAN điều khiển thống nhất, có sự trao đổi dữ liệu và thông tin giữa các thành phần của hệ. Trong đó, PLC đóng vai trò là bộ điều khiển, quản lý toàn bộ hoạt động của hệ là trạm chủ (master). Các PLC khác là các bộ điều khiển, đồng thời là thiết bị thu thập dữ liệu phục vụ cho công tác quản lý và theo dõi hệ thống trạm tớ (slave).

1.6. Các họ PLC thông dụng

1.6.1. Họ SIMATIC của SIEMENS (Đức)

Hãng SIEMENS sản xuất nhiều họ PLC khác nhau. Tuy nhiên, PLC họ SIMATIC được sử dụng phổ biến với hai thế hệ là S5 (cũ) và S7 (mới).



Hình 1.13. PLC của Siemens

PLC loại S5 là:

- Loại nhỏ: S5 90U, S5 95U
- Loại vừa: S5 100U với các CPU 100, 101, 102, 103, 104
- Loại lớn: S5 115, S5 135, S5 155 với các loại CPU 941, 942...

PLC loại S7 là:

- Loại nhỏ: S7 200 với các CPU 212, 222, 226...

- Loại vừa: S7 300 với các CPU 312, 315...
- Loại lớn: S7 400 với các CPU 412, 425...

1.6.2. Họ SYSMAC của OMRON (Nhật)

Hãng OMRON sản xuất nhiều họ PLC khác nhau như họ C, CV, CJ... Phần mềm lập trình có thể dùng Syswin, CX-Programmer. Sau đây là một số PLC loại C:

- Loại nhỏ: CPM1, CPM1A, CPM2A...
- Loại vừa: CQM1c CQM1H...
- Loại lớn: C200H, C1000H, C2000H, C2000HS...



(a)ZEN-10C



(b)CJ1M

Hình 1.14. PLC loại ZEN-10C của Omron

1.6.3. PLC của ALLEN BRADLEY (Mỹ)

- Loại nhỏ: Micrologix 500, Micrologix 1000
- Loại vừa: SLC 500 với các CPU 01, 02, 03, 04
- Loại lớn: PLC5 với các CPU 25, 41, 45, 55...

PLC loại nhỏ và vừa dùng phần mềm lập trình RSLOGIX 500, loại lớn dùng RSLOGIX5

1.6.4. PLC của Misubishi

Misubishi có các họ như Alpha, Fx, Fx0, Fx0N, Fx1N, Fx2N

- PLC loại ALPHA
- PLC loại FX0S
- PLC loại FX0N
- PLC loại FX1N
- PLC loại FN2N
- PLC loại FN2NC

Việc lựa chọn chủng loại PLC xuất phát từ yêu cầu của người dùng dựa vào một số đặc điểm sau:

- + Yêu cầu công nghệ và yêu cầu điều khiển. Điều này liên quan đến các tính năng kỹ thuật của PLC, nó có đáp ứng được các yêu cầu công nghệ và kỹ thuật (tốc độ xử lý, dung lượng bộ nhớ, quản lý I/O, khả năng mở rộng hệ thống...) đặt ra không?
- + Tính kinh tế và thời gian cung cấp thiết bị của nhà sản xuất.
- + Sự hỗ trợ và giúp đỡ của nhà cung cấp về các giải pháp kỹ thuật, phần mềm, công cụ...



(a) ALPHA



(b) FX0S



(c) FX0N



(d) FX1N



(e) FN2N

Hình 1.15. PLC họ Misubishi

1.7. Kết luận

Chương này giới thiệu cấu trúc cơ bản của một PLC, ưu/nhược điểm của PLC so với các cách điều khiển thông thường khác (như rơ le, mạch số hoặc máy tính) và ứng dụng của nó. Chương này cũng nêu khái quát quá trình hình thành và phát triển của bộ điều khiển logic khả trình PLC. Trọng tâm của chương là giới thiệu cấu tạo và nguyên lý hoạt động của một PLC như CPU (bộ vi xử lý, bộ nhớ, nguồn cung cấp, LED trạng thái, khe cắm thẻ nhớ...), các thiết bị I/O (BUS module và hệ thống BUS, module I/O). Ngoài ra, chương này còn đề cập đến các thiết bị ngoại vi của PLC như PG, PC, HMI...) và cách phân loại PLC.

BÀI TẬP CHƯƠNG I

Bài tập 1.1

Cấu trúc bên trong của một PLC?

Bài tập 1.2

Ưu điểm của PLC?

Bài tập 1.3

So sánh PLC với các cách điều khiển thông thường?

Bài tập 1.4

Trình bày về khối xử lý trung tâm (CPU) của PLC?

Bài tập 1.5

Trình bày module I/O của PLC?

Bài tập 1.6

Cách phân loại PLC theo số lượng đầu I/O?

Bài tập 1.7

Cách phân loại PLC theo khả năng (tốc độ xử lý, dung lượng bộ nhớ và số lượng đầu I/O)?

Bài tập 1.8

Ứng dụng của PLC?

Bài tập 1.9

Các thiết bị ngoại vi thường dùng với PLC?

Bài tập 1.10

Phân biệt các loại bộ nhớ trong PLC?

CHƯƠNG 2. CÁC HỌ PLC

2.1. PLC của hãng Siemens

Để tăng tính mềm dẻo trong các ứng dụng thực với phần lớn các đối tượng điều khiển có số tín hiệu đầu vào, đầu ra cũng như chủng loại tín hiệu I/O khác nhau mà các bộ điều khiển PLC được thiết kế không bị cứng hoá về cấu hình, chúng được chia nhỏ thành các module. Số các module được sử dụng nhiều hay ít tùy thuộc vào từng bài toán, song tối thiểu bao giờ cũng có module chính (module CPU, module nguồn). Các module còn lại là những module truyền nhận tín hiệu với các đối tượng điều khiển, chúng được gọi là các module mở rộng. Tất cả các module đều được gá trên một thanh Rack. Các PLC của Siemens có cấu tạo tương đối giống nhau, chỉ khác nhau về số lượng đầu I/O, tốc độ bộ vi xử lý của CPU.. Trong cuốn bài giảng này, tác giả giới thiệu về PLC cỡ trung của Siemens là S7-300.

2.1.1. Các module của PLC S7 -300

2.1.1.1. Module CPU

Đây là loại module có chứa bộ vi xử lý, hệ điều hành, bộ nhớ, Timer, Counter, cổng truyền thông,... và có thể có các cổng I/O số. Các cổng I/O tích hợp trên CPU gọi là cổng vào ra onboard.

Trong họ PLC S7-300, các module CPU có nhiều loại và được đặt tên theo bộ vi xử lý bên trong như CPU 312, CPU 314, CPU 316,... Những module cùng một bộ vi xử lý nhưng khác nhau số cổng I/O onboard cũng như các khối hàm đặc biệt thì được phân biệt bằng cụm chữ cái IFM (Integrated Function Module). Ví dụ như CPU 312IFM, CPU 314IFM,....

Ngoài ra, còn có loại module CPU có hai cổng truyền thông, trong đó cổng thứ hai dùng để nối mạng phân tán như mạng PROFIBUS (PROcess Field BUS). Loại này đi kèm với cụm từ DP (Distributed Port) trong tên gọi. Ví dụ module CPU315-DP.

2.1.1.2. Module mở rộng:

Các module mở rộng được thành 5 loại :

1. PS (Power Supply): module nguồn là module tạo ra nguồn có điện áp 24V cấp nguồn cho các module khác. Có ba loại nguồn là 2A, 5A và 10A.

2. SM (Signal Module): Module mở rộng I/O, bao gồm :

- DI (Digital Input): module mở rộng cổng vào số. Số các cổng vào số mở rộng có thể là 8, 16 hoặc 32 tùy thuộc vào từng loại module.
- DO (Digital Output): module mở rộng cổng ra số. Số các cổng ra số mở rộng có thể là 8, 16 hoặc 32 tùy thuộc vào từng loại module.

- DI/DO (Digital Input/Digital Output): module mở rộng cổng I/O số. Số các cổng I/O số mở rộng có thể là 8 vào/8 ra hoặc 16 vào/16 hoặc 32 ra tùy thuộc vào từng loại module.
- AI (Analog Input): module mở rộng cổng vào tương tự. Bản chất đây là những bộ chuyển đổi tương tự/số (ADC). Số các cổng vào tương tự có thể là 2, 4 hoặc 8 tùy thuộc vào từng loại module; số bit có thể là 8, 10, 12, 14, 16 tùy thuộc vào từng loại module.
- AO (Analog Output): module mở rộng cổng ra tương tự. Bản chất đây là những bộ chuyển đổi số/tương tự (DAC). Số các cổng ra tương tự có thể là 2 hoặc 4 tùy thuộc vào từng loại module.
- AI/AO (Analog Input/ Analog Output): module mở rộng cổng I/O tương tự. Số các cổng I/O tương tự có thể là 4 vào/2 ra hoặc 4 vào/4 ra tùy thuộc vào từng loại module.

3. IM (Interface Module): Module kết nối.

Đây là loại module dùng để kết nối từng nhóm các module mở rộng thành một khối và được quản lý bởi một module CPU. Thông thường các module mở rộng được gá liền nhau trên một thanh rack. Mỗi thanh rack chỉ có thể gá được nhiều nhất 8 module mở rộng (không kể module CPU và module nguồn). Một module CPU có thể làm việc nhiều nhất với 4 thanh rack và các rack này phải được nối với nhau bằng module IM.

4. FM (Function Module): Module có chức năng điều khiển riêng như: module điều khiển động cơ bước, module điều khiển động cơ servo, module PID,...

5. CP (Communication Processor): Module truyền thông giữa PLC với PLC hay giữa PLC với PC.

2.1.2. Kiểu dữ liệu trong PLC S7-300

1. BOOL: với dung lượng 1 bit và có giá trị là 0 hoặc 1 (đúng hoặc sai).
2. BYTE: 8 bit, dùng để biểu diễn số nguyên dương trong khoảng từ 0 đến 255 hoặc mã ASCII của một ký tự.
3. WORD: 2 byte, dùng để biểu diễn số nguyên dương trong khoảng từ 0 đến 65535.
4. INT: 2 byte, dùng để biểu diễn số nguyên trong khoảng từ -32768 (2^{15}) đến 32767 ($2^{15}-1$).
5. DINT: 4 byte, dùng để biểu diễn số nguyên trong khoảng từ (-2^{31}) đến $(2^{31}-1)$.
6. REAL: 4 byte, dùng để biểu diễn số thực dấu phẩy động.
7. S5T (hay S5TIME), dùng để tạo khoảng thời gian, được tính theo giờ/phút/giây/mili giây.

8. TOD: biểu diễn giá trị thời gian tính theo giờ/phút/giây.
9. DATE: biểu diễn giá trị thời gian tính theo năm/tháng/ngày.
10. CHAR: biểu diễn một hoặc nhiều ký tự (nhiều nhất là 4 ký tự).

2.1.3. Tổ chức bộ nhớ CPU.

- Vùng nhớ chứa các thanh ghi: ACCU1, ACCU2, AR1, AR2...
- Load memory: là vùng nhớ chứa chương trình ứng dụng do người sử dụng viết, bao gồm tất cả các khối chương trình ứng dụng OB, FC, FB, các khối chương trình trong thư viện hệ thống được sử dụng (SFC, SFB) và các khối dữ liệu DB. Vùng nhớ này được tạo bởi một phần bộ nhớ RAM của CPU và EEPROM (nếu có). Khi xoá bộ nhớ (MRES), toàn bộ các khối chương trình và khối dữ liệu nằm trong RAM sẽ bị xoá. Khi chương trình hay khối dữ liệu được tải về từ thiết bị lập trình (PG, máy tính) vào CPU, chúng sẽ được ghi lên vùng RAM của vùng nhớ Load memory.
- Work memory: là vùng nhớ chứa các khối DB đang được mở, khối chương trình (OB, FC, FB, SFC, hoặc SFB) đang được CPU thực hiện và phần bộ nhớ cấp phát cho những tham số hình thức để các khối chương trình này trao đổi tham trị với hệ điều hành và với các khối chương trình khác (local block). Tại một thời điểm nhất định, vùng Work memory chỉ chứa một khối chương trình. Sau khi khối chương trình đó thực hiện xong thì hệ điều hành sẽ xoá khỏi Work memory và nạp vào đó khối chương trình kế tiếp thực hiện.
- System memory: là vùng nhớ chứa các bộ đệm I/O số (Q, I), các biến cờ (M), thanh ghi C-Word, PV, T-bit của Timer, thanh ghi C-Word, PV, C-bit của Counter. Việc truy cập, sửa lỗi dữ liệu những ô nhớ này được phân chia hoặc bởi hệ điều hành của CPU hoặc do chương trình ứng dụng.

Có thể thấy rằng trong các vùng nhớ được trình bày ở trên không có vùng nhớ nào được dùng làm bộ đệm cho các cổng I/O tương tự. Nói cách khác các cổng I/O tương tự không có bộ đệm, và như vậy mỗi lệnh truy nhập module tương tự (đọc hoặc gửi giá trị) đều có tác dụng trực tiếp tới các cổng vật lý của module.

Bảng 2.1. Vùng địa chỉ trong PLC S7-300

Ý nghĩa	Lệnh	Kích thước tối đa (phụ thuộc vào từng loại CPU)
Process Image Input (I) Bộ đệm vào số	I	0.0 ÷ 127.7
	IB	0 ÷ 127
	IW	0 ÷ 126
	ID	0 ÷ 124
Process Image Output (Q) Bộ đệm ra số	Q	0.0 ÷ 127.7
	QB	0 ÷ 127
	QW	0 ÷ 126

	QD	0 ÷ 124
Bit memory (M) Vùng nhớ cò	M	0.0 ÷ 255.7
	MB	0 ÷ 255
	MW	0 ÷ 254
	MD	0 ÷ 252
Timer (T) Bộ định thời	T0 ÷ T255	
Counter (C) Bộ đếm	C0 ÷ C255	
Data Block (DB) Khối dữ liệu share	DBX	0.0 ÷ 65535.7
	DBB	0 ÷ 65535
	DBW	0 ÷ 65534
	DBD	0 ÷ 65532
Data Block (DI) Khối dữ liệu instance	DIX	0.0 ÷ 65535.7
	DIB	0 ÷ 65535
	DIW	0 ÷ 65534
	DID	0 ÷ 65532
Local Block (L) Miền nhớ cục bộ cho các tham số hình thức	L	0.0 ÷ 65535.7
	LB	0 ÷ 65535
	LW	0 ÷ 65534
	LD	0 ÷ 65532
Peripheral Input (PI) Đầu vào ngoại vi	PIB	0 ÷ 65535
	PIW	0 ÷ 65534
	PID	0 ÷ 65532
Peripheral Output (PQ) Đầu ra ngoại vi	PQB	0 ÷ 65535
	PQW	0 ÷ 65534
	PQD	0 ÷ 65532

Trừ phần bộ nhớ EEPROM thuộc vùng Load memory và một phần RAM duy trì đặc biệt (non-volatile) dùng để lưu giữ tham số cấu hình trạm PLC như địa chỉ trạm (MPI address), tên các module mở rộng, tất cả các phần bộ nhớ còn lại ở chế độ mặc định không có khả năng tự nhớ (non-retentive). Khi mất nguồn nuôi hoặc khi thực hiện công việc xóa bộ nhớ (MRES), toàn bộ nội dung của phần bộ nhớ non-retentive sẽ bị mất.

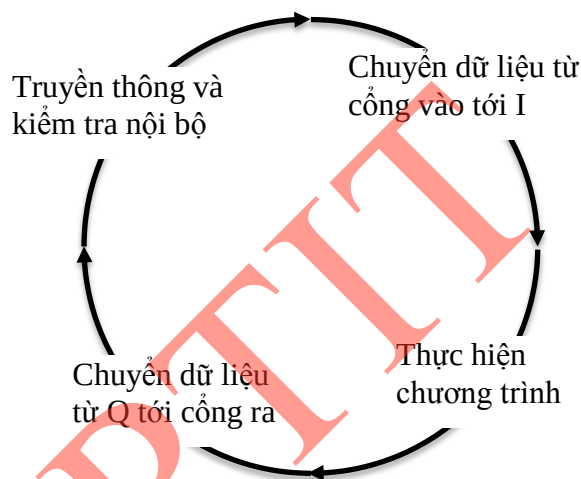
2.1.4. Vòng quét chương trình của PLC.

PLC hoạt động theo nguyên tắc quét vòng (scan), mỗi vòng quét gồm ba giai đoạn cơ bản:

Giai đoạn 1: PLC đọc trạng thái tín hiệu ở các module vào, gửi vào vùng ảnh đầu vào để làm dữ liệu thực hiện chương trình.

Giai đoạn 2: Thực hiện chương trình trong bộ nhớ. Kết quả thực hiện chương trình là dữ liệu và các quyết định được lưu giữ trong bộ nhớ để phục vụ vòng quét sau hoặc gửi đến module ra.

Giai đoạn 3: PLC gửi dữ liệu đến vùng ảnh đầu ra và biến đổi thành tín hiệu điều khiển cơ cấu chấp hành nối với module ra. Khi đó, một vòng quét kết thúc và bắt đầu vòng quét mới. Trong từng vòng quét, chương trình được thực hiện từ lệnh đầu tiên đến lệnh kết thúc của khối OB1 (Block end). Quá trình này sẽ diễn ra liên tục.



Hình 2.1. Vòng quét CPU

Quá trình đọc tín hiệu vào và gửi tín hiệu ra được gọi là quá trình quét I/O. Quá trình thực hiện chương trình gọi là quét chương trình.

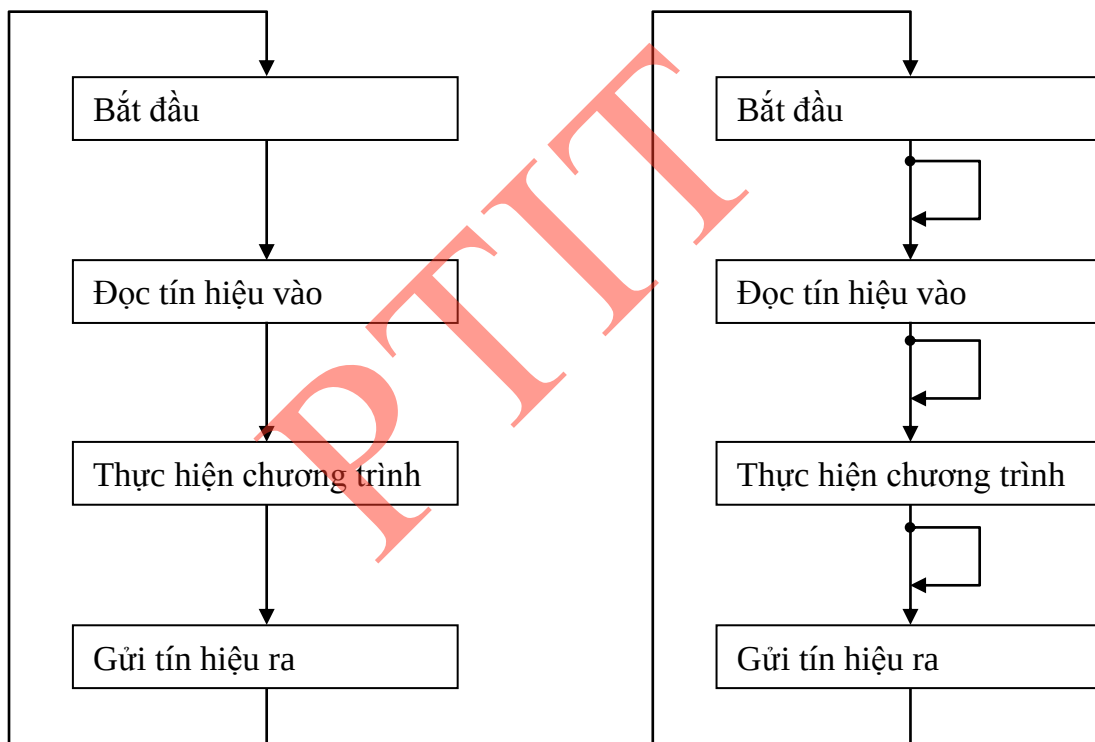
Thời gian để thực hiện một vòng quét gọi là chu kỳ quét. Chu kỳ quét ảnh hưởng đến tốc độ xử lý và năng xử lý thời gian thực của PLC. Vì vậy, PLC chỉ được sử dụng trong các bài toán điều khiển khi chu kỳ quét của PLC đủ nhỏ so với hằng số thời gian của hệ thống. Khi đó, có thể chấp nhận xử lý đồng thời (thời gian thực) thay thế bằng xử lý tuần tự. Ví dụ, tín hiệu vào thay đổi trạng thái hai lần trong một vòng quét thì PLC không thể phát hiện được (đáp ứng của PLC so với sự thay đổi đầu vào không còn chính xác nữa).

Chu kỳ quét phụ thuộc vào các yếu tố như tốc độ bộ vi xử lý của CPU, độ dài chương trình, số lượng đầu I/O, thời gian xử lý ngắt, thời gian chuyển đổi song song – nối tiếp của hệ I/O phân tán... Tuy nhiên, với một hệ thống cụ thể thì các nhân tố là cố định, trừ tốc độ bộ vi xử lý. Vì vậy, để giảm chu kỳ quét thì phải chọn CPU có tốc độ xử lý cao (đây chính là mấu chốt khi chọn thiết bị cho hệ điều khiển). Nguyên tắc quét vòng của PLC hạn chế khả năng xử lý tức thời của PLC, vậy nên PLC hiện đại được

trang bị và tăng cường các tính năng xử lý ngắt và do đó, ranh giới giữa PC và PLC ngày càng bị thu hẹp.

Xử lý vòng quét đầu tiên là vấn đề cần được quan tâm khi nghiên cứu nguyên tắc hoạt động của PLC. Ở vòng quét đầu tiên, các dữ liệu đều chưa sẵn sàng, hệ đang ở quá trình khởi tạo. Đối với hệ mà quá trình khởi tạo không ảnh hưởng đến quá trình điều khiển thì điều này có thể bỏ qua. Vì vậy, PLC đều cung cấp cờ trạng thái có giá trị bằng 1 ở vòng quét đầu tiên và bằng 0 ở các vòng quét tiếp theo. Người sử dụng có thể dùng cờ này để tiến hành khởi tạo và thiết lập các điều kiện ban đầu cho hệ thống.

PLC thực hiện chương trình theo chu trình lặp. Mỗi vòng lặp được gọi là vòng quét (scan). Mỗi vòng quét được bắt đầu bằng giai đoạn chuyển dữ liệu từ các cổng vào số tới vùng bộ đệm ảo I, tiếp theo là giai đoạn thực hiện chương trình. Sau giai đoạn thực hiện chương trình là giai đoạn chuyển các nội dung của bộ đệm ảo Q tới các cổng ra số. Vòng quét được kết thúc bằng giai đoạn truyền thông nội bộ và kiểm tra lỗi.



Hình 2.2. Sơ đồ vòng quét thực hiện chương trình của PLC

Như vậy, giữa việc đọc dữ liệu từ đối tượng để xử lý, tính toán và việc gửi tín hiệu điều khiển tới đối tượng có một khoảng thời gian trễ đúng bằng thời gian vòng quét. Nói cách khác, thời gian vòng quét quyết định tính thời gian thực của chương trình điều khiển trong PLC. Thời gian vòng quét càng ngắn, tính thời gian thực của chương trình càng cao.

Nếu sử dụng các khối chương trình đặc biệt có chế độ ngắt, ví dụ như khối OB40, OB80,... Chương trình của các khối đó sẽ được thực hiện trong vòng quét khi xuất hiện tín hiệu báo ngắt cùng chủng loại. Các khối chương trình này có thể được thực hiện tại mọi điểm trong vòng quét chứ không bị gò ép là phải ở trong giai đoạn thực hiện chương trình. Chẳng hạn nếu một tín hiệu

báo ngắt xuất hiện khi PLC đang ở giai đoạn truyền thông và kiểm tra nội bộ, PLC sẽ tạm dừng công việc truyền thông, kiểm tra, để thực hiện khối chương trình tương ứng với khối tín hiệu báo ngắt đó. Với hình thức xử lý tín hiệu ngắt như vậy, thời gian vòng quét sẽ càng lớn khi càng có nhiều tín hiệu ngắt xuất hiện trong vòng quét. Do đó, để nâng cao tính thời gian thực cho chương trình điều khiển tuyệt đối không nên viết chương trình xử lý ngắt quá dài hoặc quá lạm dụng việc sử dụng chế độ ngắt trong chương trình điều khiển.

Tại thời điểm thực hiện lệnh I/O, thông thường lệnh không làm việc trực tiếp với cổng I/O mà chỉ thông qua bộ đệm ảo của cổng trong vùng nhớ tham số. Việc truyền thông giữa bộ đệm ảo với ngoại vi trong các giai đoạn 1 và 3 do hệ điều hành CPU quản lý. Ở một số module CPU, khi gặp lệnh I/O ngay lập tức, hệ thống sẽ cho dừng mọi công việc khác, ngay cả chương trình xử lý ngắt, để thực hiện lệnh trực tiếp với cổng I/O.

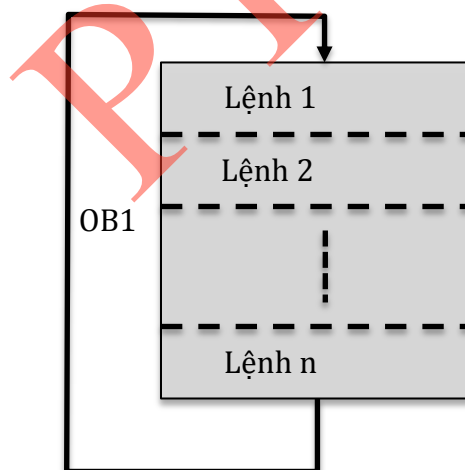
2.1.5. Cấu trúc chương trình.

Chương trình cho S7-300 được lưu trong bộ nhớ của PLC ở vùng dành riêng cho chương trình. Có thể được lập trình PLC với hai dạng cấu trúc khác nhau:

2.1.5.1. Lập trình tuyến tính

Toàn bộ chương trình điều khiển nằm trong một khối trong bộ nhớ. Loại lập trình cấu trúc chỉ thích hợp cho những bài toán tự động nhỏ, không phức tạp.

Khối được chọn OB1, là khối mà PLC luôn luôn quét và thực hiện các lệnh trong nó thường xuyên, từ lệnh đầu tiên đến lệnh cuối cùng và quay lại lệnh đầu tiên.



Hình 2.3. Lập trình tuyến tính

2.1.5.2. Lập trình cấu trúc.

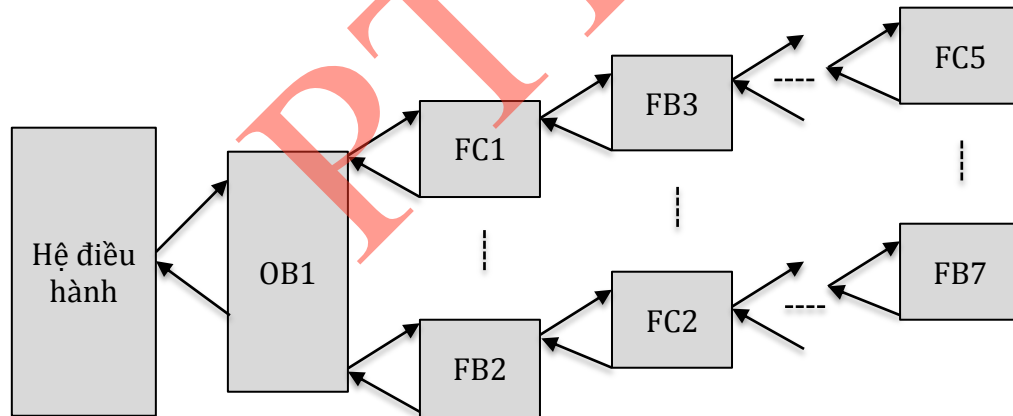
Chương trình được chia thành những phần nhỏ với từng nhiệm vụ riêng biệt và các phần này nằm trong những khối chương trình khác nhau. Loại lập trình có cấu trúc phù hợp với những bài toán điều khiển nhiều nhiệm vụ và phức tạp. Các khối cơ bản:

- Khối OB (Organization Block): khối tổ chức và quản lý chương trình điều khiển. Có nhiều loại khối OB với những chức năng khác nhau, chúng được phân biệt với nhau bằng số

nguyên theo sau nhóm ký tự OB, ví dụ như OB1, OB35, OB80...

- Khối FC (Program Block): khối chương trình với những chức năng riêng biệt giống như một chương trình con (chương trình có biến hình thức để trao đổi với chương trình đã gọi nó). Một chương trình ứng dụng có thể có nhiều khối FC và các khối FC này được phân biệt với nhau bằng số nguyên theo sau nhóm ký tự FC, ví dụ như FC1, FC2, ...
- Khối FB (Function Block): là khối FC đặt biệt, được tổ chức giống như một hàm, có khả năng trao đổi một lượng dữ liệu lớn với các khối chương trình khác. Dữ liệu này phải được tổ chức thành khối dữ liệu riêng được gọi là Data Block. Một chương trình ứng dụng có thể có nhiều khối FB và các khối FB này được phân biệt với nhau bằng số nguyên theo sau nhóm ký tự FB, ví dụ như FB1, FB2, ...
- Khối DB (Data Block): khối dữ liệu cần thiết để thực hiện chương trình. Các tham số của khối do người sử dụng tự đặt. Một chương trình ứng dụng có thể có nhiều khối DB và các khối DB này được phân biệt với nhau bằng số nguyên theo sau nhóm ký tự DB. Chẳng hạn như DB1, DB2,...

Chương trình trong các khối được liên kết với nhau bằng các lệnh gọi khối và chuyển khối. Các chương trình con được phép gọi lồng nhau, tức từ một chương trình con này gọi một chương trình con khác và từ chương trình con được gọi lại gọi một chương trình con thứ ba. Số các lệnh gọi lồng nhau tối đa phụ thuộc vào từng loại module CPU. Nếu số lần gọi lồng nhau vượt quá mức cho phép thì CPU sẽ tự động chuyển sang chế độ STOP và đặt cờ báo lỗi.



Hình 2.4. Lập trình có cấu trúc

2.1.5.3. Các khối OB đặc biệt.

1) OB10 (*Time of Day Interrupt*): Chương trình trong khối OB10 sẽ được thực hiện khi giá trị thời gian của đồng hồ thời gian thực nằm trong một khoảng thời gian đã được quy định. Việc quy định khoảng thời gian hay số lần gọi OB10 được thực hiện nhờ chương trình hệ thống SFC28 hay trong bảng tham số của module CPU nhờ phần mềm STEP 7.

2) OB20 (*Time Relay Interrupt*): Chương trình trong khối OB20 sẽ được thực hiện sau một khoảng thời gian trễ đặt trước kể từ khi gọi chương trình hệ thống SFC32 để đặt thời gian trễ.

3) OB35 (*Cyclic Interrupt*): Chương trình trong khối OB35 sẽ được thực hiện cách

đều nhau một khoảng thời gian cố định. Mặc định, khoảng thời gian này là 100ms, nhưng ta có thể thay đổi nhờ STEP 7.

4) OB40 (*Hardware Interrupt*): Chương trình trong khối OB40 sẽ được thực hiện khi xuất hiện một tín hiệu báo ngắt từ ngoại vi đưa vào CPU thông qua các cổng onboard đặc biệt, hoặc thông qua các module SM, CP, FM.

5) OB80 (*Cycle Time Fault*): Chương trình trong khối OB80 sẽ được thực hiện khi thời gian vòng quét (scan time) vượt quá khoảng thời gian cực đại đã qui định hoặc khi có một tín hiệu ngắt gọi một khối OB nào đó mà khối OB này chưa kết thúc ở lần gọi trước. Thời gian quét mặc định là 150ms.

6) OB81 (*Power Supply Fault*): Chương trình trong khối OB81 sẽ được thực hiện khi thấy có xuất hiện lỗi về bộ nguồn.

7) OB82 (*Diagnostic Interrupt*): Chương trình trong khối OB82 sẽ được thực hiện khi có sự cố từ các module mở rộng I/O. Các module này phải là các module có khả năng tự kiểm tra (diagnostic capabilities).

8) OB87 (*Communication Fault*): Chương trình trong khối OB87 sẽ được thực hiện khi xuất hiện lỗi trong truyền thông.

9) OB100 (*Start Up Information*): Chương trình trong khối OB100 sẽ được thực hiện một lần khi CPU chuyển từ trạng thái STOP sang RUN.

10) OB101 (*Cold Start Up Information* - chỉ với S7-400): Chương trình trong khối OB101 sẽ được thực hiện một lần khi công tắt nguồn chuyển từ trạng thái OFF sang ON.

11) OB121 (*Synchronous Error*): Chương trình trong khối OB121 sẽ được thực hiện khi CPU phát hiện thấy lỗi logic khi chương trình đổi sai kiểu dữ liệu hay lỗi truy nhập khối DB, FC, FB không có trong bộ nhớ.

12) OB122 (*Synchronous Error*): Chương trình trong khối OB122 sẽ được thực hiện khi có lỗi truy nhập module trong chương trình.

2.1.6. Ngôn ngữ lập trình

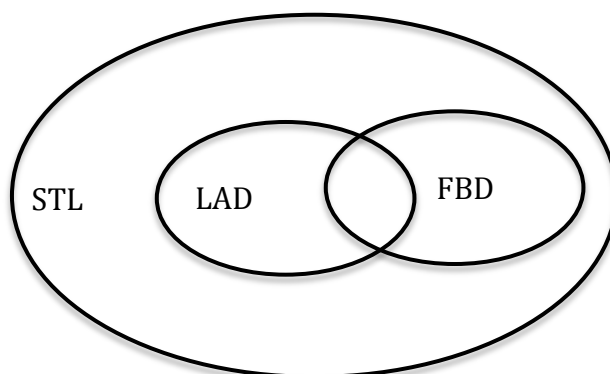
Có ba ngôn ngữ lập trình cơ bản sau:

- Ngôn ngữ lập trình liệt kê lệnh STL (*Statement List*). Đây là dạng ngôn ngữ lập trình thông thường của máy tính. Một chương trình được hoàn chỉnh bởi sự ghép nối của nhiều câu lệnh theo một thuật toán nhất định, mỗi lệnh chiếm một hàng và có cấu trúc chung "tên lệnh" + "toán hạng".
- Ngôn ngữ lập trình LAD (*Ladder Logic*). Đây là dạng ngôn ngữ đồ họa, thích hợp với những người lập trình quen với việc thiết kế mạch điều khiển logic.
- Ngôn ngữ lập trình FBD (*Function Block Diagram*). Đây cũng là dạng ngôn ngữ đồ họa, thích hợp cho những người quen thiết kế mạch điều khiển số.

Trong PLC có nhiều ngôn ngữ lập trình nhằm phục vụ cho các đối tượng sử dụng khác nhau. Tuy nhiên, một chương trình viết trên ngôn ngữ LAD hay FBD có thể chuyển sang

dạng STL, nhưng ngược lại thì không vì có nhiều lệnh có trong STL mà LAD hay FBD không có.

Hình 2.5 minh họa mối liên hệ giữa ba phương pháp lập trình trên, từ đó ta thấy STL là ngôn ngữ lập trình mạnh nhất.



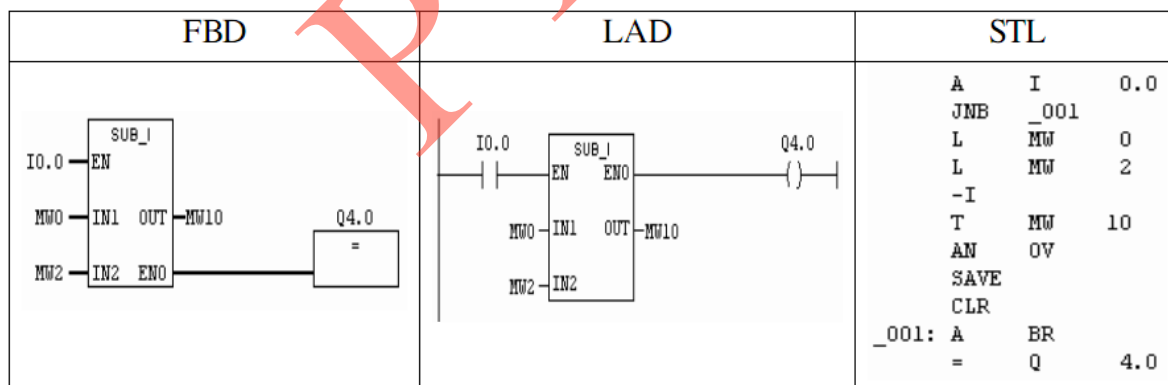
Hình 2.5. Mối liên hệ giữa ba phương pháp lập trình STL, FBD và LAD

Ví dụ: Thực hiện việc trừ hai số nguyên 16 bit được viết theo ba ngôn ngữ là FBD, LAD và STL. Dữ liệu vào và ra:

EN: BOOL IN1: INT
IN2: IN OUT: INT ENO: BOOL

Khi tín hiệu vào I0.0 = 1, đầu ra Q4.0 = 1 thì hàm sẽ thực hiện việc trừ hai số nguyên 16 bit MW0 và MW2, kết quả được cất vào MW10.

Khi tín hiệu vào I0.0 = 0, đầu ra Q4.0 = 0 thì hàm sẽ không thực hiện chức năng này.



Hình 2.6. Khối thực hiện chức năng trừ hai số nguyên 16 bit

2.2. PLC của hãng Omron.

Các bộ điều khiển PLC của Omron rất đa dạng, gồm các loại CPM1A, CPM2A, CPM2C, CQM1. PLC được tạo thành từ những module rời rạc kết nối lại với nhau, cho phép mở rộng dung lượng bộ nhớ và số lối I/O. Vì vậy chúng được sử dụng rất linh hoạt và đa dạng trên thực tế.

Ngoài ra, Omron còn sản xuất các bộ PLC có cấu trúc cố định, chúng chỉ được dùng cho các công việc đặc biệt nên không đòi hỏi tính linh hoạt cao.

Các bộ PLC đều có cấu trúc gồm:

- + Bộ nguồn.
- + CPU.
- + Các port I/O.
- + Các module I/O đặc biệt.

Bảng 2.2. các loại PLC CPM2A của Omron

Tên	Module	Số lối I/O	Nguồn cung cấp
CPU có lối ra dùng rơ le	CPM2A-20CDR-A	20	AC
	CPM2A-20CDR-D	20	DC
	CPM2A-30CDR-A	30	AC
	CPM2A-30CDR-D	30	DC
	CPM2A-40CDR-A	40	AC
	CPM2A-40CDR-D	40	DC
	CPM2A-60CDR-A	60	AC
	CPM2A-60CDR-D	60	DC
CPU có lối ra dùng transistor	CPM2A-20CDT-D	20 lối ra mức thấp	DC
	CPM2A-20CDT1-D	20 lối ra mức cao	DC
	CPM2A-30CDT-D	30 lối ra mức thấp	DC
	CPM2A-30CDT1-D	30 lối ra mức cao	DC
	CPM2A-40CDT-D	40 lối ra mức thấp	DC
	CPM2A-40CDT1-D	40 lối ra mức cao	DC
	CPM2A-60CDT-D	60 lối ra mức thấp	DC
	CPM2A-60CDT1-D	60 lối ra mức cao	DC

Để có được một PLC hoàn chỉnh thì ta phải lắp ráp các module này lại với nhau. Việc này được thực hiện khá đơn giản và có thể dễ dàng thay đổi cấu trúc PLC.

Bảng 2.2 liệt kê các loại PLC CPM2A của Omron.

* PLC CPM1A của Omron

- Kích thước nhỏ gọn.
- Có thể mở rộng lên tới 160 I/O.
- Nhiều loại CPU: 10, 20, 30, 40 I/O.
- Lối ra phát xung 2kHz, lối vào đếm tốc độ cao: 5kHz 1 pha, 2.5kHz 2 pha.
- Khối mở rộng 8 I/O, 20 I/O hoặc 40 I/O. Các module A/D, D/A, nhiệt độ, mạng Profibus, DeviceNet, Compobus-S.
- Phần mềm lập trình CX-Programmer V3.2 miễn phí.

- Có thể gắn thêm tối đa 3 khối mở rộng vào mỗi CPU 30 hoặc 40 I/O. Các CPU 10 và 20 I/O không hỗ trợ mở rộng thêm các khối I/O.

- Model thông dụng:

- CPM1A-20CDR-A-V1 loại 20 I/O, nguồn AC100-240, đầu ra role.
- CPM1A-30CDR-A-V1 loại 30 I/O, nguồn AC100-240, đầu ra role.
- CPM1A-40CDR-A-V1 loại 40 I/O, nguồn AC100-240, đầu ra role.

* PLC CPM2A của Omron

- Kích thước nhỏ gọn, mở rộng lên tới 180 I/O
- Lỗi ra phát xung 10kHz.
- Lỗi vào đếm tốc độ cao: 20kHz 1pha, 5kHz 2 pha.
- Có sẵn cổng kết nối RS-232, đồng hồ thời gian thực.
- Nhiều loại CPU: 20, 30, 40, 60 I/O.
- Khối mở rộng 8I/O, 20 I/O, hoặc 40 I/O. Các Module AD, DA, nhiệt độ, mạng Profibus, DeviceNet, Compobus-S.
- Phần mềm lập trình CX-Programmer V3.2 miễn phí.
- Có thể gắn thêm tối đa 3 khối mở rộng vào mỗi CPU 30 hoặc 40 I/O. Các CPU 10 và 20 I/O không hỗ trợ mở rộng thêm các khối I/O.

- Model thông dụng:

- CPM2A-20CDR-A loại 20 I/O, nguồn AC 100-240V, đầu ra role.
- CPM2A-30CDR-A loại 30 I/O, nguồn AC 100-240V, đầu ra role.
- CPM2A-40CDR-A loại 40 I/O, nguồn AC100-240, đầu ra role.



(a) CPM 1A



(b) CPM 2A

Hình 2.7. PLC của Omron

* Các thành phần của CPU:

1. Nguồn cung cấp: Tùy theo loại CPU mà dùng nguồn AC từ 120 đến 240 hoặc nguồn DC 24V.
2. Chân nối đất bảo vệ (đối với CPU loại dùng nguồn AC): để đảm bảo an toàn cho người sử dụng.
3. Nguồn cung cấp lỗi vào: 24V DC được dùng để cung cấp điện áp cho các thiết bị đầu vào

(đối với loại CPU dùng nguồn AC).

4. Các lối vào: Liên kết CPU với các thiết bị đầu vào.
5. Các lối ra: Liên kết CPU với các thiết bị đầu ra.
6. Đèn báo các chế độ làm việc của CPU: Cho biết chế độ làm việc hiện tại của PLC.

Bảng 2.3. Các chế độ đèn báo trong PLC của hãng Omron

Đèn báo	Trạng thái	Ý nghĩa
PWR (xanh)	ON	PLC đã được cấp nguồn
	OFF	PLC chưa được cấp nguồn
RUN (xanh)	ON	PLC đang hoạt động ở chế độ RUN hoặc MONITOR
	OFF	PLC đang ở chế độ PROGRAM hoặc bị lỗi
COMM (vàng)	Flashing	Dữ liệu đang được chuyển vào CPU thông qua cổng ngoại vi hoặc RS 232C

7. Đèn báo trạng thái lỗi vào: Khi một trong các lối vào ở trạng thái ON thì đèn báo tương ứng sẽ sáng. Khi sử dụng Counter tốc độ cao thì các đèn báo lỗi vào sẽ không sáng nếu tần số xung sáng quá nhanh.

8. Đèn báo trạng thái lỗi ra: Các đèn báo trạng thái lỗi ra sẽ sáng khi các lối ra ở trạng thái ON.

9. Cổng điều khiển tín hiệu analog: Được sử dụng khi tín hiệu vào hoặc ra là tín hiệu analog, được lưu giữ vào vùng nhớ IR250 và IR251.

10. Cổng giao tiếp với thiết bị ngoại vi: Liên kết PLC với thiết bị lập trình, máy chủ, thiết bị lập trình cầm tay.

11. Cổng giao tiếp RS 232C: Liên kết PLC với thiết bị lập trình (trừ thiết bị lập trình cầm tay và máy tính chủ).

12. Communication switch: Là công tắc, chọn để sử dụng một trong hai cổng ngoại vi hoặc RS 232C để liên kết với thiết bị lập trình.

13. Ac quy

14. Phần mở rộng: Kết nối CPU và PLC với khối mở rộng I/O hoặc khối mở rộng nói chung (I/O tương tự, cảm biến nhiệt độ...). Phần mở rộng bao gồm:

- + Đầu nối lối vào: Liên kết CPU với các thiết bị lối vào.
- + Đầu nối lối ra: Liên kết CPU với các thiết bị lối ra.
- + Các đèn báo hiển thị lối ra
- + Các đèn báo hiển thị lối vào.
- + Cáp nối phần mở rộng với CPU.

* Các thành phần của module I/O tương tự

- Module I/O tương tự thực hiện việc chuyển đổi tín hiệu tương tự sang số hoặc ngược lại để thực hiện giao tiếp giữa CPU với các thiết bị tương tự như máy phát sóng cảm biến, các dụng cụ đo và các thiết bị điều khiển khác.

- Module I/O tương tự có khoảng thay đổi tín hiệu điện áp từ $0 \div 10V$ hoặc từ $0 \div 5V$ (đối với đầu vào tương tự) và từ $-10 \div 10$ (đối với đầu ra tương tự). Một CPU có thể kết nối với ba module I/O tương tự (hai vào tương tự và một ra tương tự).

- Dữ liệu đã biến đổi được lưu trong vùng phân bố “word” của khối I/O tương tự và được sử dụng bởi lệnh đọc nội dung của “word” lối vào.

- Một chức năng khác của nó là xử lý giá trị trung bình sao cho tất cả dữ liệu ở lối ra ổn định. Nó còn có khả năng phát hiện dây dẫn bị đứt khi lối vào được đặt trong khoảng từ $4 \div 20mA$ hoặc từ $1 \div 5V$.

- Khối mở rộng tương tự gồm có:

+ Các đầu nối của khối I/O tương tự: Kết nối với các thiết bị tương tự vào hoặc ra.

+ Cáp kết nối của phần mở rộng: Kết nối khối I/O tương tự với cổng mở rộng của CPU hoặc của khối mở rộng khác.

+ Cổng mở rộng: Kết nối cổng mở rộng của khối I/O tương tự với các khối mở rộng khác (khối I/O tương tự, sensor nhiệt độ, khối kết nối I/O). Một CPU chỉ có thể kết nối được tối đa với ba khối mở rộng.

2.3. PLC của hãng Mitsubishi

2.3.1. PLC loại FX0

Đây là loại PLC kích thước nhỏ gọn, phù hợp các ứng dụng có số lượng đầu I/O nhỏ hơn 30. Với việc sử dụng bộ nhớ chương trình dạng EEPROM, nó cho phép dữ liệu chương trình được lưu lại trong bộ nhớ khi nguồn nuôi bị mất đột xuất. Dòng FX0 được tích hợp sẵn bên trong Counter tốc độ cao và các bộ tạo ngắt (rơ le trung gian), cho phép xử lý tốt một số ứng dụng phức tạp.

Nhược điểm của dòng FX0 là không có khả năng mở rộng số lượng cổng I/O, không có khả năng nối mạng, không có khả năng kết nối với các module chuyên dụng, thời gian thực hiện chương trình lâu.

2.3.2. PLC loại FX0S

Đây là loại PLC có kích thước siêu nhỏ, phù hợp với các ứng dụng với số lượng I/O nhỏ hơn 30, giảm chi phí lao động và kích cỡ panel điều khiển. Với việc sử dụng bộ nhớ chương trình bằng EEPROM cho phép dữ liệu chương trình được lưu lại trong bộ nhớ trong trường hợp mất nguồn đột xuất, giảm thiểu thời gian bảo hành sản phẩm. Dòng FX0 được tích hợp sẵn bên trong Counter tốc độ cao và các bộ tạo ngắt, cho phép xử lý tốt một số ứng dụng phức tạp.

Nhược điểm của dòng FX0 là không có khả năng mở rộng số lượng I/O, không có khả năng nối mạng, thời gian thực hiện chương trình lâu.



Hình 2.8. PLC FX0S, FX0N và FX1S của hãng Misubishi

2.3.3. PLC loại FX1S

FX1S có khả năng quản lý số lượng đầu I/O trong khoảng từ 10 đến 34. Cũng giống như FX0S, FX1S không có khả năng mở rộng hệ thống. Tuy nhiên, nó được tăng cường thêm một số tính năng đặc biệt như tăng hiệu năng tính toán, khả năng làm việc với đầu I/O tương tự thông qua các card chuyển đổi, cải thiện tính năng Counter tốc độ cao, tăng cường 6 đầu vào xử lý, trang bị thêm các chức năng truyền thông trong mạng (giới hạn tối đa là 8 trạm) hay giao tiếp với các bộ HMI đi kèm. FX1S thích hợp với các ứng dụng trong công nghiệp chế biến gỗ, đóng gói sản phẩm, điều khiển động cơ, máy móc hay các hệ thống quản lý môi trường.

2.3.4. PLC loại FX1N

FX1N thích hợp với các bài toán điều khiển có số lượng đầu I/O từ 14 đến 60. Tuy nhiên, khi sử dụng các module I/O mở rộng, FX1N có thể tăng cường số lượng đầu I/O lên tới 128. FX1N tăng khả năng truyền thông, nối mạng, cho phép tham gia với nhiều cấu trúc mạng khác nhau như EtherNet, Profibus, CC-Link, CANopen... FX1N có thể làm việc với các module tương tự, các bộ điều khiển nhiệt độ. Đặc biệt, FX1N được tăng cường chức năng điều khiển vị trí với 6 Counter tốc độ cao, 2 bộ phát xung đầu ra với tần số điều khiển tối đa là 100 KHz. Điều này cho phép các PLC thuộc dòng FX1N có thể cùng một lúc điều khiển độc lập hai động cơ servo hay tham gia các bài toán điều khiển vị trí.

FX1N có đặc điểm:

- + Cơ cấu nhỏ gọn, chi phí thấp, module màn hình và khối mở rộng có thể dễ dàng nâng cấp.
- + Vận hành tốc độ cao, đối với lệnh cơ bản tốc độ xử lý từ 0.55 đến 0.7 μ s/lệnh, đối với lệnh ứng dụng tốc độ xử lý từ 3.7 đến vào trăm μ s/lệnh.
- + Đặc tính kỹ thuật của bộ nhớ chất lượng và phong phú., bộ nhớ EEPROM 8000 bước lệnh.

- + Dây thiết bị dụng cụ đa năng như rơ le phụ trợ 1536 điểm, bộ đếm 256 điểm, Counter 235 điểm, thanh ghi dữ liệu 8000 điểm.
- + Sáu Counter tốc độ cao(60 KHz), hai bộ phát xung đầu ra với tần số điều khiển tối đa là 100KHz.
- + Các module chức năng đặc biệt có đến hai dãy mở rộng để phục vụ cho những yêu cầu riêng.
- + Dây mở rộng tự cung cấp điện 120 đến 24V AC, 12 đến 24V DC.
- + Quá trình điều khiển tăng, sử dụng lệnh PID cho những hệ thống đòi hỏi sự điều khiển chính xác.
- + Dễ kết nối.
- + Dễ lắp đặt (sử dụng thanh DIN hoặc khoảng trống có sẵn).
- + Đồng hồ thời gian thực.
- + Sử dụng phần mềm GX Developer hoặc FX-PCS/WIN-E.
- + Nâng cấp hệ thống bằng khối mở rộng hoặc kết nối module. Bảng mở rộng có thể được sử dụng để kết nối chức năng truyền thông bằng cách dùng bộ kết nối tương thích RS-232C, RS-485 hoặc RS-422 kết hợp với lối I/O tương tự hoặc số.
- + Mạng truyền thông đa dạng.



(a) FX1N

(b) FX2N

(c) FX2NC

Hình 2.9. PLC FX1N, FX2N và FX2NC của hãng Misubishi

2.3.5. PLC loại FX2N

Đây là PLC có tính năng mạnh nhất trong dòng FX. FX2N được trang bị tất cả tính năng của FX1N nhưng tốc độ xử lý được cải thiện, thời gian thực thi lệnh cơ bản giảm xuống còn 0.08 μ s/lệnh. FX2N thích hợp cho các bài toán điều khiển có số lượng đầu I/O từ 16 đến 128, trong trường hợp cần thiết có thể mở rộng lên 256. Tuy nhiên khi mở rộng lên 256 đầu I/O thì FX2N sẽ mất lợi thế về giá cả và không gian lắp đặt.

Bộ nhớ của FX2N là 8 kstep, bộ nhớ RAM có thể mở rộng đến 16 kstep cho phép điều khiển các bài toán phức tạp. Ngoài ra, ra FX2N còn được trang bị các hàm xử lý PID với tính năng tự chỉnh, các hàm xử lý số thực cùng đồng hồ thời gian thực tích hợp sẵn bên trong.

Những tính năng vượt trội trên cùng với khả năng truyền thông, nối mạng nói chung của dòng FX1N đã đưa FX2N lên vị trí hàng đầu trong dòng FX, có thể đáp ứng tốt các bài toán yêu cầu khắt khe nhất đối với các ứng dụng sử dụng trong các hệ thống điều khiển cỡ nhỏ và trung bình. FX2N phù hợp với các bài toán trong dây chuyền sản xuất, xử lý rác thải, các hệ thống xử lý môi trường, điều khiển máy dệt, lắp ráp tàu biển.

2.3.6. PLC loại FX2NC

FX2NC với kích thước siêu gọn, thích hợp cho các ứng dụng đòi hỏi cao về yêu cầu tiết kiệm không gian lắp đặt. FX2NC có đầy đủ tính năng của FX2N nhưng lại tiết kiệm đến 27% không gian sử dụng. Lĩnh vực ứng dụng của FX2NC chủ yếu là trong xây dựng, trong các hệ thống bơm hay các bài toán điều khiển liên quan đến môi trường.



Hình 2.10. PLC AnS và AnSH/QnAS của hãng Mitsubishi

2.3.7. AnS PLC

Xét về mặt tính năng, PLC dòng A1S của Mitsubishi là một bước phát triển nhảy vọt so với PLC dòng FX. A1S PLC với các đặc trưng cơ bản là giá thành thấp, thiết kế module nhỏ gọn, hiệu năng tính toán cao, có thể đáp ứng được các dạng bài toán điều khiển trong công nghiệp với giá cả cạnh tranh.

Cũng giống như dòng FX, AnS có thể giải quyết tốt các bài toán điều khiển với số lượng I/O nằm trong khoảng 30-140. Điểm vượt trội so với dòng FX là khả năng giải quyết các bài toán điều khiển cấp cao hơn trong đó số lượng I/O quản lý lên tới 512 I/O. AnS PLC được tăng cường thêm khả năng truyền thông và khả năng ghép nối mạng cho phép tham gia giải quyết các bài toán điều khiển giám sát sử dụng máy tính hay các bài toán điều khiển mạng phức tạp ở các cấp điều khiển khác nhau, trong đó tổng khoảng cách truyền thông có thể lên tới 1km, tốc độ truyền tối đa là 10Mbps. Với ưu thế được trang bị khả năng kết nối máy tính theo cổng RS232C/RS-422/RS-485, AnS cho phép người sử dụng có thể tùy chọn thiết lập cấu hình trong các bài toán điều khiển giám sát sử dụng máy tính, số lượng các PLC được giám sát có thể lên tới 32 PLC trong trường hợp sử dụng cấu trúc multidrop. Ngoài ra, AnS còn được trang bị thêm khả năng ghép nối, sử dụng các module chức năng đặc biệt như các Counter tốc độ cao, các bộ

tạo ngắt, bộ A/D, D/A, các bộ điều khiển nhiệt độ.

2.3.8. *AnSH/QnAS PLC*

Đây là dòng PLC có cấu trúc module nhỏ gọn, có thể giải quyết chính xác nhiều bài toán khác nhau. Tùy theo yêu cầu ứng dụng, người sử dụng có thể lắp đặt 60 module khác nhau. Bên cạnh ưu điểm là hiệu suất cao, PLC dòng QnAS/AnSH còn có ưu điểm là tiết kiệm không gian làm việc. Các PLC này có thể điều khiển cùng một lúc 160 đầu vào ra trên một diện tích lắp đặt siêu nhỏ với kích thước 32,5x13cm. AnSH/QnA PLC được hỗ trợ đầy đủ các khả năng về truyền thông, có thể tham gia hoạt động trong các cấu trúc mạng của Mitsubishi như MelsecNet/10, MelsecNet B, MelsecNet Mini hay các cấu trúc mạng mở thông dụng trên thế giới như Profibus, DeviceNet, CC-Link. Đặc biệt, các bộ điều khiển lập trình dòng AnSH/QnAS có thể tham gia các bài toán điều khiển vị trí phức tạp, điều khiển 32 trục khác nhau trên cùng một module hay khả năng điều khiển 96 động cơ bước độc lập. Những ưu thế này đã làm cho AnSH/QnAS PLC trở thành một trong những dòng PLC linh hoạt nhất trên thị trường hiện nay, đáp ứng được các yêu cầu khắt khe trong các bài toán điều khiển ứng dụng cho các nhà máy xí nghiệp hiện đại. Lĩnh vực ứng dụng chính của các bộ PLC này bao gồm: điều khiển quá trình cung cấp xử lý nước, điều khiển các máy công cụ hay các dây chuyền sản xuất dùng trong công nghiệp nhựa, sản xuất giấy, thuốc lá, các dây chuyền lắp ráp, công nghiệp đóng tàu.

2.3.9. *QnA/Q4AR*

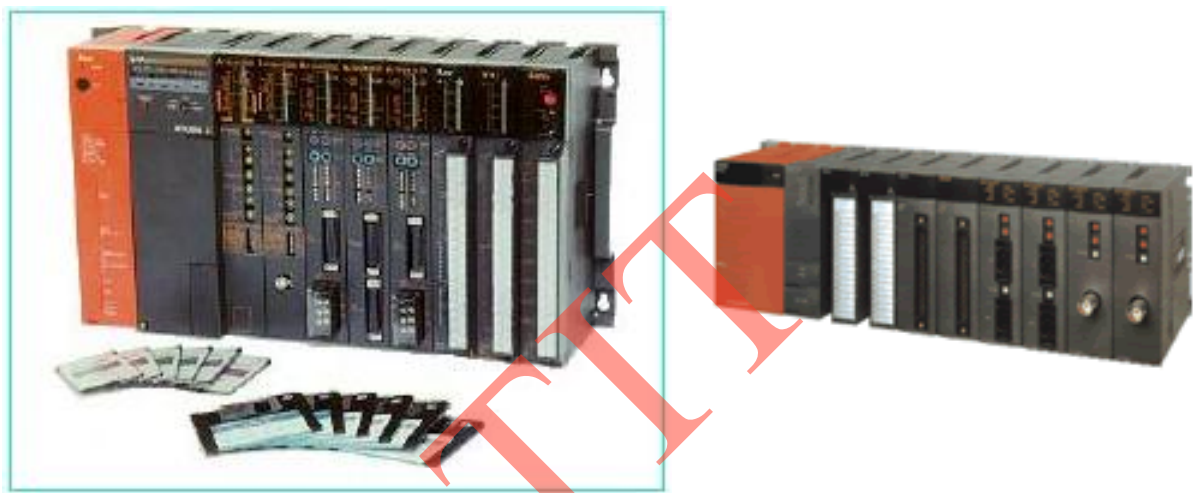
QnA/Q4AR PLC ra đời kế tiếp sự phát triển trong công nghệ sản xuất PLC của Mitsubishi. QnA/Q4AR phát triển các tính năng của dòng QnAS ở mức độ cao hơn, cho phép tại một thời điểm 4 CPU tham gia điều khiển quá trình, giảm thiểu thời gian quét chương trình, tăng tốc độ xử lý. Cấu trúc chương trình điều khiển được tổ chức theo kiểu project cho phép dễ dàng kiểm tra, bắt lỗi và nâng cấp. Đặc biệt, dòng Q4AR có thêm module CPU dự phòng sử dụng để backup chương trình, nâng cao khả năng dự phòng của hệ thống. Chương trình giữa CPU chủ và CPU dự phòng luôn được đồng bộ một cách tự động. Do đó, khi có bất kỳ sự cố nào xảy ra trên CPU chính, quá trình xử lý được tự động chuyển sang CPU dự phòng mà không làm ảnh hưởng đến hệ thống. Một điểm khá thú vị là các bộ PLC dòng QnA/Q4AR cho phép tiến hành bảo dưỡng thiết bị trực tuyến mà không cần phải dừng hệ thống. Sử dụng các khoá trên bề mặt CPU, người dùng hoàn toàn có thể đặt chế độ active/inactive cho CPU tương ứng, các inactive CPU có thể được tháo ra khỏi hệ thống một cách an toàn. QnA/Q4AR PLC có thể được sử dụng trong các nhà máy điện để điều khiển các tuabin, máy phát, trong công nghiệp sản xuất, lắp ráp ô tô; trong công nghiệp hoá dầu...

2.3.10. *Qn PLC*

Đây là bộ điều khiển lập trình mạnh nhất của Mitsubishi trong giai đoạn hiện nay. Bộ PLC dòng Q ra đời nhằm đáp ứng các yêu cầu mở rộng không ngừng của các hệ thống sản xuất tích hợp các kỹ thuật mới, các yêu cầu về truyền thông nhằm phá bỏ các hạn chế của các bộ điều

khien lập trình truyền thống. Điểm nổi bật của các PLC dòng Q là kỹ thuật multi-processor, cho phép tại một thời điểm 4 CPU cùng tham gia xử lý các quá trình điều khiển máy móc, điều khiển vị trí, truyền thông... Do đó, tính năng thời gian thực được tăng cường, thời gian quét vòng của chương trình giảm xuống còn 0.5-2ms. Ở PLC dòng Q, các chức năng mạng được đặc biệt tăng cường, cho phép thiết lập cấu hình mạng MELSECNET với tổng khoảng cách truyền thông lên tới 13,6 km với tốc độ đường truyền tối đa có thể đạt được là 25Mbps. Đặc biệt, các PLC dòng Q được hỗ trợ chức năng nối mạng Internet, cho phép truyền các email cảnh báo đến cấp điều khiển cao hơn ở khoảng cách rất xa.

Dòng Qn PLC được sử dụng phổ biến trong các ngành công nghiệp yêu cầu mức độ tự động hoá cao, trong công nghệ bán dẫn, truyền thông (IT), trong các dây chuyền đóng gói sản phẩm, các hệ thống máy dệt, điều khiển các hệ thống phun sơn, hàn đường.



(a) QnA/Q4AR

(b) Qn

Hình 2.11. PLC QnA/Q4AR và Qn của hãng Mitsubishi

2.4. Kết luận

Chương này giới thiệu PLC của một số hãng như Siemens (Đức), Omron và Mitsubishi (Nhật). Với PLC của Siemens, đi sâu giới thiệu đặc điểm PLC S7-300 như:

- + Các module (CPU, module mở rộng)
- + Kiểu dữ liệu (Bool, byte, word, int, dint...)
- + Tổ chức bộ nhớ CPU
- + Vòng quét chương trình
- + Cấu trúc chương trình (lập trình tuyến tính và lập trình cấu trúc).
- + Các loại ngôn ngữ lập trình dùng cho PLC S7-300 (STL, LAD, FBD).

Với PLC của Omron và Mitsubishi thì tác giả chỉ liệt kê và giới thiệu khái quát các đặc điểm nổi bật của từng họ để từ đó biết được ưu, nhược điểm của chúng.

BÀI TẬP CHƯƠNG 2

Bài tập 2.1

Nêu đặc điểm của từng module mở rộng trong PLC S7-300?

Bài tập 2.2

Liệt kê và nêu đặc điểm của các kiểu dữ liệu trong PLC S7-300?

Bài tập 2.3

Tổ chức bộ nhớ CPU trong PLC S7-200?

Bài tập 2.4

Các giai đoạn của một vòng quét trong PLC S7-300?

Bài tập 2.5

Nêu đặc điểm của hai dạng cấu trúc lập trình trong PLC S7-300?

Bài tập 2.6

Các khối OB đặc biệt trong PLC S7-300?

Bài tập 2.7

Nêu đặc điểm chính của PLC CPM1A của Omron?

Bài tập 2.8

Các thành phần của module I/O tương tự trong PLC CPM1A/CPM2A của Omron?

Bài tập 2.9

Nêu đặc điểm chính của PLC FX1N của Misubishi?

Bài tập 2.10

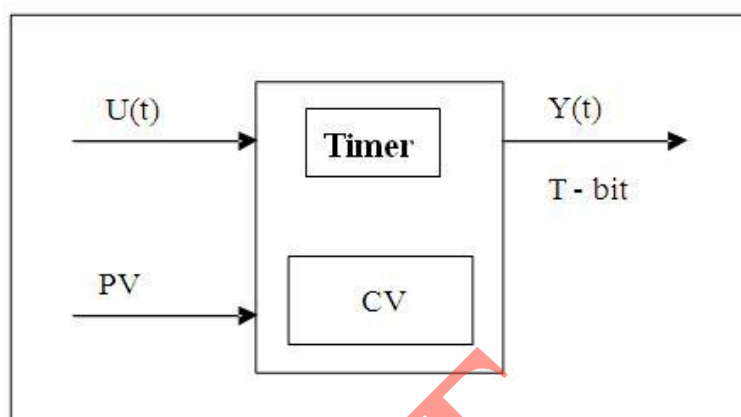
So sánh PLC FX1N và FX2N của Misubishi?

CHƯƠNG 3. BỘ ĐỊNH THỜI VÀ BỘ ĐẾM

3.1. Bộ định thời

3.1.1. Cấu tạo

Bộ định thời (Timer) là bộ tạo thời gian trễ T mong muốn giữa tín hiệu logic đầu vào U(t) và đầu ra Y(t).



Hình 3.1. Timer

S7-300 có năm kiểu khác nhau, chúng bắt đầu tạo thời gian trễ tín hiệu kể từ thời điểm có sườn lên của tín hiệu kích đầu vào, tức là khi có tín hiệu đầu vào U(t) chuyển trạng thái từ logic "0" lên logic "1", được gọi là thời điểm Timer được kích.

Thời gian trễ T mong muốn được khai báo với Timer bằng giá trị 16 bits, bao gồm hai thành phần:

- Độ phân giải (đơn vị là ms): Timer của S7-300 có bốn độ phân giải khác nhau là 10ms, 100ms, 1s và 10s.
- Số nguyên BCD (trong khoảng từ 0 đến 999): được gọi là PV (Preset Value - giá trị trễ đặt trước).

Như vậy thời gian trễ T mong muốn sẽ được tính như sau:

$$T = \text{Độ phân giải} \times PV \quad (3.1)$$

Tùy theo ngôn ngữ lập trình mà có thể khai báo thời gian trễ theo hai cách sau:

- Cách 1: **S5T#5s**: Cách khai báo này dùng được cho các loại ngôn ngữ lập trình Step 7

- Cách 2: **L#W#16#1350**, cách khai báo này chỉ dùng được cho ngôn ngữ STL

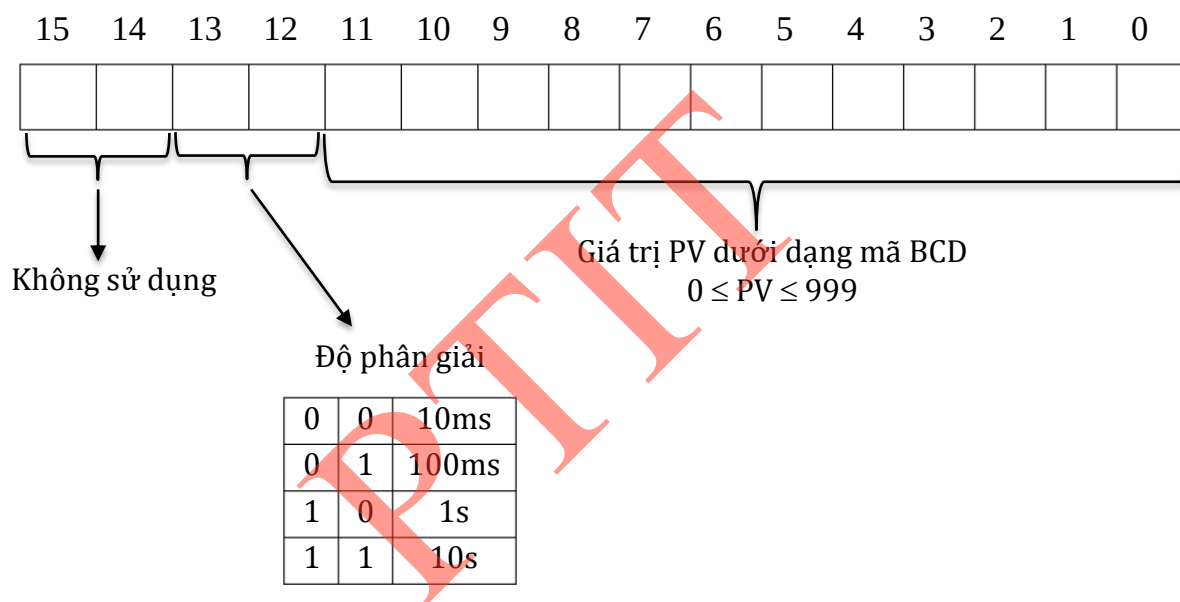
Để xác định được độ phân giải trong cách khai báo thứ nhất ta có thể tính như sau:

Áp dụng công thức tính: **T = Độ phân giải x PV**; trong đó PV là số nguyên lớn nhất có thể nằm trong khoảng $0 \div 999$. Như vậy, nếu khai báo **S5T#5s** thì có thể tính như sau: $5s = 10ms \times 500$, vậy độ phân giải là 10ms. Với cách khai báo này ta không thể thay đổi được độ phân giải vì phần mềm Step7 tự gán cho nó độ phân giải.

Với cách khai báo thứ 2 ta có thể lựa chọn độ phân giải tùy ý. Ví dụ muốn khai báo khoảng thời gian trễ là 5s ta có thể khai báo như sau:

W#16#1050 hoặc **W#16#2005**. Trong đó, chữ số 1 hoặc 2 là độ phân giải được quy định theo bảng 3.1, còn ba chữ số đứng sau là giá trị đặt. Như vậy, trong ví dụ trên với cùng một giá trị thời gian trễ 5s ta có thể đặt được độ phân giải là 100ms hoặc 1s.

Ngay tại thời điểm kích Timer, giá trị PV được chuyển vào thanh ghi 16 bits của Timer T-Word (gọi là thanh ghi CV- Curren value- giá trị tức thời). Timer sẽ ghi nhớ khoảng thời gian trôi qua kể từ khi kích bằng cách giảm dần nội dung thanh ghi CV. Nếu nội dung thanh ghi CV trở về bằng 0 thì Timer đã đạt được thời gian mong muốn T và điều này được báo ra ngoài bằng cách thay đổi trạng thái tín hiệu đầu ra Y(t). Việc thông báo ra ngoài bằng cách đổi trạng thái tín hiệu đầu ra Y(t) như thế nào còn phụ thuộc vào loại Timer được sử dụng.

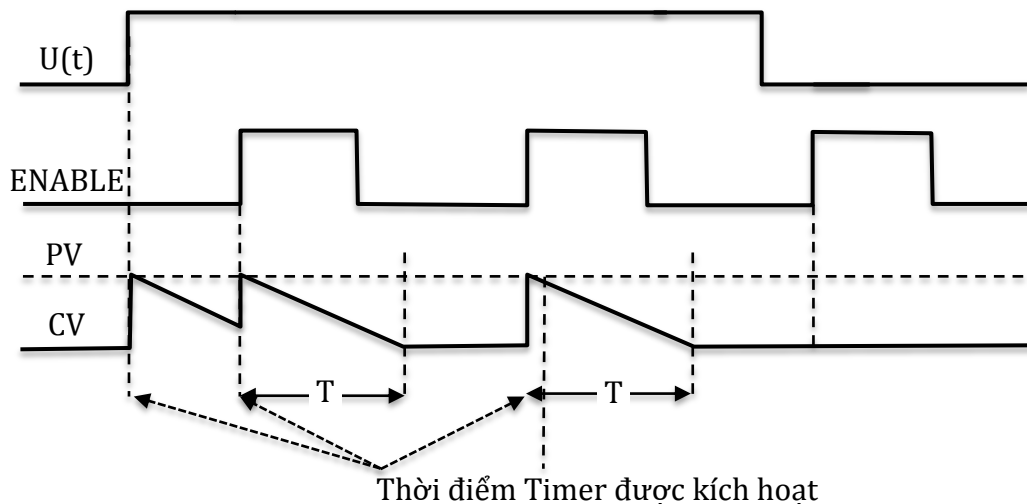


Hình 3.2. Cấu hình giá trị thời gian trễ đặt trước trong PLC S7-300

Bảng 3.1. Giá trị tương ứng độ phân giải trong Timer của S7-300

0	10 ms
1	100 ms
2	1 s
3	10 s

Bên cạnh sườn lên của tín hiệu đầu vào U(t), Timer còn có thể kích bằng sườn lên của tín hiệu kích chủ động có tên là tín hiệu ENABLE nếu như tại thời điểm có sườn lên của tín hiệu ENABLE, tín hiệu đầu vào U(t) có giá trị là "1". Hình 3.3 minh họa nguyên tắc làm việc của Timer.



Hình 3.3. Nguyên tắc làm việc của Timer

Từng loại Timer được đánh số từ 0 đến 255 (tùy thuộc vào từng loại CPU). Một Timer được đặt tên là T_x , trong đó x là số hiệu của Timer ($0 \leq x \leq 255$). Ký hiệu T_x cũng đồng thời là tín hiệu hình thức của thanh ghi CV (T-Word) và đầu ra T-bit của Timer đó. Tuy chúng có cùng địa chỉ hình thức, nhưng T-Word và T-bit vẫn được phân biệt với nhau nhờ kiểu lệnh sử dụng toán hạng T_x . Khi dùng làm việc với “word” T_x được hiểu là T-Word, còn khi làm việc với “bit” thì T_x được hiểu là T-bit.

Để xóa tức thời trạng thái của T-word và T-bit người ta sử dụng một tín hiệu Reset Timer. Tại thời điểm sườn lên của tín hiệu này giá trị T-Word và T-bit đồng thời có giá trị bằng 0, tức là thanh ghi tức thời CV được đặt về 0 và tín hiệu đầu ra cũng có trạng thái logic là "0". Trong thời gian tín hiệu Reset có giá trị logic là "1" Timer sẽ không làm việc.

3.1.2. Khai báo sử dụng

Các tín hiệu điều khiển cho một Timer phải được khai báo bao gồm các bước sau:

- Khai báo tín hiệu ENABLE (nếu muốn sử dụng tín hiệu chủ động kích): dạng dữ liệu BOOL
- Khai báo tín hiệu đầu vào $U(t)$: dạng dữ liệu BOOL
- Khai báo thời gian trễ mong muốn PV: dạng dữ liệu WORD
- Khai báo loại Timer được sử dụng (SP, SE, SD, SS, SF).
- Khai báo tín hiệu xóa Timer nếu muốn sử dụng chế độ Reset chủ động (R): dạng dữ liệu BOOL

Trong các bước trên, khai báo tên Timer, tín hiệu đầu vào, thời gian trễ mong muốn là bắt buộc.

* Khai báo tín hiệu chủ động kích ENABLE:

A <Địa chỉ bit>

FR <Tên Timer>

Toán hạng thứ nhất <Địa chỉ bit> xác định tín hiệu sẽ được sử dụng làm tín hiệu chủ động kích cho Timer trong toán hạng thứ hai.

Lệnh FR tác động lên thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	-	-	0

* Khai báo tín hiệu vào U(t)

A <Địa chỉ bit>

<Địa chỉ bit> trong toán hạng xác định đầu vào U(t) cho Timer.

Ví dụ:

A **I0.0** //Tín hiệu tại cổng vào I0.0 là tín hiệu ENABLE

FR **T1** //Sử dụng Timer T1

A **I0.1** // Tín hiệu tại cổng vào I0.1 là tín hiệu vào

* Khai báo thời gian trễ mong muốn:

L <Hằng số>

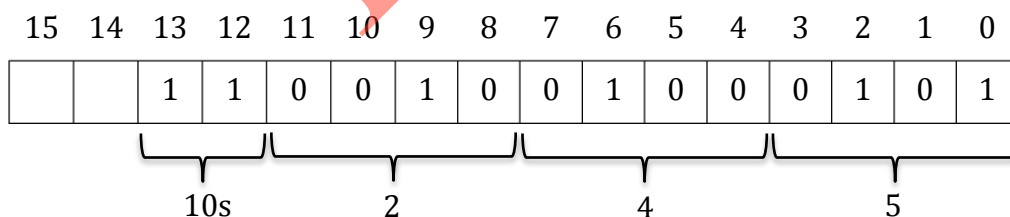
<Hằng số> trong toán hạng là thời gian trễ cho Timer, được xác định theo hai cách:

+ S5T#giờH_phútM_giâyS_miligiâyMS.

L **S5T#2H32M12S00MS** //Tạo trễ 2 giờ 32 phút 12 giây

+ Dạng số nguyên 16 bits như hình 3.4.

L **W#16#3245** //Tạo thời gian trễ 2450s (245*10s)



Hình 3.4 Tạo khoảng thời gian trễ 2450 giây cho Timer

* Khai báo Timer

S7-300 có năm loại Timer được khai báo theo lệnh:

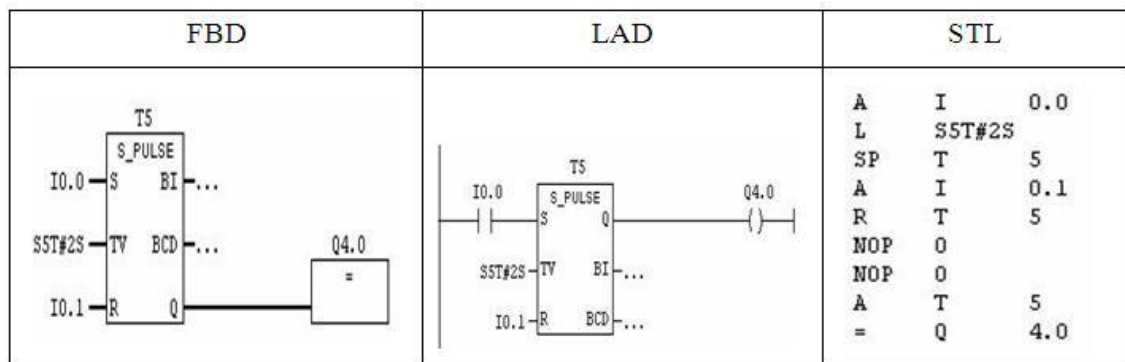
- SP: Tạo xung không nhớ
- SE: Tạo xung có nhớ
- SD: Trễ theo sườn lên không nhớ
- SS: Trễ theo sườn lên có nhớ
- SF: Trễ theo sườn xuống

Những loại này tác động vào thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	-	-	0

- Timer SP (Pulse Timer):

SP <Tên Timer>



Hình 3.5. Khai báo Timer SP bằng ba ngôn ngữ FBD, LAD và STL

+ Nguyên lý làm việc

Thời gian trễ được bắt đầu tính từ khi có sườn lên của tín hiệu vào (hoặc khi có sườn lên của tín hiệu ENABLE đồng thời tín hiệu vào bằng 1). Ngay thời điểm đó, giá trị PV được chuyển vào thanh ghi T-Word (CV). Trong khoảng thời gian trễ, khi T-Word $\neq 0$, T-bit có giá trị bằng 1, ngoài khoảng thời gian trễ T-bit có giá trị bằng 0.

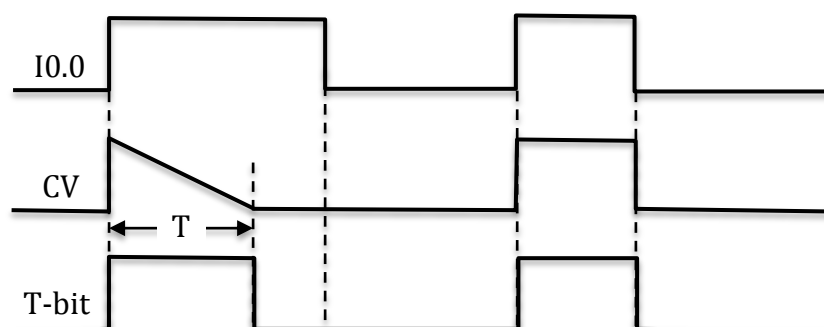
Chưa hết thời gian trễ mà tín hiệu vào về 0 thì T-bit và T-Word cũng về giá trị 0.

Hình 3.5 minh họa cách khai báo Timer SP theo ba ngôn ngữ lập trình là STL, LAD và FBD và hình 3.6 minh họa nguyên tắc hoạt động của nó.

```

A      I0.0 //Tín hiệu vào
L      S5T#15S20MS //Thời gian trễ là 15 giây 20 mili giây
SP     T1 // T1 tạo xung theo sườn lên không nhớ

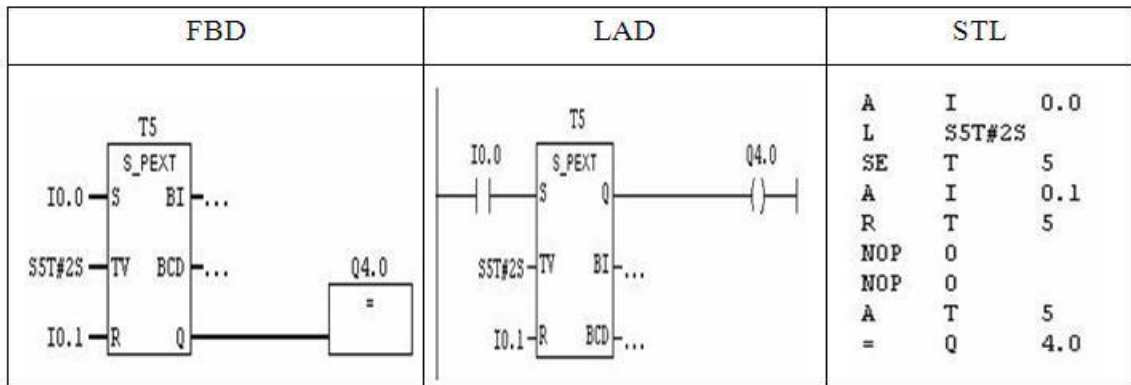
```



Hình 3.6. Timer SP

- Timer SE (Extended Pulse Timer)

SP <Tên Timer>



Hình 3.7. Khai báo Timer SE bằng ba ngôn ngữ FBD, LAD và STL

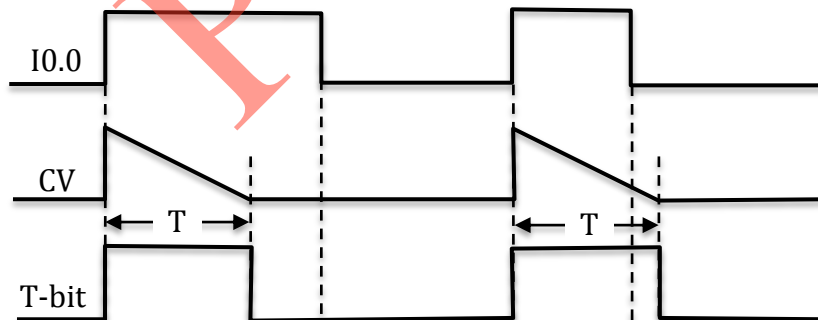
+ Nguyên lý làm việc

Thời gian trễ được bắt đầu tính từ thời điểm có sườn lên của tín hiệu vào (hoặc khi có sườn lên của tín hiệu ENABLE đồng thời tín hiệu vào bằng 1). Ngay thời điểm đó, giá trị PV được chuyển vào thanh ghi T-Word (CV). Trong khoảng thời gian trễ, khi T-Word $\neq 0$, T-bit có giá trị bằng 1, ngoài khoảng thời gian trễ T-bit có giá trị bằng 0.

Nếu chưa hết thời gian trễ mà tín hiệu vào về 0 thì thời gian trễ vẫn được tiếp tục cho đến hết. Vậy T-Word và T-bit không về 0 theo tín hiệu vào.

Hình 3.7 minh họa cách khai báo Timer SE theo ba ngôn ngữ lập trình là STL, LAD và FBD và hình 3.8 minh họa nguyên tắc hoạt động của nó.

A **I0.0** //Tín hiệu vào
L **S5T#15S20MS** //Thời gian trễ là 15 giây 20 mili giây
SE **T1** // T1 tạo xung theo sườn lên có nhớ



Hình 3.8. Timer SE

- Bộ thời gian SD (On Delay Timer)

SD **<Tên Timer>**

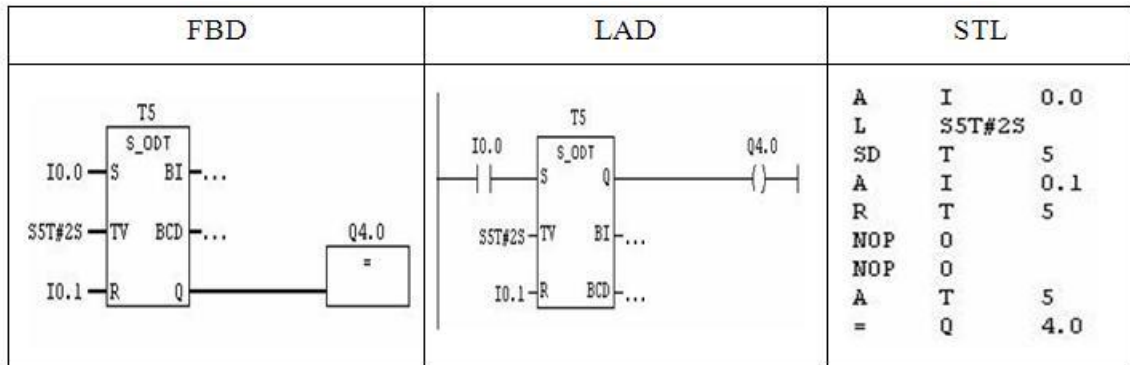
+ Nguyên lý làm việc:

Thời gian trễ được bắt đầu tính từ thời điểm có sườn lên của tín hiệu vào (hoặc khi có sườn lên của tín hiệu ENABLE đồng thời tín hiệu vào bằng 1). Ngay thời điểm đó, giá trị PV được chuyển vào thanh ghi T-Word (CV). Trong khoảng thời gian trễ, khi T-Word $\neq 0$, T-bit có giá trị bằng 0, ngoài khoảng thời gian trễ T-bit có giá trị bằng 1. Như vậy T-bit có giá trị bằng 1 khi T-Word=0

Khoảng thời gian trễ chính là khoảng thời gian giữa thời điểm xuất hiện sườn lên của tín hiệu đầu vào và sườn lên của T-bit.

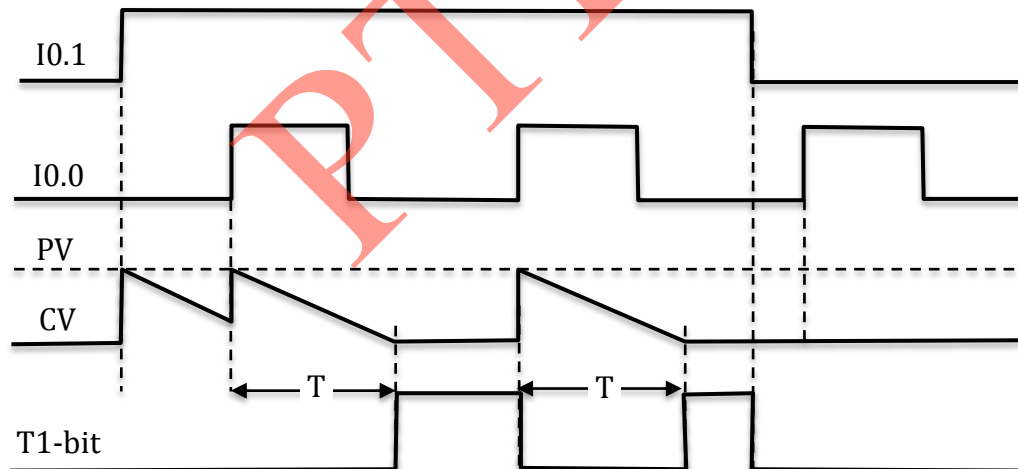
Khi tín hiệu vào bằng 0, cả T-Word và T-bit đều bằng 0.

Hình 3.9 minh hoạ cách khai báo Timer SD theo ba ngôn ngữ lập trình là STL, LAD và FBD và hình 3.10 minh hoạ nguyên tắc hoạt động của nó.



Hình 3.9. Khai báo Timer SD bằng ba ngôn ngữ FBD, LAD và STL

A I0.0 //Tín hiệu ENABLE
FR T1 //Timer T1
A I0.1 //Tín hiệu vào
L SST#15S20MS //Thời gian trễ là 15 giây 20 mili giây
SD T1 // T1 trễ theo sườn lên không nhớ



Hình 3.10. Timer SD

- Timer SS (*Retentive On Delay Timer*)

SS <Tên Timer>

+ Nguyên lý làm việc:

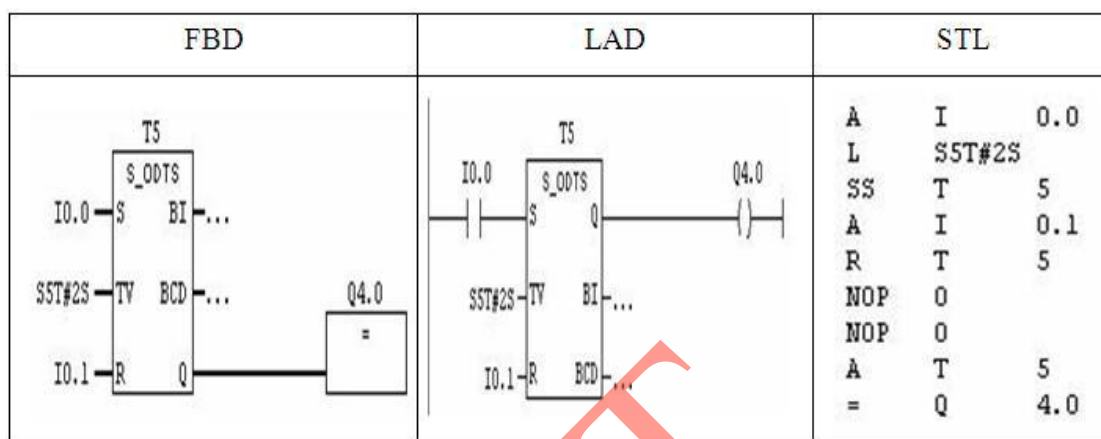
Thời gian trễ được bắt đầu tính từ thời điểm có sườn lên của tín hiệu vào (hoặc khi có sườn lên của tín hiệu ENABLE đồng thời tín hiệu vào bằng 1). Ngay thời điểm đó, giá trị PV được chuyển vào thanh ghi T-Word (CV). Trong khoảng thời gian trễ, khi

T-Word $\neq$ 0, T-bit có giá trị bằng 0, ngoài khoảng thời gian trễ T-bit có giá trị bằng 1. Như vậy T-bit có giá trị bằng 1 khi T-Word=0

Khoảng thời gian trễ chính là khoảng thời gian giữa thời điểm xuất hiện sườn lên của tín hiệu đầu vào và sườn lên của T-bit.

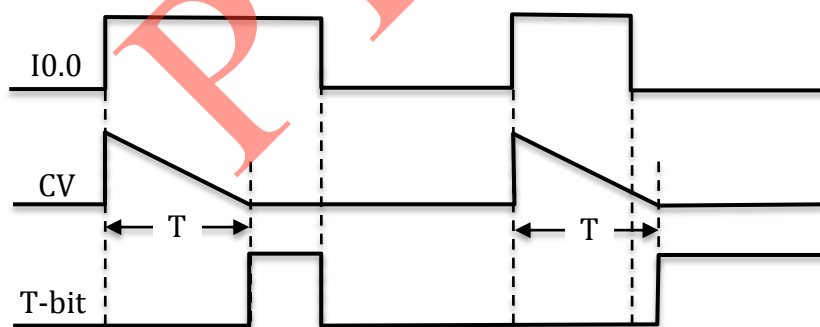
Thời gian trễ vẫn được tính kể cả khi tín hiệu đầu vào đã về 0.

Hình 3.11 minh họa cách khai báo Timer SS theo ba ngôn ngữ lập trình là STL, LAD và FBD và hình 3.12 minh họa nguyên tắc hoạt động của nó.



Hình 3.11. Khai báo Timer SS bằng ba ngôn ngữ FBD, LAD và STL

A I0.0 //Tín hiệu vào
L SST#15S20MS //Thời gian trễ là 15 giây 20 mili giây
SS T1 // T1 trễ theo sườn lên có nhớ



Hình 3.12. Timer SS

- Bộ thời gian SF (*Off Delay Timer*):

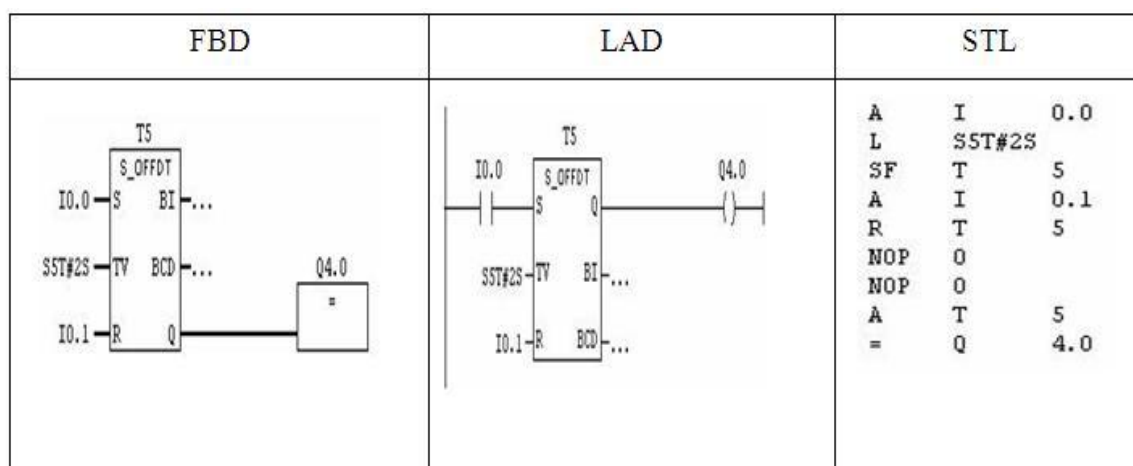
SF <Tên Timer>

+ Nguyên lý làm việc:

Thời gian trễ được bắt đầu tính từ thời điểm có sườn xuống của tín hiệu vào (hoặc khi có sườn xuống của tín hiệu ENABLE đồng thời tín hiệu vào bằng 1). Ngay thời điểm đó, giá trị PV được chuyển vào thanh ghi T-Word (CV).

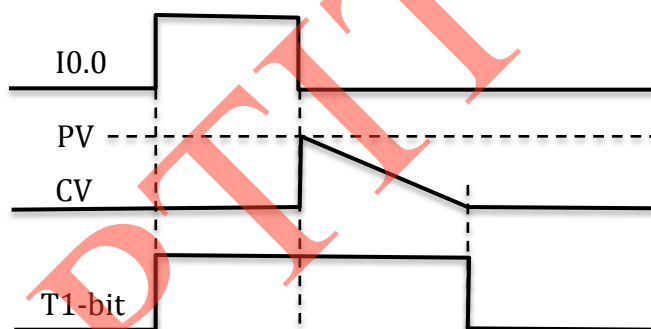
Trong khoảng giữa sườn lên của tín hiệu đầu vào hoặc T-Word $\neq$ 0, T-bit có giá trị bằng 1, ngoài khoảng thời đó T-bit có giá trị bằng 0.

Hình 3.13 minh họa cách khai báo Timer SF theo ba ngôn ngữ lập trình là STL, LAD và FBD và hình 3.14 minh họa nguyên tắc hoạt động của nó.



Hình 3.13. Khai báo Timer SF bằng ba ngôn ngữ FBD, LAD và STL

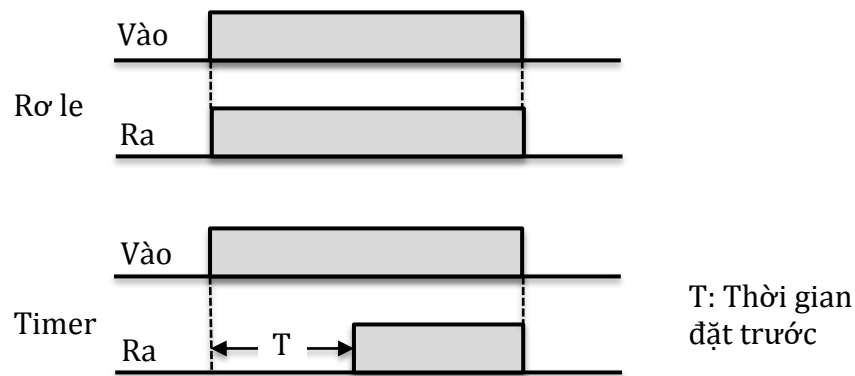
A **I0.0** // Tín hiệu vào
L **SST#15S20MS** // Thời gian trễ là 15 giây 20 mili giây
SF **T1** // T1 trễ theo sườn xuống có nhớ



Hình 3.14. Timer SF

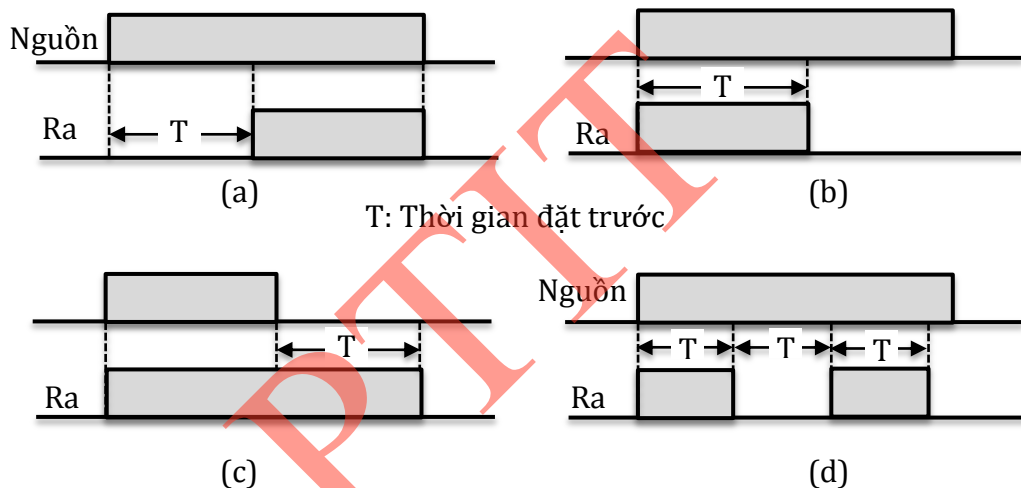
Trong những nhiệm vụ điều khiển có yêu cầu điều khiển theo thời gian, ví dụ như động cơ hoặc bơm mở/ đóng theo thời gian thì phải sử dụng đến chức năng của Timer trong PLC. Các nhà sản xuất không thống nhất về cách lập trình cho Timer và vai trò của chúng. Điểm chung là xem Timer như là các rơ le hay cuộn dây, khi được cấp nguồn sẽ đóng hoặc mở các tiếp điểm sau một khoảng thời gian xác định trước. Một số nhà sản xuất lại coi Timer là khối trễ, khi được chèn vào sẽ là làm trễ tín hiệu của lối ra.

Nếu rơ le chuyển mạch tiếp điểm và gửi tín hiệu ra tại thời điểm khi cuộn dây có điện áp kích thì Timer khi nhận tín hiệu vào nó sẽ bắt đầu đếm thời gian. Khi đạt được thời gian cài đặt trước, nó sẽ chuyển mạch tiếp điểm và gửi tín hiệu ra. Như vậy Timer khác rơ le ở chỗ nó cộng thêm yếu tố thời gian vào tín hiệu ra. Hình 3.15 biểu thị sự khác nhau giữa Timer và rơ le.



Hình 3.15. Hoạt động của rơ le và Timer.

Chế độ hoạt động của Timer liên quan đến việc làm thế nào để tín hiệu ra ON/OFF khi đạt tới thời gian trễ yêu cầu. Có 4 chế độ hoạt động thường xuyên được sử dụng trong Timer, nhưng phổ biến nhất là chế độ bật trễ (On Delay).



Hình 3.16. Các chế độ hoạt động của Timer: On Delay (a), Interval (b), Off Delay (c) và Flicker (d)

+ On Delay: Khi Timer được cấp nguồn, nó sẽ xuất một tín hiệu ở lối ra sau khoảng thời gian đặt trước (T).

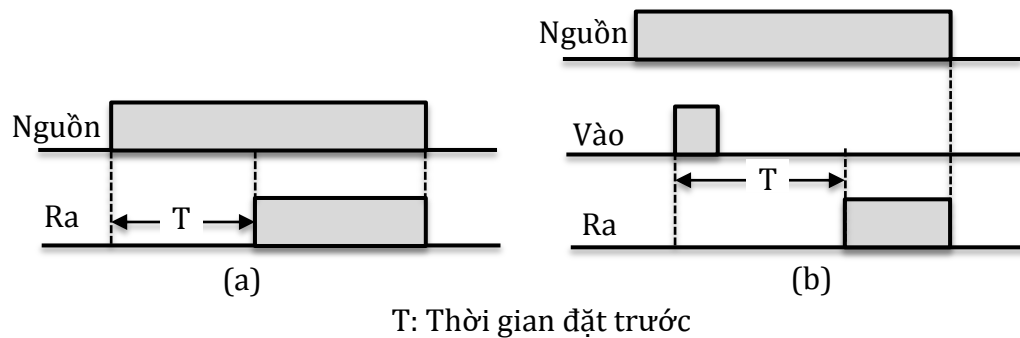
+ Interval: Khi Timer được cấp nguồn điện, sẽ có (ON) tín hiệu ở lối ra cùng thời điểm Timer được cấp nguồn, và duy trì ON trong khoảng thời gian chỉ định (T).

+ Off Delay: Khi Timer được cung cấp nguồn điện, tín hiệu lối ra sẽ ON cùng thời điểm Timer được cấp nguồn, và khi ngắt nguồn cung cấp cho Timer, tín hiệu lối ra sẽ OFF sau khoảng thời gian định trước (T).

+ Flicker: Khi Timer được cấp nguồn, lối ra ON/OFF theo chu kỳ thời gian (T) đã cài đặt.

* Phương thức hoạt động:

Chế độ hoạt động được chia thành hai loại theo phương thức khởi động là bằng nguồn hoặc bằng tín hiệu. Hình 3.17 chỉ ra sự khác biệt của hai phương thức này với Timer On Delay Timer.



Hình 3.17. Phương thức hoạt động của Timer: khởi động bằng nguồn (a) và khởi động bằng tín hiệu (b).

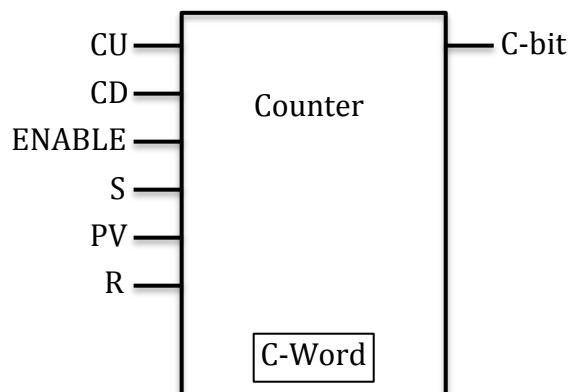
- Khởi động bằng nguồn (Power On Start): Thời gian được tính ngay khi Timer được cấp nguồn điện (hoặc ngắt nguồn điện).
- Khởi động bằng tín hiệu (Signal Start): Điện áp của Timer được cấp trước, và thời gian được bắt đầu tính khi kích hoạt ON/OFF tín hiệu lối vào.

3.2. Counter

3.2.1. Cấu tạo

Counter thực hiện chức năng đếm tại các sườn lên của các xung đầu vào. S7-300 có tối đa 256 Counter phụ thuộc vào từng loại CPU, được ký hiệu là Cx, trong đó x là số nguyên trong khoảng từ 0 đến 255. Trong S7-300 có hai loại Counter thường sử dụng là CU (Count Up), đếm tiến theo sườn lên của tín hiệu vào, và CD (Count Down), đếm lùi theo sườn dương của tín hiệu vào.

Một Counter tổng quát có thể được mô tả như hình 2.18



Hình 3.18. Counter

Trong đó:

CU: tín hiệu kích đếm tiến

CD: tín hiệu kích đếm lùi

S: tín hiệu đặt

PV: giá trị đặt trước

R: tín hiệu xoá

CV: giá trị đếm ở hệ đếm 16 hoặc hệ đếm BCD

C-bit: báo trạng thái C-Word

Thông thường Counter chỉ đếm sườn lên của tín hiệu CU, CD, nhưng cũng có thể mở rộng để đếm cả mức tín hiệu của chúng bằng cách sử dụng thêm tín hiệu kích đếm ENABLE. Nếu có tín hiệu ENABLE, Counter sẽ đếm tiến khi có sườn lên ENABLE tại thời điểm CU có mức tín hiệu bằng 1 và đếm lùi khi có sườn lên ENABLE tại thời điểm CD có mức tín hiệu bằng 1.

Quá trình làm việc của Counter được mô tả như sau:

Số sườn xung đếm được, được ghi vào thanh ghi 2 bytes của Counter, gọi là thanh ghi C-Word. Nội dung của thanh ghi C-Word được gọi là giá trị đếm tức thời của Counter và ký hiệu bằng CV (Current Value) (đếm ở hệ 16) và CV_BCD (đếm ở hệ BCD). Counter báo trạng thái của C-Word ra ngoài qua chân C-bit. Nếu $CV \neq 0$, C-bit có giá trị "1". Ngược lại khi $CV = 0$, C-bit nhận giá trị 0. CV luôn là giá trị không âm. Counter sẽ không đếm lùi khi $CV = 0$.

Đối với Counter, giá trị đặt trước PV chỉ được chuyển vào C-Word tại thời điểm xuất hiện sườn lên của tín hiệu đặt tới chân S.

Counter sẽ được xoá tức thời bằng tín hiệu xoá R (Reset). Khi Counter được xoá, cả C-Word và C-bit đều nhận giá trị 0.

3.2.2. Khai báo sử dụng

Việc khai báo sử dụng một Counter bao gồm các bước sau:

- Khai báo tín hiệu ENABLE nếu muốn sử dụng tín hiệu chủ động kích đếm (S): dạng dữ liệu BOOL
- Khai báo tín hiệu đầu vào đếm tiến CU: dạng dữ liệu BOOL
- Khai báo tín hiệu đầu vào đếm lùi CD: dạng dữ liệu BOOL
- Khai báo giá trị đặt trước PV: dạng dữ liệu WORD
- Khai báo tín hiệu đặt (S): dạng dữ liệu BOOL
- Khai báo tín hiệu xoá (R): dạng dữ liệu BOOL

Trong đó cần chú ý các tín hiệu sau bắt buộc phải khai báo: Tên của Counter cần sử dụng, tín hiệu kích đếm CU hoặc CD.

* Khai báo tín hiệu kích đếm ENABLE

A **<Địa chỉ bit>**

FR <Tên Counter>

Toán hạng thứ nhất <Địa chỉ bit> xác định tín hiệu sẽ được sử dụng làm tín hiệu kích cho Counter có tên trong toán hạng thứ hai. Tên của Counter là Cx với $0 \leq x \leq 255$.

A **I0.0** //Tín hiệu tại cổng vào I0.0 dùng làm tín hiệu chủ động kích ENABLE

FR C1 //Sử dụng Counter có tên C1

Lệnh FR tác động vào thanh ghi trạng thái như sau:

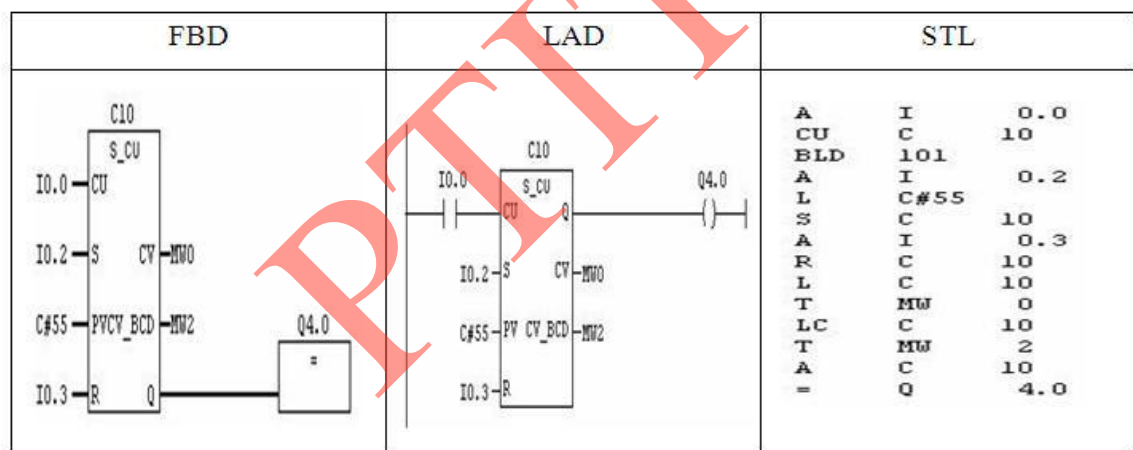
BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	-	-	0

* Khai báo tín hiệu đếm tiến theo sườn lên

A <Địa chỉ bit>

CU <Tên Counter>

Toán hạng thứ nhất <Địa chỉ bit> xác định tín hiệu mà sườn lên của nó sẽ được Counter có tên trong toán hạng thứ hai đếm tiến. Mỗi khi xuất hiện sườn lên của tín hiệu này, Counter sẽ tăng nội dung thanh ghi C-Word (CV) lên 1 đơn vị. Lệnh CU tác động vào thanh ghi trạng thái giống lệnh FR.



Hình 3.19. Khai báo Counter tiến bằng ba ngôn ngữ FBD, LAD và STL

Nguyên lý hoạt động của Counter trong hình 3.19:

Khi tín hiệu I0.2 chuyển từ "0" lên "1" Counter được đặt giá trị là 55. Giá trị đầu ra Q4.0 = 1.

Counter sẽ thực hiện đếm tiến tại các sườn lên của tín hiệu tại chân CU khi tín hiệu I0.0 chuyển giá trị từ "0" lên "1".

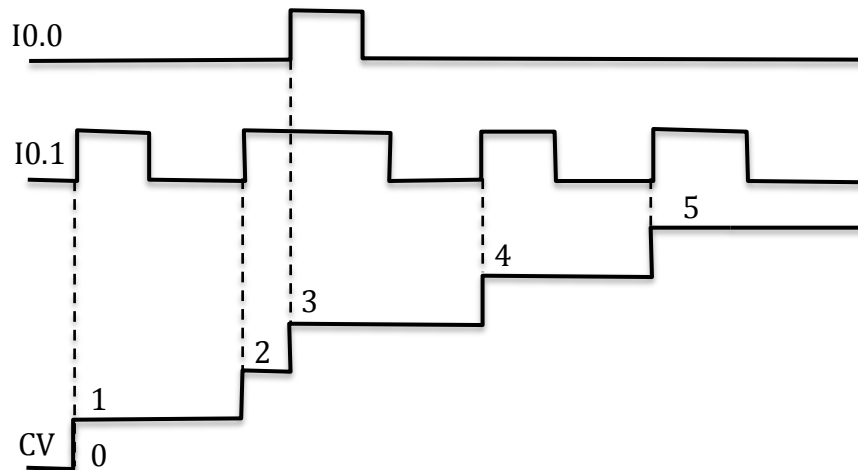
Giá trị của Counter sẽ trở về 0 khi có tín hiệu tại sườn lên của chân R (I0.3). Counter sẽ chỉ đếm đến giá trị nhỏ hơn hoặc bằng 999.

A **I0.0** //I0.0 được dùng làm tín hiệu ENABLE

FR **C1** //Sử dụng Counter C1

A **I0.1** //I0.1 được dùng làm tín hiệu vào

CV **C1** // C1 đếm tiến khi có sườn lên của I0.1 hoặc có sườn lên của I0.0 tại thời điểm I0.1=1



Hình 3.20. Counter tiến theo sườn lên

* Khai báo tín hiệu đếm lùi theo sườn lên

A <Địa chỉ bit>

CD <Tên Counter>

Toán hạng thứ nhất <Địa chỉ bit> xác định tín hiệu mà sườn lên của nó sẽ được Counter có tên trong toán hạng thứ hai đếm lùi. Mỗi khi xuất hiện sườn lên của tín hiệu này, Counter sẽ giảm nội dung thanh ghi C-Word (CV) xuống 1 đơn vị. Lệnh CD tác động vào thanh ghi trạng thái giống lệnh FR.

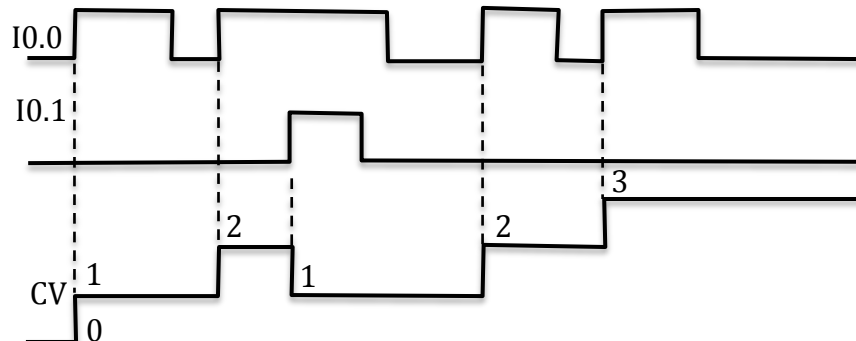
FBD	LAD	STL
		<pre> A I 0.0 CD C 10 BLD 101 A I 0.1 L C#55 S C 10 A I 0.2 R C 10 L C 10 T MW 0 LC C 10 T MW 2 A C 10 = Q 4.0 </pre>

Hình 3.21. Khai báo Counter lùi bằng ba ngôn ngữ FBD, LAD và STL

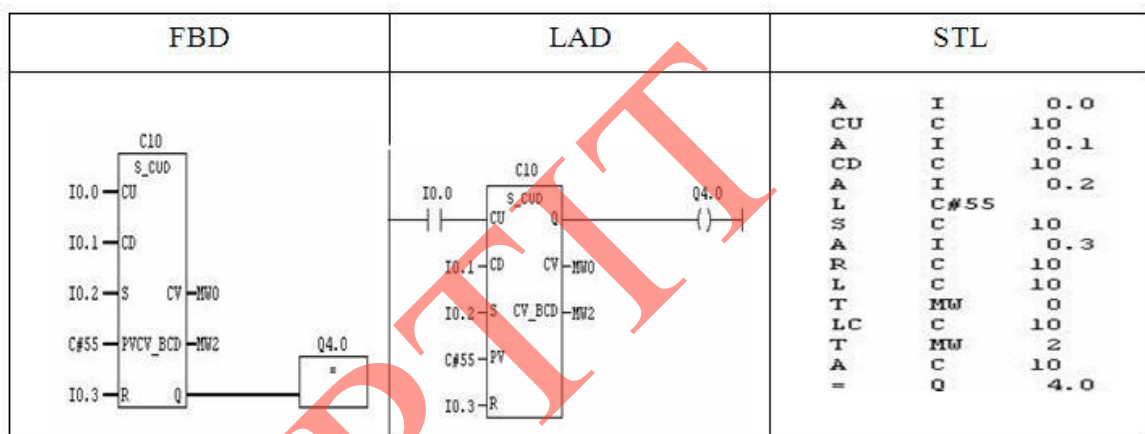
Nguyên lý hoạt động của Counter trong hình 3.21:

- Khi tín hiệu I0.2 chuyển từ "0" lên "1" Counter được đặt giá trị là 55. Giá trị đầu ra Q4.0 =1.
- Counter sẽ thực hiện đếm lùi tại các sườn lên của tín hiệu tại chân CD khi tín hiệu I0.0 chuyển giá trị từ "0" lên "1".
- Giá trị của Counter sẽ trở về 0 khi có tín hiệu tại sườn lên của chân R(I0.3). Counter sẽ chỉ đếm đến giá trị lớn hơn hoặc bằng 0.

A **I0.0** //I0.0 được dùng làm tín hiệu vào để đếm tiến
CU **C1** //Sử dụng Counter tiến C1
A **I0.1** //I0.1 được dùng làm tín hiệu vào để đếm lùi
CD **C1** // C1 đếm lùi khi có sườn lên của I0.1 hoặc có sườn lên của I0.0 tại thời điểm I0.1=1



Hình 3.22. Counter đếm tiến, lùi theo sườn lên



Hình 3.23. Khai báo Counter tiến/lùi bằng ba ngôn ngữ FBD, LAD và STL

Nguyên lý hoạt động của Counter trong hình 3.23:

Khi tín hiệu I0.2 chuyển từ 0 lên 1 Counter được đặt giá trị là 55. Giá trị đầu ra Q4.0 =1.

Counter sẽ thực hiện đếm tiến tại các sườn lên của tín hiệu tại chân CU khi tín hiệu I0.0 chuyển giá trị từ "0" lên "1".

Counter sẽ đếm lùi tại các sườn lên của tín hiệu tại chân I0.1 khi tín hiệu chuyển từ "0" lên "1". Giá trị của Counter sẽ trở về 0 khi có tín hiệu tại sườn lên của chân R (I0.3)

3.3. Kết luận

Timer và Counter là hai thành phần quan trọng trong PLC, hầu hết các ứng dụng đều phải sử dụng chức năng của chúng. Chương này giới thiệu sâu về đặc điểm, phân loại, cách lập trình cho từng loại Timer và Counter của PLC S3-300 của Siemens theo ba ngôn ngữ lập trình là STL, LAD và FBD.

S7-300 có 256 Timer với năm loại là:

- + SP: Tạo xung không nhớ
- + SE: Tạo xung có nhớ
- + SD: Trễ theo sườn lên không nhớ
- + SS: Trễ theo sườn lên có nhớ
- + SF: Trễ theo sườn xuống

S7-300 có 256 Counter với hai loại là:

- + CU (Count Up): đếm tiến theo sườn lên của tín hiệu vào
- + CD (Count Down): đếm lùi theo sườn dương của tín hiệu vào

PTIT

BÀI TẬP CHƯƠNG 3

Bài tập 3.1

- Cấu tạo của Timer trong PLC S7-300?
- Đặt giá trị PV bằng 2h35m23s trong Timer bằng cách nào?

Bài tập 3.2

- Cấu tạo của Counter trong PLC S7-300?
- Đặt giá trị PV bằng 984 trong Counter thì làm thế nào?

Bài tập 3.3

Phân biệt C-word và C-bit trong Timer SD và SS?

Bài tập 3.4

- Nêu đặc điểm của các Timer SP, SE, SD, SS và SF trong S7-300
- Tín hiệu vào có dạng xung:



Vẽ đồ thị dạng xung của các Timer SP, SE, SD, SS và SF

Bài tập 3.5

Cho đoạn chương trình viết bằng ngôn ngữ STL:

```
A      I1.0
L      S5T#20s
SF     T1
A      I1.2
R      T1
A      T1
=      Q2.0
L      T1
T      MW10
LC     T1
T      MW20
```

- Chú thích ý nghĩa từng dòng lệnh
- Vẽ và nêu cấu tạo và ý nghĩa của T1. Giá trị PV trong T1 là 85s được biểu diễn như thế nào?

Bài tập 3.6

Cho đoạn chương trình viết bằng ngôn ngữ STL:

```

A      I2.0
FR     T2
A      I2.1
L      S5T#50s
SE     T2
A      I2.2
R      T2
A      T2
=      Q4.0
L      T2
T      MW10
LC     T2
T      MW20

```

- Chú thích ý nghĩa từng dòng lệnh
- Vẽ và nêu cấu tạo và ý nghĩa của T2. Giá trị PV trong T2 là 105s được biểu diễn như thế nào?

Bài tập 3.7

Cho đoạn chương trình viết bằng ngôn ngữ STL:

```

A      I1.0
FR     T1
A      I1.1
L      S5T#25s
SD     T1
A      I1.2
R      T1
A      T1
=      Q2.0
L      T1
T      MW10
LC     T1
T      MW20

```

- Chú thích ý nghĩa từng dòng lệnh
- Vẽ và nêu cấu tạo và ý nghĩa của T1. Giá trị PV trong T1 là 94s được biểu diễn như thế nào?

Bài tập 3.8

Cho đoạn chương trình viết bằng ngôn ngữ STL:

```

A      I2.0
FR     C2
A      I2.1
CU     C2
A      I2.2
CD     C2
A      I2.3
L      C#5
S      C2
LC     C2
T      MW0
L      C2

```

```

T      MW1
A      C2
=      Q1.0

```

- Chú thích ý nghĩa từng dòng lệnh
- Vẽ và nêu cấu tạo của C2. Giá trị PV trong C2 là 875 được biểu diễn như thế nào?

Bài tập 3.9

Cho đoạn chương trình viết bằng ngôn ngữ STL:

```

A      I1.0
FR     C1
A      I1.1
CU     C1
A      I1.2
CD     C1
A      I1.3
L      C#3
S      C1
A      I1.4
R      C1

```

Chú thích ý nghĩa từng dòng lệnh và vẽ đồ thị minh họa tác dụng của tín hiệu đặt I1.0, I1.1, I1.2, I1.3 và I1.4.

Bài tập 3.10

Cho đoạn chương trình viết bằng ngôn ngữ STL:

```

A      I2.0
FR     C1
A      I2.1
CD     C1
A      I2.2
CU     C1
A      I2.3
L      C#5
S      C1
A      I2.4
R      C1

```

Chú thích ý nghĩa từng dòng lệnh và vẽ đồ thị minh họa tác dụng của tín hiệu đặt I2.0, I2.1, I2.2, I2.3 và I2.4.

CHƯƠNG 4. NGÔN NGỮ LẬP TRÌNH CHO PLC

4.1. Giới thiệu chung

Như đã giới thiệu trong phần chương 2, PLC được lập trình theo ba ngôn ngữ chính là SLT, LAD và FBD. Với dòng PLC của Siemens hiện nay, chương trình điều khiển được viết trên phần mềm STEP7.

* Sơ lược về STEP7:

STEP7 là phần mềm dùng cho việc cấu hình và lập trình các bộ điều khiển PLC do Siemens thiết kế bao gồm những Version sau:

- STEP7 Micro/Dos và STEP7 Micro/Win dành cho các ứng dụng chuẩn, đơn giản trên SIMATIC S7-200.
- STEP7 Mini dành cho các ứng dụng chuẩn, đơn giản trên SIMATIC S7-300 và SIMATIC C7-620.
- STEP7 dành cho các ứng dụng trên SIMATIC S7-300/S7-400, SIMATIC S7-300/M7-400 và SIMATIC C7 với các chức năng rộng hơn:
 - + Gán thông số cho các module hàm và các bộ xử lý truyền thông.
 - + Hoạt động ở chế độ nhiều máy tính.
 - + Truyền thông dữ liệu toàn cục.
 - + Truyền dữ liệu theo sự kiện sử dụng các khối hàm truyền thông (communication function blocks).
 - + Đặt cấu hình kết nối.

* Cài đặt STEP7

Yêu cầu phần cứng :

- + Hệ điều hành: Windows 95/98/NT.
- + Phần cứng: các dòng máy tính hiện nay đều thỏa mãn.
- Cài đặt:
 - + Cho đĩa STEP 7 vào ổ đĩa CD-ROM.
 - + Chạy chương trình setup trên đĩa (như cài đặt các phần mềm khác). Điểm khác biệt so với các cài đặt thông thường:
 - Khai báo số hiệu sản phẩm (luôn đi kèm theo đĩa). Do đó, trong quá trình cài đặt khi có yêu cầu này thì phải điền đầy đủ thông tin vào các mục yêu cầu.
 - Đăng ký bản quyền: Do Siemens cung cấp và thường chứa trong đĩa mềm riêng. Có thể đăng ký bản quyền ngay trong quá trình cài đặt hoặc sau khi cài xong phần mềm thì chạy AuthorsW.exe có trong danh sách của SIMATIC.

* Việc cần làm với phần mềm STEP7:

- Lập kế hoạch cho bộ điều khiển.
- Thiết kế cấu trúc chương trình.

- Khởi động STEP7
- Tạo cấu trúc project.
- Đặt cấu hình cho trạm.
- Đặt cấu hình mạng và các kết nối truyền thông.
- Định nghĩa các ký hiệu.
- Tạo chương trình.
- Đối với S7: tạo và đánh giá các dữ liệu tham chiếu.
- Đặt cấu hình các thông điệp.
- Đặt cấu hình các biến điều khiển.
- Download chương trình xuống bộ điều khiển.
- Kiểm tra chương trình.
- Quan sát hoạt động và chẩn đoán lỗi.

4.2. Tập lệnh của S7-300

4.2.1. Cấu trúc lệnh

* Cấu trúc của một lệnh STL có dạng:

"Tên lệnh" + "Toán hạng"

Ví dụ:

L PIW274 // Đọc nội dung cổng vào của module tương tự

Trong đó toán hạng có thể là dữ liệu hoặc một địa chỉ ô nhớ.

* Toán hạng là dữ liệu:

- Dữ liệu logic TRUE (0) hoặc FALSE (0) có độ dài 1 bit:

CALL FC1

In_Bit_1 = TRUE //Giá trị logic 1 được gán cho biến hình thức
In_Bit_1

In_Bit_2 = FALSE // Giá trị logic được gán cho biến hình thức
In_Bit_2

Ret_val = MW0 //Giá trị trả về

- Dữ liệu số nhị phân:

L 2#110011 //Nạp số nhị phân 110011 vào thanh ghi ACCU1

- Dữ liệu là số Hexadecimal x có độ dài 1 byte (B#16#x), 1 từ (W#16#x) hoặc 1 từ kép (DW#16#x):

L B#16#1E //Nạp số 1E vào byte thấp của thanh ghi ACCU1

L W#16#3A //Nạp số 3A2 vào 2 byte thấp của thanh ghi ACCU1

L DW#16#D3A2E //Nạp số D3A2E vào thanh ghi ACCU1

- Dữ liệu là số nguyên x với độ dài 2 bytes cho biến kiểu INT:

L 930

L -1025

- Dữ liệu là số nguyên x với độ dài 4 bytes dạng L#x cho biến kiểu DINT.

L L#930

L L#-2047

- Dữ liệu là số thực x cho biến kiểu REAL.

L 1.234567e+13

L 930.0

- Dữ liệu thời gian cho biến kiểu S5T dạng giờ_phút_giây_mili giây

L S5T#3h:20m:23s:10ms

- Dữ liệu thời gian cho biến TOD dạng giờ:phút:giây

L TOD#2:20:00

- DATE: Biểu diễn giá trị thời gian tính theo năm/tháng/ngày.

L DATE#1999 - 12 - 8.

- C: Biểu diễn giá trị số đếm đặt trước cho Counter.

L C#20

- P: Dữ liệu biểu diễn địa chỉ của một bit ô nhớ.

L P#Q0.0

- Dữ liệu kiểu "kí tự".

L 'ABCD'

L 'E'

* Toán hạng là địa chỉ

Địa chỉ ô nhớ trong S7_300 gồm hai phần là phần chữ và phần số. Ví dụ:

PIW 304 hoặc M 300.4

- Phần chữ chỉ vị trí và kích thước của ô nhớ. Chúng có thể là:

+ M: Chỉ ô nhớ trong miền các biến cờ có kích thước là 1 bit.

+ MB: Chỉ ô nhớ trong miền các biến cờ có kích thước là 1 byte (8 bits).

+ MW: Chỉ ô nhớ trong miền các biến cờ có kích thước là 2 bytes (16 bits).

+ MD: Chỉ ô nhớ trong miền các biến cờ có kích thước là 4 bytes (32 bits).

+ I: Chỉ ô nhớ có kích thước 1 bit trong miền bộ đệm cổng vào số.

+ IB: Chỉ ô nhớ có kích thước 1 byte trong miền bộ đệm cổng vào số.

+ IW: Chỉ ô nhớ có kích thước 1 từ trong miền bộ đệm cổng vào số.

+ ID: Chỉ ô nhớ có kích thước 2 từ trong miền bộ đệm cổng vào số.

+ Q: Chỉ ô nhớ có kích thước 1 bit trong miền bộ đệm cổng ra số.

+ QB: Chỉ ô nhớ có kích thước 1 byte trong miền bộ đệm cổng ra số.

+ QW: Chỉ ô nhớ có kích thước 1 từ trong miền bộ đệm cổng ra số.

+ QD: Chỉ ô nhớ có kích thước 2 từ trong miền bộ đệm cổng ra số.

+ PIB: Chỉ ô nhớ có kích thước 1 byte trong miền đầu vào ngoại vi (Peripheral Input).

Thường là địa chỉ cổng vào của các module tương tự (I/O External Input).

+ PIW: Chỉ ô nhớ có kích thước 1 từ trong miền đầu vào ngoại vi (Peripheral Input). Thường là địa chỉ cổng vào của các module tương tự (I/O External Input).

+ PID: Chỉ ô nhớ có kích thước 2 từ trong miền đầu vào ngoại vi (Peripheral Input). Thường là địa chỉ cổng vào của các module tương tự (I/O External Input).

+ PQB: Chỉ ô nhớ có kích thước 1 byte trong miền đầu ra ngoại vi (Peripheral Output). Thường là địa chỉ cổng ra của các module tương tự (I/O External Output).

+ PQW: Chỉ ô nhớ có kích thước 1 từ trong miền đầu ra ngoại vi (Peripheral Output). Thường là địa chỉ cổng ra của các module tương tự (I/O External Output).

+ PQD: Chỉ ô nhớ có kích thước 2 từ trong miền đầu ra ngoại vi (Peripheral Output). Thường là địa chỉ cổng ra của các module tương tự (I/O External Output).

+ DBX: Chỉ ô nhớ có kích thước 1 bit trong khối dữ liệu DB được mở bằng lệnh **OPEN DB** (Open Data Block).

+ DBB: Chỉ ô nhớ có kích thước 1 byte trong khối dữ liệu DB được mở bằng lệnh **OPEN DB** (Open Data Block).

+ DBW: Chỉ ô nhớ có kích thước 1 từ trong khối dữ liệu DB được mở bằng lệnh **OPEN DB** (Open Data Block).

+ DBD: Chỉ ô nhớ có kích thước 2 từ trong khối dữ liệu DB được mở bằng lệnh **OPEN DB** (Open Data Block).

+ DBxDBX: Chỉ trực tiếp ô nhớ có kích thước 1 bit trong khối dữ liệu DBx, với x là chỉ số của khối dữ liệu DB. Ví dụ: **DB3.DBX 1**.

+ DBxDBB: Chỉ trực tiếp ô nhớ có kích thước 1 byte trong khối dữ liệu DBx, với x là chỉ số của khối dữ liệu DB. Ví dụ: **DB3.DBB 1**.

+ DBxDBW: Chỉ trực tiếp ô nhớ có kích thước 1 từ trong khối dữ liệu DBx, với x là chỉ số của khối dữ liệu DB. Ví dụ: **DB3.DBW 1**.

+ DBxDBD: Chỉ trực tiếp ô nhớ có kích thước 2 từ trong khối dữ liệu DBx, với x là chỉ số của khối dữ liệu DB. Ví dụ: **DB3.DBD 1**.

+ DIX: Chỉ ô nhớ có kích thước 1 bit trong khối dữ liệu DB, được mở bằng lệnh **OPEN DI** (Open Instance Data Block).

+ DIX: Chỉ ô nhớ có kích thước 1 byte trong khối dữ liệu DB, được mở bằng lệnh **OPEN DI** (Open Instance Data Block).

+ DIW: Chỉ ô nhớ có kích thước 1 từ trong khối dữ liệu DB, được mở bằng lệnh **OPEN DI** (Open Instance Data Block).

+ DID: Chỉ ô nhớ có kích thước 2 từ trong khối dữ liệu DB, được mở bằng lệnh **OPEN DI** (Open Instance Data Block).

+ L: Chỉ ô nhớ có kích thước 1 bit trong miền dữ liệu địa phương (Local Block) của các khối chương trình OB, FC, FB.

+ LB: Chỉ ô nhớ có kích thước 1 byte trong miền dữ liệu địa phương (Local Block) của các khối chương trình OB, FC, FB

+ LW: Chỉ ô nhớ có kích thước 1 từ trong miền dữ liệu địa phương (Local Block) của các khối chương trình OB, FC, FB.

+ LD: Chỉ ô nhớ có kích thước 2 từ trong miền dữ liệu địa phương (Local Block) của các khối chương trình OB, FC, FB

- Phần số chỉ địa chỉ của byte hoặc của bit trong miền nhớ đã xác định.

+ Nếu ô nhớ đã được xác định thông qua phần chữ là có kích thước 1 bit thì phần số sẽ gồm địa chỉ của byte và số thứ tự của bit trong byte đó, được tách với nhau bằng dấu chấm:

I **1.3** // Chỉ bit thứ 3 trong byte 1 của miền nhớ bộ đếm công vào số.

M **101.5** // Chỉ bit thứ 5 trong byte 101 của miền các biến cờ M.

Q **4.5** // Chỉ bit thứ 5

+ Nếu ô nhớ đã được xác định là byte, từ hoặc từ kép thì phần số sẽ là địa chỉ byte đầu tiên trong mảng byte của ô nhớ đó:

DIB **15** // Chỉ ô nhớ có kích thước 1 byte (byte 15) trong khối DB đã được mở bằng lệnh OPN DI

DBW **18** // Chỉ ô nhớ có kích thước 1 từ gồm 2 bytes 18 và 19 trong khối DB đã được mở bằng lệnh OPN DB.

DB2.DBW **15** // Chỉ ô nhớ có kích thước 2 byte 15 và 16 trong khối dữ liệu DB2.

MD **105** // Chỉ ô nhớ có kích thước 2 từ gồm 4 byte 105, 106, 107, 108 trong miền nhớ các biến cờ M.

+ T: Miền nhớ phục vụ Timer bao gồm việc lưu giữ giá trị thời gian đặt trước (PV – Preset Value), giá trị đếm thời gian tức thời (CV – Current Value) cũng như giá trị logic đầu ra của Timer.

+ C: Miền nhớ phục vụ Counter bao gồm việc lưu giữ giá trị đếm đặt trước (PV – Preset Value), giá trị đếm tức thời (CV – Current Value) cũng như giá trị logic đầu ra của Counter.

* Thanh ghi trạng thái

Khi thực hiện lệnh, CPU sẽ ghi nhận lại trạng thái của phép tính trung gian cũng như của kết quả vào một thanh ghi đặc biệt 16 bits, được gọi là thanh ghi trạng thái (Status Word). Mặc dù thanh ghi trạng thái này có độ dài 16 bits nhưng chỉ sử dụng 9 bits với cấu trúc như sau:

8	7	6	5	4	3	2	1	0
BR	CC1	CC0	OV	OS	OR	STA	RLO	FC

- FC (First Check): Khi phải thực hiện một dãy các lệnh logic liên tiếp nhau gồm các phép tính $\wedge$, $\vee$ và nghịch đảo, bit FC có giá trị bằng 1. Nói cách khác, FC = 0 khi dãy lệnh logic tiếp điểm vừa được kết thúc.

A **I0.2** //FC = 1

AN **I0.3** //FC = 1

= **Q4.0** //FC = 0

- RLO (Result of Logic Operation): Kết quả tức thời của phép tính logic vừa được thực hiện.

Ví dụ lệnh:

A **I0.3** //RLO nhận kết quả bit I0.3

+ Nếu trước khi thực hiện bit FC = 0 thì có tác dụng chuyển nội dung của cổng vào số I0.3 vào bit trạng thái RLO.

+ Nếu trước khi thực hiện bit FC = 1 thì có tác dụng thực hiện phép tính $\wedge$ giữa RLO và giá trị logic cổng vào I0.3. Kết quả của phép tính được ghi lại vào bit trạng thái RLO.

- STA (Status Bit) : Bit trạng thái này luôn có giá trị logic của tiếp điểm được chỉ định trong lệnh. Ví dụ cả hai lệnh sau đều gán cho bit STA cùng một giá trị là nội dung của cổng vào số I0.3.

A **I 0.3**

AN **I0.3**

- OR: Ghi lại giá trị của phép tính logic $\wedge$, cuối cùng được thực hiện để phụ giúp cho việc thực hiện phép toán $\vee$ sau đó. Điều này là cần thiết vì trong một biểu thức hàm hai trị, phép tính $\wedge$ bao giờ cũng phải được thực hiện trước các phép tính $\vee$.

- OS (Store Overflow Bit): Ghi lại giá trị bit bị tràn ra ngoài ô nhớ.

- OV (Overflow bit): Bit báo kết quả phép tính bị tràn ra ngoài mảng ô nhớ.

- CC0 và CC1 (Condition Code): Hai bit báo thái của kết quả phép tính với số nguyên, số thực, phép dịch chuyển hoặc phép tính logic trong thanh ghi ACCU.

+ Khi thực hiện các lệnh toán học như cộng, trừ, nhân, chia với số nguyên hoặc số thực.

Bảng 4.1. Giá trị của CC0, CC1 khi thực hiện lệnh toán học

CCC1	CC0	Ý nghĩa
0	0	Kết quả = 0
0	1	Kết quả < 0
1	0	Kết quả > 0

+ Khi thực hiện toán học với số nguyên nhưng kết quả bị tràn ô nhớ.

Bảng 4.2. Giá trị của CC0, CC1 khi thực hiện lệnh toán học với số nguyên bị tràn ô nhớ

CC1	CC0	Ý nghĩa
0	0	Kết quả quá nhỏ khi thực hiện lệnh cộng (+I, +D)
0	1	Kết quả quá nhỏ khi thực hiện lệnh nhân (*I, *D) hoặc quá lớn khi thực hiện lệnh cộng/trừ (+I, +D, -I, -D)
1	0	Kết quả quá lớn khi thực hiện lệnh nhân/chia (*I, *D, /I, /D) hoặc

		quá nhỏ khi thực hiện lệnh cộng/trừ (+I, +D, -I, -D)
1	1	Kết quả bị tràn khi thực hiện lệnh chia cho 0 (/I, /D)

+ Khi thực hiện toán học với số thực nhưng kết quả bị tràn ô nhớ.

Bảng 4.3. Giá trị của CC0, CC1 khi thực hiện lệnh toán học với số thực bị tràn ô nhớ

CCC1	CC0	Ý nghĩa
0	0	Kết quả có số mũ e quá lớn
0	1	Kết quả có mantissa quá nhỏ
1	0	Kết quả có mantissa quá lớn
1	1	Phép tính sai quy chuẩn

+ Khi thực hiện lệnh dịch chuyển.

Bảng 4.4. Giá trị của CC0, CC1 khi thực hiện lệnh dịch chuyển

CCC1	CC0	Ý nghĩa
0	0	Giá trị của bit bị đẩy ra bằng 0
1	0	Giá trị của bit bị đẩy ra bằng 1

+ Khi thực hiện lệnh logic trong ACCU.

Bảng 4.5. Giá trị của CC0, CC1 khi thực hiện lệnh logic trong ACCU

CCC1	CC0	Ý nghĩa
0	0	Kết quả bằng 0
1	0	Kết quả khác 0

- BR (Binary Result Bit): Bit trạng thái cho phép liên kết hai loại ngôn ngữ lập trình STL. Chẳng hạn, cho phép người sử dụng có thể viết một khối chương trình FB hoặc FC trên ngôn ngữ STL nhưng gọi và sử dụng chúng trong một chương trình khác viết trên LAD. Để tạo ra được mối liên kết đó, ta cần phải kết thúc chương trình trong FB, FC bằng lệnh ghi:

+ 1 vào BR, nếu chương trình chạy không có lỗi.

+ 0 vào BR, nếu chương trình chạy có lỗi.

Khi sử dụng các khối hàm đặc biệt của hệ thống (SFC hoặc SFB), trạng thái làm việc của chương trình cũng được thông báo ra ngoài qua bit trạng thái BR như sau:

+ 1 nếu SFC hay SFB thực hiện không có lỗi.

+ 0 nếu có lỗi khi thực hiện SFC hay SFB.

Chú ý: Một chương trình viết trên STL (tùy thuộc vào từng người lập trình) có thể bao gồm nhiều Network. Mỗi một Network chứa một công đoạn cụ thể. Ở mỗi đầu Network, thanh ghi trạng thái nhận giá trị 0, chỉ sau lệnh đầu tiên của Network, các bit trạng thái mới

thay đổi theo kết quả phép tính.

Network 1

...

Đoạn chương trình 1

...

Network 2

...

Đoạn chương trình 2

...

Network 3

...

Đoạn chương trình 3

...

4.2.2. Các lệnh cơ bản

4.2.2.1. Nhóm lệnh logic

A	-	AND
AN	-	AND NOT
O	-	OR
ON	-	OR NOT
X	-	XOR
XN	-	XNOR

* Lệnh gán:

Cú pháp = <toán hạng>

Toán hạng là địa chỉ I, Q, M, L, D.

Lệnh gán giá trị logic của RLO tới ô nhớ có địa chỉ được chỉ thị trong toán hạng. Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau (Kí hiệu “-“ chỉ nội dung bit không bị thay đổi, “x” là bị thay đổi theo lệnh):

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	x	-	1

Ví dụ: Thực hiện $Q4.0 = I0.3$

Network 1

A I0.3 //Đọc nội dung của I0.3 vào RLO

= Q4.0 //Đưa kết quả ra cổng Q4.0

* Lệnh AND

Cú pháp A <toán hạng>

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán giá trị logic của toán hạng vào RLO.

Ngược lại, khi FC = 1, nó sẽ thực hiện phép tính AND giữa RLO với toán hạng và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau (kí hiệu “-“ chỉ nội dung bit không bị thay đổi, “x” là bị thay đổi theo lệnh):

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1

Ví dụ1: Thực hiện $Q4.0 = I0.3 \text{ AND } I0.4$ (mắc nối tiếp hai công tắc)

Network 1

A I0.3 // Đọc nội dung của I0.3 vào RLO

A I0.4 //Kết hợp AND với nội dung cổng I0.4

= Q4.0 //Đưa kết quả ra cổng Q4.0.

*** Lệnh AND NOT**

. Cú pháp **AN** <toán hạng>

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán giá trị logic nghịch đảo của toán hạng vào RLO.

Ngược lại, khi FC = 1 nó sẽ thực hiện phép tính AND giữa RLO với giá trị nghịch đảo của toán hạng và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1

Ví dụ: Thực hiện $Q4.0 = I0.3 \text{ AND NOT } (I0.4)$ (mắc nối tiếp hai công tắc)

Network 1

A I0.3 // Đọc nội dung của I0.3 vào RLO

AN I0.4 //Kết hợp AND với đảo nội dung cổng I0.4

= Q4.0 //Đưa kết quả ra cổng Q4.0

*** Lệnh OR**

. Cú pháp **O** <toán hạng>

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán giá trị logic nghịch đảo của toán hạng vào RLO.

Ngược lại, khi FC = 1, nó sẽ thực hiện phép tính OR giữa RLO với toán hạng và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1

* **Lệnh OR NOT**

. Cú pháp ON <toán hạng>

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán giá trị logic nghịch đảo của toán hạng vào RLO.

Ngược lại, khi FC = 1 nó sẽ thực hiện phép tính OR giữa RLO với giá trị nghịch đảo của toán hạng và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1

* **Lệnh AND với một biểu thức:**

. Cú pháp A (

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán giá trị của biểu thức trong dấu ngoặc vào RLO.

Ngược lại, khi FC = 1 nó sẽ thực hiện phép tính AND giữa RLO với giá trị của biểu thức trong dấu ngoặc và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	1	-	0

* **Lệnh AND với giá trị nghịch đảo của một biểu thức:**

. Cú pháp AN (

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán nghịch đảo giá trị của biểu thức trong dấu ngoặc vào RLO.

Ngược lại, khi FC = 1, nó sẽ thực hiện phép tính AND giữa RLO với giá trị nghịch đảo của biểu thức trong dấu ngoặc và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	1	-	0

* **Lệnh OR với một biểu thức:**

. Cú pháp O (

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán giá trị của biểu thức trong dấu ngoặc vào RLO.

Ngược lại, khi FC = 1 nó sẽ thực hiện phép tính OR giữa RLO với giá trị của biểu thức trong dấu ngoặc và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	1	-	0

*** Lệnh OR với giá trị nghịch đảo của một biểu thức:**

Cú pháp **ON (**

Toán hạng là dữ liệu kiểu BOOL hoặc địa chỉ I, Q, M, L, D, T, C.

Nếu FC = 0 lệnh sẽ gán nghịch đảo giá trị của biểu thức trong dấu ngoặc vào RLO.

Ngược lại, khi FC = 1, nó sẽ thực hiện phép tính OR giữa RLO với giá trị nghịch đảo của biểu thức trong dấu ngoặc và ghi lại kết quả vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	1	-	0

*** Lệnh ghi giá trị logic 1 vào RLO**

Cú pháp **SET**

Lệnh không có toán hạng, có tác dụng ghi 1 vào RLO.

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	-	1	1	0

*** Lệnh gán có điều kiện giá trị logic 1 vào ô nhớ**

Cú pháp **S <toán hạng>**

Toán hạng là địa chỉ bit I, Q, M, L, D.

Nếu RLO = 1, lệnh sẽ ghi giá trị 1 vào ô nhớ có địa chỉ cho trong toán hạng

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	x	-	0

*** Lệnh gán có điều kiện giá trị logic 0 vào ô nhớ**

Cú pháp **R <toán hạng>**

Toán hạng là địa chỉ bit I, Q, M, L, D.

Nếu RLO = 1, lệnh sẽ ghi giá trị 0 vào ô nhớ có địa chỉ cho trong toán hạng

Lệnh tác động vào thanh ghi trạng thái (Status Word) như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	x	-	0

*** Lệnh phát hiện sườn lên**

Cú pháp **FP <toán hạng>**

Toán hạng là địa chỉ bit I, Q, M, L, D.

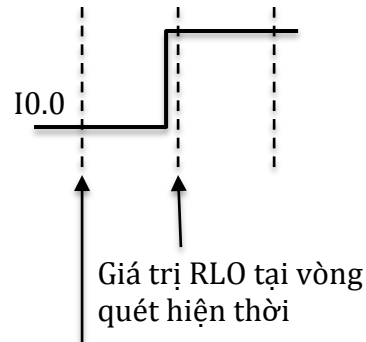
Được sử dụng như một biến cờ để ghi nhận lại giá trị của RLO tại vị trí này trong chương trình, nhưng của vòng quét trước. Tại mỗi vòng lệnh sẽ kiểm tra, nếu biến cờ (toán hạng) có giá trị 0 và RLO có giá trị 1 thì sẽ ghi 1 vào RLO, các trường hợp khác thì ghi 0, đồng thời chuyển nội dung của RLO vào lại biến cờ. Như vậy, RLO sẽ có giá trị 1 trong một vòng quét khi có sườn lên trong RLO.

Ví dụ: Lệnh phát hiện sườn lên:

A **I0.0**
FP **M10.0**
= **Q4.5**

Sẽ tương đương với đoạn chương trình sau:

A **I0.0**
AN **M10.0**
= **Q4.5**
A **I0.0**
= **M10.0**



Giá trị RLO tại vòng quét trước được nhớ vào biến cờ M10.0

Hình 4.1. Minh họa lệnh FP

Lệnh tác động vào thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	x	x	1

*** Lệnh phát hiện sườn xuống**

Cú pháp **FN** <toán hạng>

Toán hạng là địa chỉ bit I, Q, M, L, D.

Được sử dụng như một biến cờ để ghi nhận lại giá trị của RLO tại vị trí này trong chương trình, nhưng của vòng quét trước. Tại mỗi vòng lệnh sẽ kiểm tra, nếu biến cờ (toán hạng) có giá trị 1 và RLO có giá trị 0 thì sẽ ghi 1 vào RLO, các trường hợp khác thì ghi 0, đồng thời chuyển nội dung của RLO vào lại biến cờ. Như vậy, RLO sẽ có giá trị 1 trong một vòng quét khi có sườn xuống trong RLO.

Lệnh tác động vào thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	x	x	1

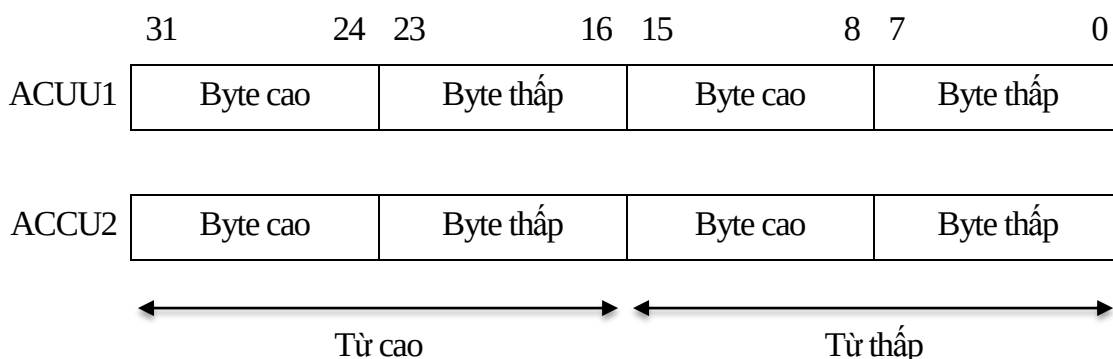
*** Lệnh chuyển giá trị của RLO vào BR:**

Cú pháp **SAVE**

Lệnh chuyển giá trị của RLO vào bit trạng thái BR, lệnh không làm thay đổi nội dung các bit còn lại của thanh ghi trạng thái.

4.2.2.2. Lệnh đọc và ghi trong thanh ghi trạng thái

Các CPU của S7_300 thường có hai thanh ghi *Accumulator* (ACCU) kí hiệu là ACCU1 và ACCU2. Hai thanh ghi ACCU có cùng kích thước 32 bits (1 từ kép). Mọi phép tính toán trên số thực, số nguyên, các phép tính logic với mảng đều được thực hiện trên hai thanh ghi này.



Hình 4.2. Cấu tạo thanh ghi ACCU trong S7-300

* Lệnh đọc vào ACCU:

Cú pháp L <Toán hạng>

- Toán hạng là số liệu (số nguyên, thực, nhị phân) hoặc địa chỉ.

Bảng 4.6 Các dạng hợp lệ trong thanh ghi ACCU

Dữ liệu	Ví dụ	Giải thích
± ...	L 5	Ghi 5 vào từ thấp của ACCU1
B# (... ,...)	L B# (1 , 8)	Ghi 1 vào byte cao của từ thấp và 8 vào byte thấp của từ thấp trong ACCU1
L# ...	L L#5	Ghi 5 vào ACCU1(số nguyên 32 bits)
16# ...	L B#16#3B L W#2FD5 L DW#2C3E_3AB2	Dữ liệu dạng cơ số 16
2# ...	L 2#10011110	Dữ liệu dạng cơ số 2
'...'	L 'ABCD' L 'DE'	Dữ liệu dạng kí tự
C# ...	L C#500	Dữ liệu là giá trị đặt trước cho Counter (PV)
S5T# ...	L S5T#2h:20m	Dữ liệu là giá trị đặt trước cho (PV)
P# ...	L Q#M10.0	Dữ liệu là địa chỉ ô nhớ (dùng cho con trỏ)
D# ...	L D#2014-12-3	Dữ liệu là giá trị ngày-tháng-năm (16 bits)
TOD# ...	L TOD#2:30:13	Dữ liệu là giá trị giờ-phút-giây (32 bits)

Nếu là dữ liệu:

+ Byte: IB, QB, PIB, MB, LB, DBB, DIB trong khoảng 0 ÷ 65535

+ Từ: IW, QW, PIW, MW, LW, DBW, DIW trong khoảng 0 ÷ 65534

+ Từ kép: ID, QD, PID, MD, LD, DBD, DID trong khoảng 0 ÷ 65534

Nếu là dữ liệu thì các dạng hợp lệ được liệt kê trong bảng 4.6.

Lệnh L có tác dụng chuyển dữ liệu hoặc nội dung của ô nhớ có địa chỉ là toán hạng vào thanh ghi ACCU1. Nội dung cũ của ACCU1 được chuyển vào ACCU2. Trong trường hợp giá trị chuyển vào có kích thước nhỏ hơn từ kép thì chúng sẽ được ghi vào theo thứ tự byte thấp của từ thấp, byte cao của từ thấp, byte thấp của từ cao, byte cao của từ cao. Những bit còn trống trong ACCU1 được ghi 0.

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

Ví dụ:

L IB0

Chuyển nội dung của IB0 vào ACCU1 như sau:

	31	24	23	16	15	8	7	0
ACCU1	0	0	0	0	0	0	0	0

L MW20

Chuyển nội dung của MW20 gồm 2 byte MB20, MB21 vào ACCU1 như sau:

	31	24	23	16	15	8	7	0
ACCU1	0	0	0	0	0	0	0	0

* Lệnh chuyển nội dung ACCU tới ô nhớ

Cú pháp **T <Toán hạng>**

- Toán hạng là địa chỉ:

+ Byte: IB, QB, PIB, MB, LB, DBB, DIB trong khoảng 0 ÷ 65535

+ Từ: IW, QW, PIW, MW, LW, DBW, DIW trong khoảng 0 ÷ 65534

+ Từ kép: ID, QD, PID, MD, LD, DBD, DID trong khoảng 0 ÷ 65534

Lệnh chuyển nội dung của ACCU1 vào ô nhớ có địa chỉ ghi trong toán hạng. Lệnh không ảnh hưởng đến nội dung của ACCU2. Trong trường hợp ô nhớ có kích thước nhỏ hơn từ kép thì nội dung của ACCU1 sẽ được chuyển ra theo thứ tự byte thấp của từ thấp, byte cao của từ thấp, byte thấp của từ cao, byte cao của từ cao.

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

Ví dụ:

T QB0

Sẽ chuyển nội dung của byte thấp của từ thấp trong ACCU1 vào QB0.

T QW0

Sẽ chuyển nội dung byte cao của từ thấp vào QW0, byte thấp của từ thấp vào QW1.

* Lệnh đọc nội dung của thanh ghi trạng thái vào ACCU1

Cú pháp **L STW**

Lệnh chuyển nội dung của thanh ghi trạng thái vào từ thấp của ACCU1.

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

* Lệnh ghi nội dung của ACCU1 vào thanh ghi trạng thái

Cú pháp **T** **STW**

Lệnh chuyển 9 bits của từ thấp trong ACCU1 vào thanh ghi trạng thái.

* Lệnh chuyển nội dung của ACCU2 vào ACCU1

Cú pháp **POP**

Lệnh không có toán hạng, không ảnh hưởng đến nội dung trong ACCU2

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

* Lệnh chuyển nội dung của ACCU1 vào ACCU2

Cú pháp **PUSH**

Lệnh không có toán hạng, không ảnh hưởng đến nội dung trong ACCU1

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

* Lệnh đảo nội dung của ACCU1 và ACCU2

Cú pháp **TAK**

Lệnh không có toán hạng, nội dung trong ACCU1 được ghi vào ACCU2 và ngược lại, nội dung trong ACCU2 được chuyển vào ACCU1.

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

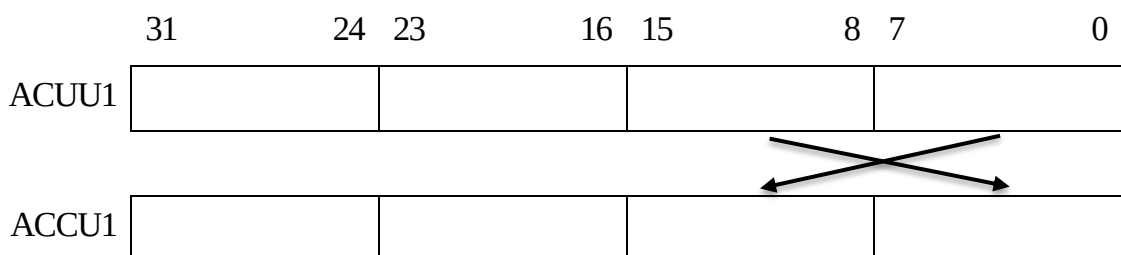
* Lệnh đảo nội dung hai bytes của từ thấp trong ACCU1

Cú pháp **CAW**

Lệnh không có toán hạng.

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

Lệnh này được minh hoạ trên hình 4.3.



Hình 4.3. Minh hoạ lệnh CAW

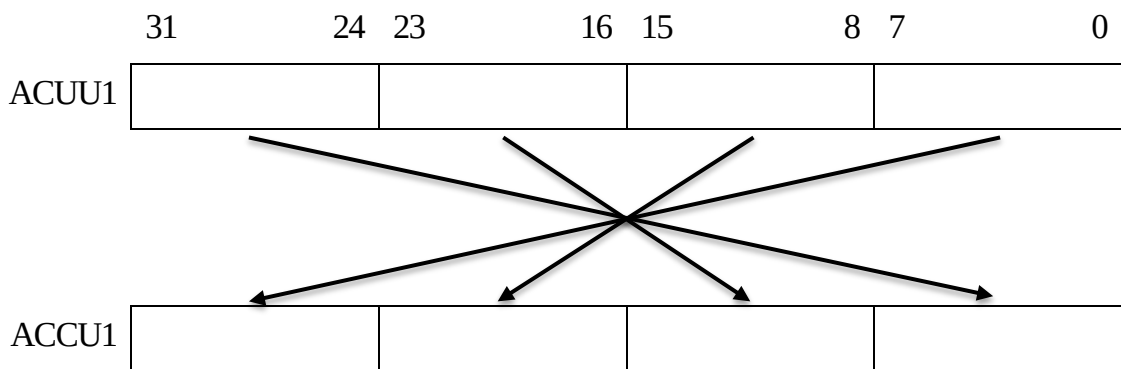
* Lệnh đảo nội dung các bytes trong ACCU1

Cú pháp **CAD**

Lệnh không có toán hạng.

Lệnh không ảnh hưởng đến thanh ghi trạng thái.

Lệnh này được minh hoạ trên hình 4.4.



Hình 4.4. Minh hoạ lệnh CAD

4.2.2.4 Nhóm lệnh so sánh số nguyên 16 bits

Tất cả các lệnh so sánh số nguyên 16 bits đều nằm trong từ thấp của hai thanh ghi ACCU1 và ACCU2, và điều tác động vào thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	x	x	0	-	0	x	x	1

Trong đó hai bits CC1 và CC0 thay đổi theo quy tắc như bảng 4.7.

Bảng 4.7. Quy tắc thay đổi của CC0 và CC1 với nhóm lệnh số nguyên 16 bits

CC1	CC0	Ý nghĩa
0	0	Từ thấp ACCU2 = Từ thấp ACCU1
0	1	Từ thấp ACCU2 < Từ thấp ACCU1
1	0	Từ thấp ACCU2 > Từ thấp ACCU1

* Lệnh so sánh bằng nhau hai số nguyên 16 bits

Cú pháp ==I

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 16 bits trong từ thấp hai thanh ghi ACCU1 và ACCU2. Nếu chúng giống nhau thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh không bằng nhau hai số nguyên 16 bits

Cú pháp <>I

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 16 bits trong từ thấp hai thanh ghi ACCU1 và ACCU2. Nếu chúng khác nhau thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh lớn hơn hai số nguyên 16 bits

Cú pháp >I

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 16 bits trong từ thấp hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong từ thấp của ACCU2 lớn hơn ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh nhỏ hơn hai số nguyên 16 bits

Cú pháp <I

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 16 bits trong từ thập hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong từ thập của ACCU2 nhỏ hơn ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh lớn hơn hoặc bằng hai số nguyên 16 bits

Cú pháp >=I

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 16 bits trong từ thập hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong từ thập của ACCU2 lớn hơn hoặc bằng ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh nhỏ hơn hoặc bằng hai số nguyên 16 bits

Cú pháp <=I

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 16 bits trong từ thập hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong từ thập của ACCU2 nhỏ hơn hoặc bằng ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

4.2.2.5. Nhóm lệnh so sánh số nguyên 32 bits

Tất cả các lệnh so sánh số nguyên 32 bits của hai thanh ghi ACCU1 và ACCU2 đều tác động vào thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	x	x	0	-	0	x	x	1

Trong đó hai bits CC1 và CC0 thay đổi theo quy tắc như bảng 4.8.

Bảng 4.8. Quy tắc thay đổi của CC0 và CC1 với nhóm lệnh số nguyên 32 bits

CC1	CC0	Ý nghĩa
0	0	ACCU2 = ACCU1
0	1	ACCU2 < ACCU1
1	0	ACCU2 > ACCU1

* Lệnh so sánh bằng nhau hai số nguyên 32 bits

Cú pháp ==D

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu chúng giống nhau thì RLO = 1, ngược lại thì RLO = 0.

Ví dụ: Viết chương trình báo đèn Q1.0 sáng nếu số nguyên trong ô nhớ MD10 bằng 100.

L MD10

L **100**

=D

= **Q1.0**

* Lệnh so sánh không bằng nhau hai số nguyên 32 bits

Cú pháp **<>D**

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu chúng khác nhau thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh lớn hơn hai số nguyên 32 bits

Cú pháp **>D**

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong ACCU2 lớn hơn ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh nhỏ hơn hai số nguyên 32 bits

Cú pháp **<D**

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong ACCU2 nhỏ hơn ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh lớn hơn hoặc bằng hai số nguyên 32 bits

Cú pháp **>=D**

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong ACCU2 lớn hơn hoặc bằng ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

* Lệnh so sánh nhỏ hơn hoặc bằng hai số nguyên 32 bits

Cú pháp **<=D**

Lệnh không có toán hạng.

Lệnh so sánh hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu số nguyên trong ACCU2 nhỏ hơn hoặc bằng ACCU1 thì RLO = 1, ngược lại thì RLO = 0.

4.2.2.6. Nhóm lệnh so sánh số thực 32 bits

Tất cả các lệnh so sánh số thực 32 bits của hai thanh ghi ACCU1 và ACCU2 đều tác động vào thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	x	x	x	x	0	x	x	1

Trong đó hai bits CC1 và CC0 thay đổi theo quy tắc như bảng 4.9

Bảng 4.9. Quy tắc thay đổi của CC0 và CC1 với nhóm lệnh số thực

CC1	CC0	Ý nghĩa
0	0	$ACCU2 = ACCU1$
0	1	$ACCU2 < ACCU1$
1	0	$ACCU2 > ACCU1$

* Lệnh so sánh bằng nhau hai số thực 32 bits

Cú pháp $\quad ==R$

Lệnh không có toán hạng.

Lệnh so sánh hai số thực 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu chúng giống nhau thì $RLO = 1$, ngược lại thì $RLO = 0$.

* Lệnh so sánh không bằng nhau hai số thực 32 bits

Cú pháp $\quad <>R$

Lệnh không có toán hạng.

Lệnh so sánh hai số thực 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu chúng khác nhau thì $RLO = 1$, ngược lại thì $RLO = 0$.

* Lệnh so sánh lớn hơn hai số thực 32 bits

Cú pháp $\quad >R$

Lệnh không có toán hạng.

Lệnh so sánh hai số thực 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu số thực trong ACCU2 lớn hơn ACCU1 thì $RLO = 1$, ngược lại thì $RLO = 0$.

* Lệnh so sánh nhỏ hơn hai số thực 32 bits

Cú pháp $\quad <R$

Lệnh không có toán hạng.

Lệnh so sánh hai số thực 32 bits trong thanh ghi ACCU1 và ACCU2. Nếu số thực trong ACCU2 nhỏ hơn ACCU1 thì $RLO = 1$, ngược lại thì $RLO = 0$.

* Lệnh so sánh lớn hơn hoặc bằng hai số thực 32 bits

Cú pháp $\quad >=R$

Lệnh không có toán hạng.

Lệnh so sánh hai số thực 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu số thực trong ACCU2 lớn hơn hoặc bằng ACCU1 thì $RLO = 1$, ngược lại thì $RLO = 0$.

* Lệnh so sánh nhỏ hơn hoặc bằng hai số thực 32 bits

Cú pháp $\quad <=R$

Lệnh không có toán hạng.

Lệnh so sánh hai số thực 32 bits trong hai thanh ghi ACCU1 và ACCU2. Nếu số thực trong ACCU2 nhỏ hơn hoặc bằng ACCU1 thì $RLO = 1$, ngược lại thì $RLO = 0$.

4.2.3 Các lệnh toán học

Tất cả các lệnh thực hiện với nội dung hai thanh ghi ACCU1 và ACCU2 đều tác động vào thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	X	X	X	X	-	-	-	-

Trong đó hai bits CC1 và CC0 thay đổi theo quy tắc như bảng 4.10.

Bảng 4.10. Quy tắc thay đổi của CC0 và CC1 với các lệnh toán học

CC1	CC0	Ý nghĩa
0	0	Kết quả = 0
0	1	Kết quả < 0
1	0	Kết quả > 0

4.2.3.1. Nhóm lệnh làm việc với số nguyên 16 bits

* Lệnh cộng

Cú pháp **+I**

Lệnh không có toán hạng.

Lệnh thực hiện phép cộng hai số nguyên 16 bits trong từ thập hai thanh ghi ACCU1 và ACCU2, kết quả được ghi vào từ thập của ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-32768 \div 32767$ thì bit OV có giá trị bằng 0, ngược lại 2 bit OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

Ví dụ: Cộng hai số nguyên 16 bits chứa trong IW10 và IW20, cất kết quả vào ô nhớ DBW20 của khối dữ liệu DB1.

L **IW10**

L **IW20**

+I

T **DB1 . DBW20**

* Lệnh trừ

Cú pháp **-I**

Lệnh không có toán hạng.

Lệnh thực hiện phép trừ số nguyên 16 bits trong từ thập của thanh ghi ACCU2 cho số nguyên 16 bits trong từ thập của thanh ghi ACCU1. Kết quả được ghi vào từ thập của ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-32768 \div 32767$ thì bit OV có giá trị bằng 0, ngược lại 2 bit OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh nhân

Cú pháp ***I**

Lệnh không có toán hạng.

Lệnh thực hiện phép nhân hai số nguyên 16 bits trong từ thấp hai thanh ghi ACCU1 và ACCU2, kết quả được ghi vào từ thấp của ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-32768 \div 32767$ thì bit OV có giá trị bằng 0, ngược lại 2 bit OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh chia

Cú pháp /I

Lệnh không có toán hạng.

Lệnh thực hiện phép chia số nguyên 16 bits trong từ thấp của thanh ghi ACCU2 cho số nguyên 16 bits trong từ thấp của thanh ghi ACCU1. Kết quả là một số nguyên 16 bits sẽ được ghi vào từ thấp của ACCU1, phần dư của phép chia được ghi vào từ cao trong thanh ghi ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-32768 \div 32767$ thì bit OV có giá trị bằng 0, ngược lại 2 bit OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

4.2.3.2. Nhóm lệnh làm việc với số nguyên 32 bits

* Lệnh cộng

Cú pháp +D

Lệnh không có toán hạng.

Lệnh thực hiện phép cộng hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2, kết quả được ghi vào ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-2147483648 (-2^{31}) \div 2147483647 (2^{31}-1)$ thì bit OV có giá trị bằng 0, ngược lại 2 bit OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh trừ

Cú pháp -D

Lệnh không có toán hạng.

Lệnh thực hiện phép trừ số nguyên 32 bits trong thanh ghi ACCU2 cho số nguyên 32 bits trong thanh ghi ACCU1. Kết quả được ghi vào ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-2147483648 (-2^{31}) \div 2147483647 (2^{31}-1)$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh nhân

Cú pháp *D

Lệnh không có toán hạng.

Lệnh thực hiện phép nhân hai số nguyên 32 bits trong hai thanh ghi ACCU1 và ACCU2, kết quả được ghi vào ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-2147483648 (-2^{31}) \div 2147483647 (2^{31}-1)$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh chia

Cú pháp /D

Lệnh không có toán hạng.

Lệnh thực hiện phép chia số nguyên 32 bits trong thanh ghi ACCU2 cho số nguyên 32 bits trong thanh ghi ACCU1. Kết quả là một số nguyên 32 bits sẽ được ghi vào ACCU1. Nếu kết quả nằm trong khoảng -2^{31} ÷ $2^{31}-1$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh lấy phần dư

Cú pháp MOD

Lệnh không có toán hạng và xác định phần dư của phép chia số nguyên 32 bits trong thanh ghi ACCU2 cho số nguyên 32 bits trong thanh ghi ACCU1. Kết quả là một số nguyên 32 bits sẽ được ghi vào ACCU1. Nếu kết quả nằm trong khoảng -2^{31} ÷ $2^{31}-1$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

4.2.3.3. Nhóm lệnh làm việc với số thực 32 bits

* Lệnh cộng

Cú pháp +R

Lệnh không có toán hạng.

Lệnh thực hiện phép cộng hai số thực dấu phẩy động nằm trong hai thanh ghi ACCU1 và ACCU2, kết quả được ghi vào ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-3.402823E+38$ ÷ $-1.175495E-38$ hoặc $1.175495E-38$ ÷ $3.402823E+38$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh trừ

Cú pháp -R

Lệnh không có toán hạng.

Lệnh thực hiện phép trừ số thực 32 bits trong thanh ghi ACCU2 cho số thực 32 bits trong thanh ghi ACCU1. Kết quả được ghi vào ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-3.402823E+38$ ÷ $-1.175495E-38$ hoặc $1.175495E-38$ ÷ $3.402823E+38$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

Ví dụ: Thực hiện phép trừ hai số thực MD10 - MD14, kết quả cất vào MD20:

L MD10

L MD14

-R

T MD20

* Lệnh nhân

Cú pháp *R

Lệnh không có toán hạng.

Lệnh thực hiện phép nhân hai số thực 32 bits trong hai thanh ghi ACCU1 và ACCU2, kết

quả được ghi vào ACCU1, nội dung thanh ghi ACCU2 không thay đổi. Nếu kết quả nằm trong khoảng $-3.402823E+38 \div -1.175495E-38$ hoặc $1.175495E-38 \div 3.402823E+38$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh chia

Cú pháp /R

Lệnh không có toán hạng.

Lệnh thực hiện phép chia số thực 32 bits trong thanh ghi ACCU2 cho số thực 32 bits trong thanh ghi ACCU1. Kết quả là một số thực 32 bits sẽ được ghi vào ACCU1. Nếu kết quả nằm trong khoảng $-3.402823E+38 \div -1.175495E-38$ hoặc bằng 0 hoặc nằm trong khoảng $1.175495E-38 \div 3.402823E+38$ thì bit OV có giá trị bằng 0, ngược lại 2 bits OV và OS trong thanh ghi trạng thái có giá trị bằng 1.

* Lệnh lấy giá trị tuyệt đối

Cú pháp ABS

Lệnh này không có toán hạng, nó xác định giá trị tuyệt đối của số thực trong ACCU1, kết quả ghi lại vào ACCU1. Lệnh này không làm thay đổi các bit trạng thái.

2.2.4. Lệnh logic tiếp điểm trên thanh ghi trạng thái

Do các lệnh toán học ở phần trên khi thực hiện không làm thay đổi nội dung bit RLO trong thanh ghi trạng thái nên nó được kết hợp với lệnh logic như AND, OR... dưới dạng lệnh logic tiếp điểm trên thanh ghi trạng thái.

Các lệnh này tác động lên thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1

2.2.4.1. Lệnh AND

* Lệnh AND nhỏ hơn

Cú pháp A <0

Lệnh tính $RLO \wedge CC0 \wedge \overline{CC1}$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có nhỏ hơn 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\wedge$ với RLO.

Ví dụ: Nếu số nguyên 16 bits x trong MW10 thỏa mãn $-2 \leq x < 3$ thì báo đèn Q4.0 sáng.

L -2

L MW10

<=I

L 3

-I

A <0

=Q4.0

* Lệnh AND lớn hơn

Cú pháp **A** **>0**

Lệnh tính $RLO \wedge \overline{CC0} \wedge CC1$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có lớn hơn 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\wedge$ với RLO.

* Lệnh AND khác nhau

Cú pháp **A** **<>0**

Lệnh tính $RLO \wedge [(\overline{CC0} \wedge CC1) \vee (CC0 \wedge \overline{CC1})]$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có khác 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\wedge$ với RLO.

* Lệnh AND bằng nhau

Cú pháp **A** **=0**

Lệnh tính $RLO \wedge \overline{CC0} \wedge \overline{CC1}$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có bằng 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\wedge$ với RLO.

* Lệnh AND lớn hơn hoặc bằng

Cú pháp **A** **>=0**

Lệnh tính $RLO \wedge \overline{CC0}$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có lớn hơn hoặc bằng 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\wedge$ với RLO.

* Lệnh AND nhỏ hơn hoặc bằng

Cú pháp **A** **<=0**

Lệnh tính $RLO \wedge \overline{CC1}$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có nhỏ hơn hoặc bằng 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\wedge$ với RLO.

2.2.4.2. Lệnh OR

* Lệnh OR nhỏ hơn

Cú pháp **O** **<0**

Lệnh tính $RLO \vee (CC0 \wedge \overline{CC1})$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có nhỏ hơn 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\vee$ với RLO.

Ví dụ: Nếu số nguyên 16 bits x trong MW10 thỏa mãn $x < -1$ hoặc $x > 2$ thì báo đèn Q4.0 sáng.

L **2**

L **MW10**

<I

L **-1**

-I

O <0

= Q4.0

* Lệnh OR lớn hơn

Cú pháp O >0

Lệnh tính RLO $\vee (\overline{CC0} \wedge CC1)$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có lớn hơn 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\vee$ với RLO.

* Lệnh OR khác nhau

Cú pháp O <>0

Lệnh tính RLO $\vee [(\overline{CC0} \wedge CC1) \vee (CC0 \wedge \overline{CC1})]$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có khác 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\vee$ với RLO.

* Lệnh OR bằng nhau

Cú pháp O ==0

Lệnh tính RLO $\vee (\overline{CC0} \wedge \overline{CC1})$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có bằng 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\vee$ với RLO.

* Lệnh OR lớn hơn hoặc bằng

Cú pháp O >=0

Lệnh tính RLO $\vee \overline{CC0}$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có lớn hơn hoặc bằng 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\vee$ với RLO.

* Lệnh OR nhỏ hơn hoặc bằng

Cú pháp O <=0

Lệnh tính RLO $\vee \overline{CC1}$, kết quả được ghi lại vào RLO. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có nhỏ hơn hoặc bằng 0 hay không (0 – sai, 1 – đúng) rồi thực hiện phép $\vee$ với RLO.

2.2.4.3. Lệnh EXCLUSIVE OR (X)

* Lệnh X nhỏ hơn

Cú pháp X <0

Lệnh đảo nội dung RLO nếu $CC0 \wedge \overline{CC1} = 1$. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có nhỏ hơn 0 hay không, nếu kết quả kiểm tra đúng thì đảo nội dung bit RLO.

Ví dụ: Nếu số nguyên 16 bits x trong MW10 thỏa mãn $x > 2$ thì đảo trạng thái đèn Q4.0.

A Q4.0

L 2

L **MW10**

-I

X **<0**

= **Q4.0**

* Lệnh X lớn hơn

Cú pháp **X** **>0**

Lệnh đảo nội dung RLO nếu $\overline{CC0} \wedge CC1 = 1$. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có lớn hơn 0 hay không, nếu kết quả kiểm tra đúng thì đảo nội dung bit RLO.

* Lệnh X khác nhau

Cú pháp **X** **<>0**

Lệnh đảo nội dung RLO nếu $\overline{CC0} \wedge CC1 = 1$ hoặc $CC0 \wedge \overline{CC1} = 1$. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có khác 0 hay không, nếu kết quả kiểm tra đúng thì đảo nội dung bit RLO.

* Lệnh X bằng nhau

Cú pháp **X** **==0**

Lệnh đảo nội dung RLO nếu $\overline{CC0} \wedge \overline{CC1} = 1$. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có bằng 0 hay không, nếu kết quả kiểm tra đúng thì đảo nội dung bit RLO.

* Lệnh X lớn hơn hoặc bằng

Cú pháp **X** **>=0**

Lệnh đảo nội dung RLO nếu $\overline{CC0} = 1$. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có lớn hơn bằng 0 hay không, nếu kết quả kiểm tra đúng thì đảo nội dung bit RLO.

* Lệnh X nhỏ hơn hoặc bằng

Cú pháp **X** **<=0**

Lệnh đảo nội dung RLO nếu $\overline{CC1} = 1$. Như vậy, lệnh kiểm tra phép tính vừa thực hiện (cộng, trừ, nhân, chia...) có nhỏ hơn bằng 0 hay không, nếu kết quả kiểm tra đúng thì đảo nội dung bit RLO.

4.2.5. Các lệnh điều khiển chương trình

4.2.5.1. Nhóm lệnh kết thúc chương trình

* Lệnh kết thúc vô điều kiện

Cú pháp **BEU**

Lệnh không có toán hạng, thực hiện kết thúc chương trình trong khối một cách vô điều kiện. Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	0	0	1	-	0

* Lệnh kết thúc có điều kiện

Cú pháp **BEC**

Lệnh không có toán hạng, thực hiện kết thúc chương trình trong khối nếu RLO có giá trị 1.
Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	x	0	1	1	0

Ví dụ: Chương trình sẽ thay đổi trạng thái tín hiệu công ra Q1.0 ở vòng quét có số thứ tự chia hết cho 500 và không làm gì ở các vòng quét khác.

```

L      1
L      MW10 //thanh ghi đếm số vòng quét
+I
T      MW10
L      500
<>I
BEC //kết thúc nếu chưa phải là vòng quét thứ 500
L      0
T      MW10 // nạp lại số vòng quét từ đầu
AN     Q1.0
=      Q1.0
BEU

```

4.2.5.2. Nhóm lệnh rẽ nhánh theo bit trạng thái

Đây là loại lệnh thực hiện bước nhảy nhằm bỏ qua một đoạn chương trình để nhảy đến một đoạn chương trình khác được đánh dấu bằng “Nhãn” nếu điều kiện kiểm tra trong thanh ghi trạng thái được thoả mãn. Nơi nhảy tới phải thuộc cùng một khối chương trình với lệnh, không thể nhảy từ khối chương trình này sang khối chương trình khác.

Nhãn là một dãy với nhiều nhất là 4 ký tự hoặc số và phải được bắt đầu bằng một ký tự. Khoảng cách bước nhảy tính theo ô nhớ chương trình, phải có ít hơn 32767 từ. Nơi nhảy đến có thể nằm trước hoặc sau lệnh nhảy. Hình 4.5 mô tả phương thức làm việc của lệnh rẽ nhánh theo bit trạng thái tới địa điểm “Nhãn” trong chương trình.

* Rẽ nhánh khi BR=1

Cú pháp **JB** <nhãn>

Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

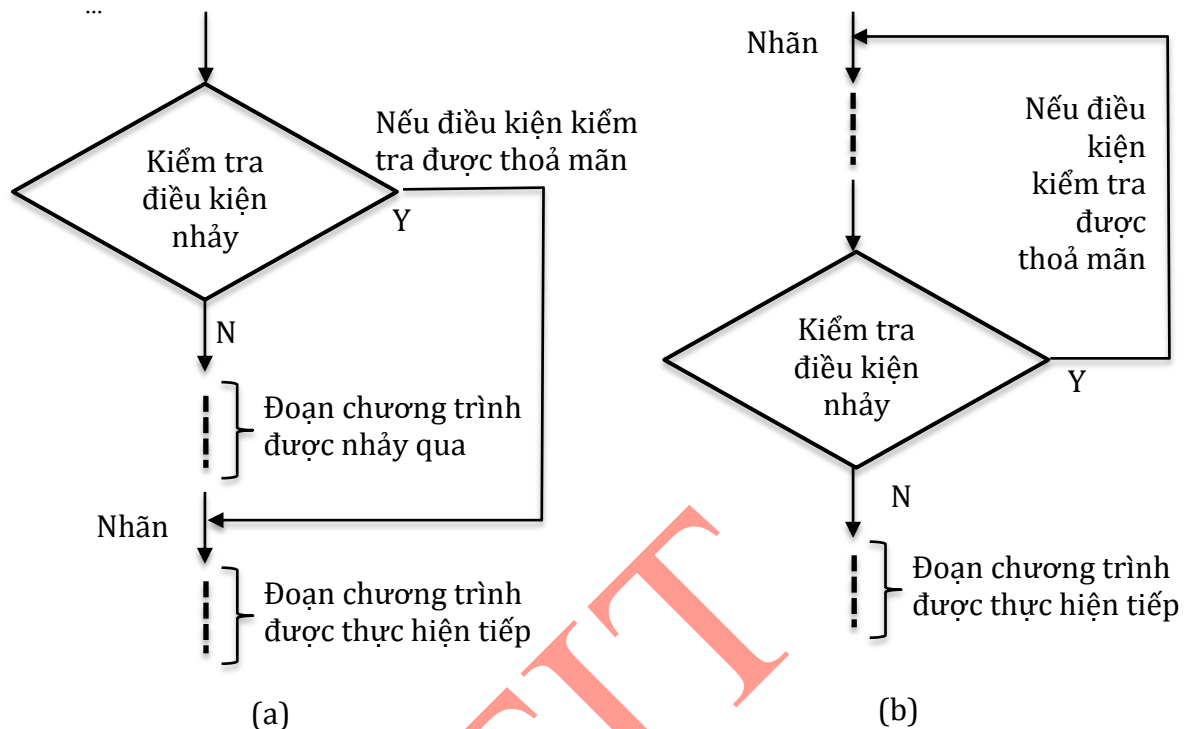
BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	1	-	0

Ví dụ:

```

JBI    next //nhảy đến next nếu BR=1
...
next:  A    I0.0
...

```



Hình 4.5. Lệnh rẽ nhánh theo bit trạng thái (a) nhảy xuôi và (b) nhảy ngược.

* Rẽ nhánh khi BR=0

Cú pháp **JNBI** <nhãn>

Lệnh thay đổi nội dung thanh ghi trạng thái giống lệnh JBI.

Ví dụ:

```

next:  A    I0.0
...
JNBI    next //nhảy đến next nếu BR=0
...

```

* Rẽ nhánh khi RLO=1

Cú pháp **JC** <nhãn>

Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	1	1	0

Ví dụ:

```

JC    next //nhảy đến next nếu RLO=1
...
next:  A    I0.0

```

...

* Rẽ nhánh khi $RLO=0$

Cú pháp **JCN** <nhãn>

Lệnh thay đổi nội dung thanh ghi trạng thái giống lệnh JC.

Ví dụ:

JCN **next** //nhảy đến next nếu $RLO=0$

...

next: **A** **IO.0**

...

* Rẽ nhánh khi $CC1=0$ và $CC0=1$

Cú pháp **JM** <nhãn>

Lệnh không làm thay đổi nội dung thanh ghi trạng thái, nó được sử dụng để rẽ nhánh khi phép tính trước đó có kết quả âm.

next: **A** **IO.0**

...

JM next //nhảy đến next nếu nội dung của ACCU1 là một số âm

...

* Rẽ nhánh khi $CC1=1$ và $CC0=0$

Cú pháp **JP** <nhãn>

Lệnh không làm thay đổi nội dung thanh ghi trạng thái, nó được sử dụng để rẽ nhánh khi phép tính trước đó có kết quả dương.

* Rẽ nhánh khi $CC1=CC0=0$

Cú pháp **JZ** <nhãn>

Lệnh không làm thay đổi nội dung thanh ghi trạng thái, nó được sử dụng để rẽ nhánh khi phép tính trước đó có kết quả bằng 0.

* Rẽ nhánh khi $CC1 \neq CC0$

Cú pháp **JN** <nhãn>

Lệnh không làm thay đổi nội dung thanh ghi trạng thái, nó được sử dụng để rẽ nhánh khi phép tính trước đó có kết quả khác 0.

* Rẽ nhánh khi $CC1=CC0=0$ hoặc $CC1=0$ và $CC0=1$

Cú pháp **JMZ** <nhãn>

Lệnh không làm thay đổi nội dung thanh ghi trạng thái, nó được sử dụng để rẽ nhánh khi phép tính trước đó có kết quả không dương (nhỏ hơn hoặc bằng 0).

* Rẽ nhánh khi $CC1=CC0=0$ hoặc $CC1=1$ và $CC0=0$

Cú pháp **JPZ** <nhãn>

Lệnh không làm thay đổi nội dung thanh ghi trạng thái, nó được sử dụng để rẽ nhánh khi phép tính trước đó có kết quả không âm (lớn hơn hoặc bằng 0).

* Rẽ nhánh vô điều kiện

Cú pháp JU <nhãn>

Lệnh không làm thay đổi nội dung thanh ghi trạng thái, nó được thực hiện một cách vô điều kiện mà không phụ thuộc bất cứ bit trạng thái nào.

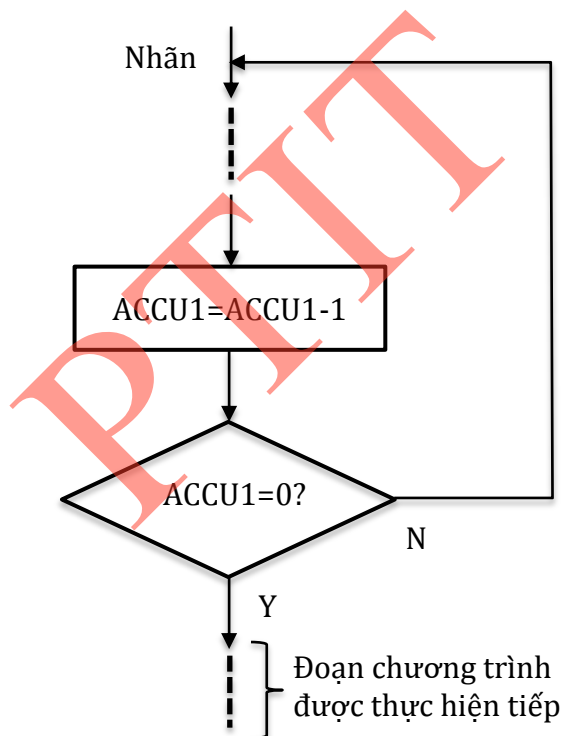
4.2.5.3. Lệnh xoay vòng (LOOP)

Cú pháp LOOP <nhãn>

Khi gặp lệnh LOOP, CPU của S7 – 300 sẽ tự động giảm nội dung của từ thấp trong thanh ghi ACCU1 xuống 1 đơn vị và kiểm tra kết quả có bằng 0 hay không? Nếu kết quả khác 0, CPU sẽ thực hiện bước nhảy đến đoạn chương trình được đánh dấu bởi “nhãn”. Ngược lại thì CPU sẽ thực hiện lệnh kế tiếp.

Lệnh xoay vòng này có thể được sử dụng để mô phỏng nguyên tắc làm việc giống lệnh ‘for’ trong ngôn ngữ lập trình C bằng cách thực hiện bước nhảy ngược (hình 4.6). Đoạn chương trình nằm giữa “nhãn” và lệnh LOOP sẽ được thực hiện cho tới khi nội dung thanh ghi ACCU1 bằng 0.

Lệnh này không làm thay đổi nội dung thanh ghi trạng thái.



Hình 4.6. Nguyên tắc làm việc của lệnh LOOP

Ví dụ: Viết chương trình mô phỏng thực hiện lệnh xoay vòng 10 lần đoạn chương trình nằm giữa nhãn “next” và lệnh LOOP.

```
L          10
next:      T      MW10
...
// Đoạn chương trình thực hiện lệnh LOOP
...
L          MW10
```


LOOP next

4.3. Ngôn ngữ Ladder (LAD)

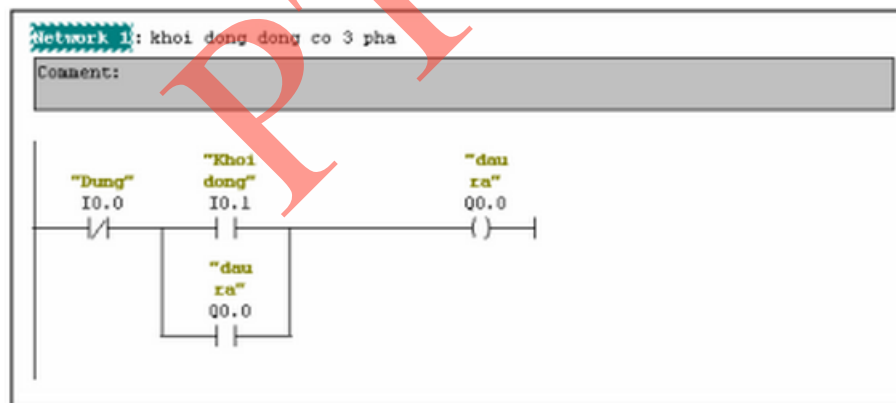
4.3.1. Đặc điểm

Đây là ngôn ngữ lập trình “hình thang”, dạng ngôn ngữ đồ họa, thích hợp với những người quen thiết kế mạch logic. Phương pháp LAD biểu thị các chức năng điều khiển bằng các loại ký hiệu sơ đồ mạch như tiếp điểm, Timer, Counter... Phương pháp này có tính trực quan cao vì nó biểu diễn mạch điện tương tự mạch điều khiển rơ le.

Trong các mạch điều khiển logic, các phần tử logic kiểu rơ le được nối với nhau theo sơ đồ thiết kế để thực hiện hàm điều khiển xác định, gọi là các mạch logic nối dây cứng (Hard – wired Logic). Biểu diễn mạch logic nối dây cứng bằng giản đồ hình thang (Ladder) và các phần tử rơ le. Ngôn ngữ này thích hợp với người quen thiết kế mạch điều khiển logic.

Hình 4.7 mô phỏng chương trình khởi động động cơ ba pha, nó tương đương với đoạn lệnh được viết theo ngôn ngữ STL:

```
AN I      0.0      Dng
A(
O      I      0.1      Khoi dong
O      Q      0.0      dau ra
)
=      Q      0.0      dau ra
```



Hình 4.7. Chương trình khởi động động cơ ba pha

4.3.2 Tập lệnh trong S7-300

PLC là một bộ vi xử lý, có thể xử lý các lệnh như máy tính nên nó có tập lệnh đa dạng. Tùy vào từng ứng dụng thực tế mà người sử dụng lựa chọn các lệnh cụ thể sao cho hợp lý nhất. Dưới đây là một số lệnh cơ bản trong ngôn ngữ LAD.

4.3.2.1 Nhóm lệnh với bit

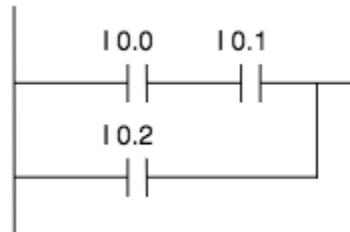
* Tiếp điểm thường mở (ON): --| |--

Tiếp điểm sẽ đóng khi giá trị trong <địa chỉ> bằng 1. Khi tiếp điểm đóng thì dòng trên thang sẽ qua tiếp điểm, RLO=1. Ngược lại, tiếp điểm sẽ mở, dòng trên thang sẽ không đi qua tiếp điểm, RLO=0

Kiểu dữ liệu BOOL, toán hạng là địa chỉ I, Q, M, L, D, T, C.

Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1



Hình 4.8. Mạch sử dụng tiếp điểm thường mở

Hình 4.8 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng tiếp điểm thường mở. Mạch điện sẽ chỉ đóng khi thỏa mãn một trong hai điều kiện sau:

- + Hai tiếp điểm thường mở là I0.0 và I0.1 đều có giá trị bằng 1
- + Tiếp điểm thường mở I0.2. có giá trị bằng 1.

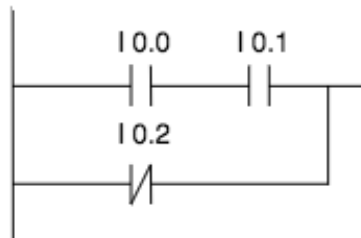
* Tiếp điểm thường đóng (OFF): --|/|--

Tiếp điểm sẽ đóng khi giá trị trong <địa chỉ> bằng 0. Khi tiếp điểm đóng thì dòng trên thang sẽ qua tiếp điểm, RLO=1. Ngược lại, tiếp điểm sẽ mở, dòng trên thang sẽ không đi qua tiếp điểm, RLO=0

Kiểu dữ liệu BOOL, toán hạng là địa chỉ I, Q, M, L, D, T, C.

Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1



Hình 4.9. Mạch sử dụng tiếp điểm thường đóng

Hình 4.9 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng tiếp điểm thường đóng. Mạch điện sẽ chỉ đóng khi thỏa mãn một trong hai điều kiện sau:

- + Hai tiếp điểm thường mở là I0.0 và I0.1 đều có giá trị bằng 1
- + Tiếp điểm thường đóng I0.2. có giá trị bằng 0.

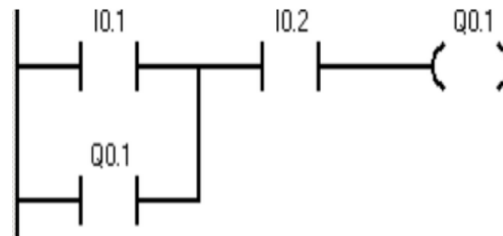
* Tiếp điểm ra (OUT): --()--

Nếu có dòng qua tiếp điểm ($RLO = 1$), bit ở địa chỉ tương ứng có giá trị bằng 1. Ngược lại, ($RLO = 0$), bit ở địa chỉ tương ứng có giá trị bằng 0.

Kiểu dữ liệu BOOL, toán hạng là địa chỉ I, Q, M, L, D, T, C.

Chỉ sử dụng một lệnh OUT cho một địa chỉ.

Lệnh xuất tín hiệu điều khiển ở lối ra hoặc cho các lệnh trung gian.



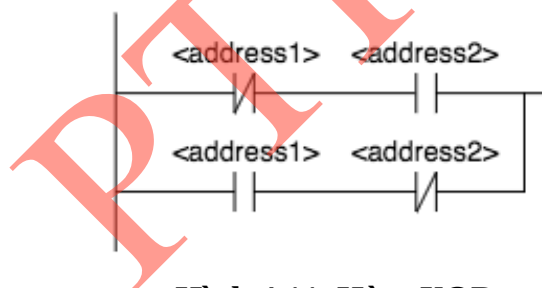
Hình 4.10. Mạch sử dụng tiếp điểm ra

Hình 4.10 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng tiếp điểm phát ra

Điều khiển lối ra OUT Q1.0 bằng hai tiếp điểm thường mở là I0.1 và I0.2. Nếu I0.1 và I0.2, Q1.0 sẽ đóng, tới khi I0.1 và I0.2 không còn tác động thì Q1.0 sẽ mở nên cần thêm lối Q1.0 ở lối vào để duy trì mạch điện đầu ra.

* Bit XOR (Exclusive OR):

Sử dụng tiếp điểm thường đóng và thường mở như hình 4.11 để tạo hàm XOR.



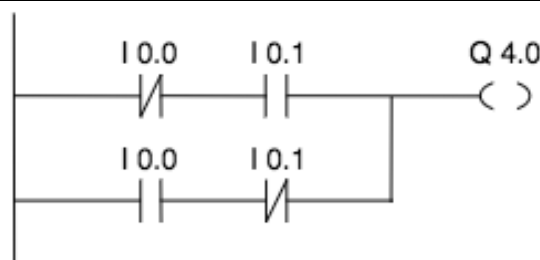
Hình 4.11. Hàm XOR

XOR làm $RLO=1$ khi trạng thái tín hiệu của hai bit address1 và address2 là khác nhau.

Kiểu dữ liệu BOOL, toán hạng là địa chỉ I, Q, M, L, D, T, C.

Lệnh thay đổi nội dung thành ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	x	x	x	1



Hình 4.12. Mạch sử dụng hàm XOR

Hình 4.12 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD mô tả hàm XOR.

$$Q4.0 = \overline{I0.0} * I0.1 + I0.0 * \overline{I0.1} \quad (4.1)$$

* Lệnh NOT: --| NOT |--

Lệnh đảo bit RLO.

Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

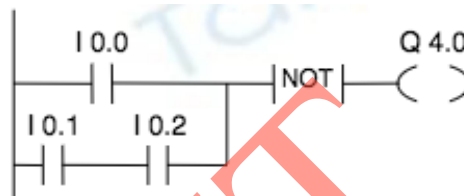
BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	-	1	x	-

Hình 4.13 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng hàm NOT.

Q4.0=0 khi:

+ Tín hiệu tại I0.0 có giá trị bằng 1.

+ Tín hiệu tại I0.1 và I0.2 đồng thời có giá trị bằng 1.



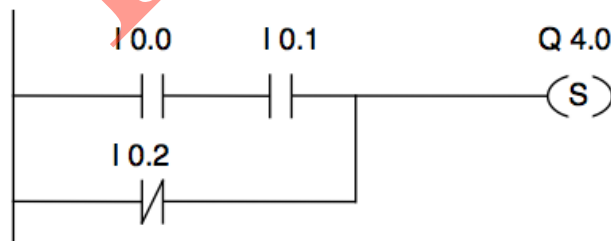
Hình 4.13. Mạch sử dụng hàm NOT

* Lệnh SET: --(S)--

Giá trị của bit sẽ bằng 1 khi đầu vào của lệnh này bằng 1 (đưa giá trị lên khi có điện), khi đầu vào chuyển về 0 thì các bit này vẫn giữ nguyên trạng thái.

Toán hạng là địa chỉ Q, M, SM, T, C, IB, QB, MB.

Hình 4.14 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng lệnh SET.



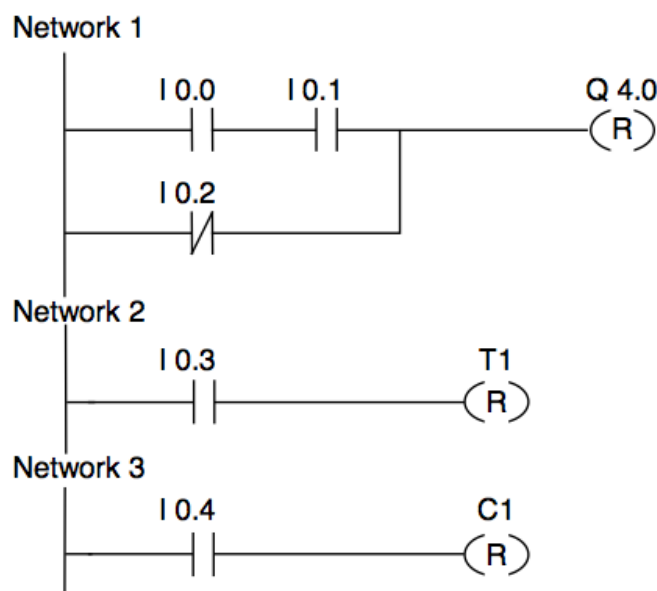
Hình 4.14. Mạch điện sử dụng hàm SET(S)

* Lệnh Reset: --(R)--

Giá trị của bit sẽ bằng 0 khi đầu vào của lệnh này bằng 1 (đưa giá trị về 0 khi có điện), khi đầu vào chuyển về 0 thì các bit này vẫn giữ nguyên trạng thái.

Toán hạng: Q, M, SM, T, C, IB, QB, MB.

Hình 4.15 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng lệnh Reset.



Hình 4.15. Mạch điện sử dụng lệnh Reset (R)

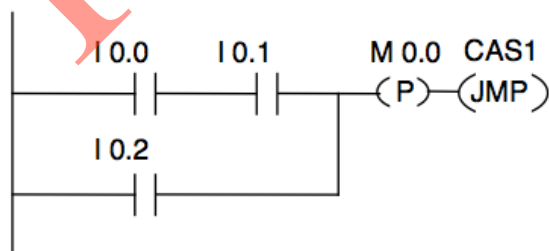
* Tiếp điểm phát hiện sườn lên: $--|P|--$

Nó sẽ phát hiện ra xung khi đầu vào tiếp điểm P có sự chuyển đổi từ 0 lên 1 và biểu diễn nó thông qua việc cho RLO=1 sau lệnh này. Trạng thái tín hiệu hiện tại của RLO được so sánh với trạng thái tín hiệu của địa chỉ, bit nhớ sườn. Nếu trạng thái tín hiệu của địa chỉ là 0 và RLO bằng 1 trước lệnh này thì RLO sẽ bằng 1 sau lệnh này, và bằng 0 trong tất cả các trường hợp còn lại.

Độ rộng của xung này bằng thời gian một chu kỳ quét.

Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	x	x	1



Hình 4.16. Mạch điện sử dụng tiếp điểm phát hiện sườn lên

Hình 4.16 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng tiếp điểm phát hiện sườn lên.

Bit nhớ sườn M0.0 sẽ lưu giữ trạng thái RLO cũ. Khi có tín hiệu thay đổi RLO từ 0 lên 1, chương trình sẽ nhảy tới nhãn CAS1.

* Tiếp điểm phát hiện sườn xuống: $--|N|--$

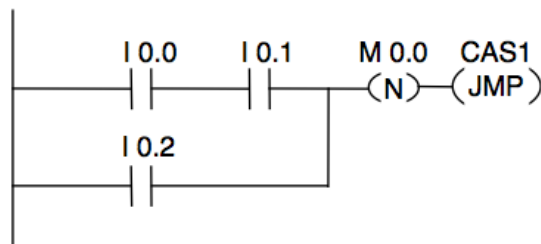
Nó sẽ phát hiện ra xung khi đầu vào tiếp điểm N có sự chuyển đổi từ 1 xuống 0 và biểu diễn nó thông qua việc cho RLO=1 sau lệnh này. Trạng thái tín hiệu hiện tại của RLO được so sánh với trạng thái tín hiệu của địa chỉ, bit nhớ sườn. Nếu trạng thái tín hiệu của địa chỉ là 1 và RLO

bằng 0 trước lệnh này thì RLO sẽ bằng 1 sau lệnh này, và bằng 0 trong tất cả các trường hợp còn lại. Độ rộng của xung này bằng thời gian một chu kỳ quét.

Hai lệnh phát hiện sườn lên và sườn xuống được sử dụng khi muốn lỗi ra tác động chính xác.

Lệnh thay đổi nội dung thanh ghi trạng thái như sau:

BR	CC1	CC0	OV	OS	OR	STA	RLO	FC
-	-	-	-	-	0	x	x	1



Hình 4.17. Mạch sử dụng tiếp điểm phát hiện sườn xuống

Hình 4.17 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng tiếp điểm phát hiện sườn xuống.

Bit nhớ sườn M0.0 sẽ lưu giữ trạng thái RLO cũ. Khi có tín hiệu thay đổi RLO từ 1 về 0, chương trình sẽ nhảy đến nhãn CAS1.

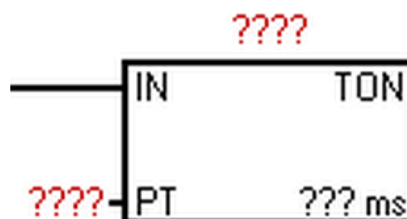
4.3.2.2. Timer

Timer có địa chỉ từ T0 đến T255, được dùng thường xuyên trong các chương trình điều khiển, lập trình PLC. Và có 3 loại cơ bản sau:

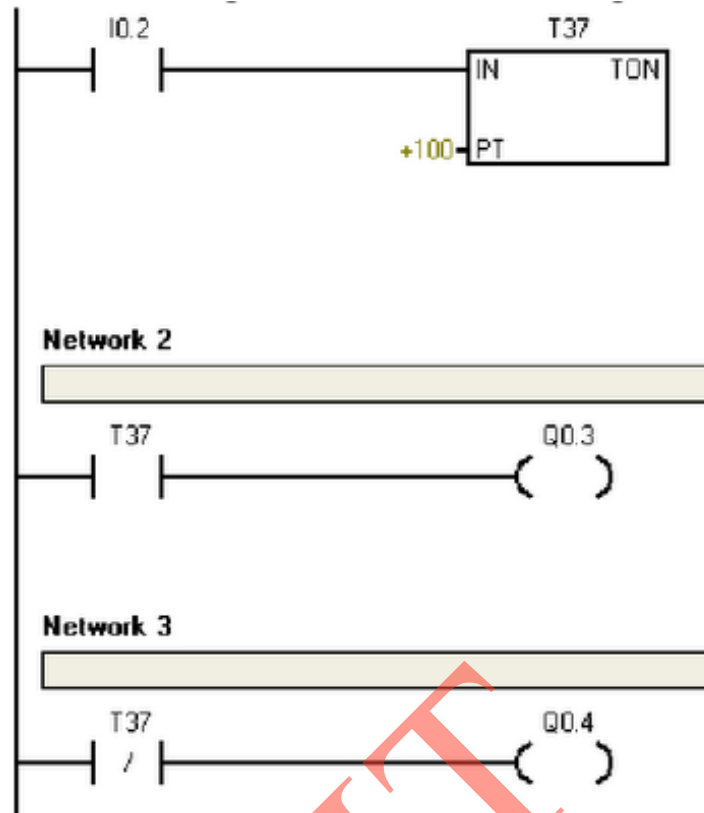
- Đóng mạch chậm - TON (On-delay Timer).
- Đóng mạch chậm có nhớ - TONR (Retentive On-delay Timer).
- Mở mạch chậm - TOF (Off-delay Timer).

Khi sử dụng một Timer, ta cần quan tâm một số vấn đề sau đây:

- Loại Timer: ON, OFF, ONR
- Thời gian delay của Timer: 1ms, 10ms, 100ms...
- Các thông số khi đưa Timer ra sử dụng, cài đặt.



Hình 4.18. Timer TON



Hình 4.19. Mạch sử dụng Timer để đóng ngắt động cơ.

Hình 4.19 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng Timer.

Timer TON sẽ hoạt động khi lối vào IN có giá trị bằng 1. Sau một thời gian (giá trị đặt ở chân PT), Timer sẽ tác động và khi đó ra sẽ dùng các lối ra logic TON để điều khiển các nhánh.

Muốn dừng Timer, ta dùng lệnh reset (R) hoặc ngắt nguồn vào IN.

Dùng Timer loại này khi muốn trì hoãn một khoảng thời gian rồi sau đó mới tác động.

Trong ví dụ trên ta thấy dùng công tắc I0.2 đưa vào lối IN của Timer nên khi tác động vào công tắc I0.2 thì T37 tác động. Đặt giá trị 100 vào chân PT, giả sử Timer có độ phân giải 100ms thì sau 10s, T37 sẽ tác động. Khi đó sẽ đóng Q0.3 (network 2) và mở Q0.4 (network 3).

Nếu muốn Timer hết tác dụng ta tắt công tắc I0.2.

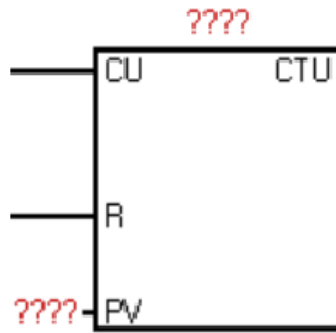
4.3.2.3. Counter

* Counter CU

Counter sẽ đếm khi có xung kích vào chân CU.

Khi muốn Reset Counter thì kích hoạt R. Nếu tác động chân R lên 1 thì Counter reset các tiếp điểm, Counter sẽ trở về trạng thái ban đầu (tức các tiếp điểm hở sẽ trở về hở và đóng sẽ về đóng).

Giá trị đếm của Counter được gán vào chân PV.



Hình 4.20. Counter CU

Khi giá trị đếm vào chân CU lên bằng (hoặc cao hơn) giá trị đã gán ở chân PV. Các tiếp điểm của Counter sẽ tác động.

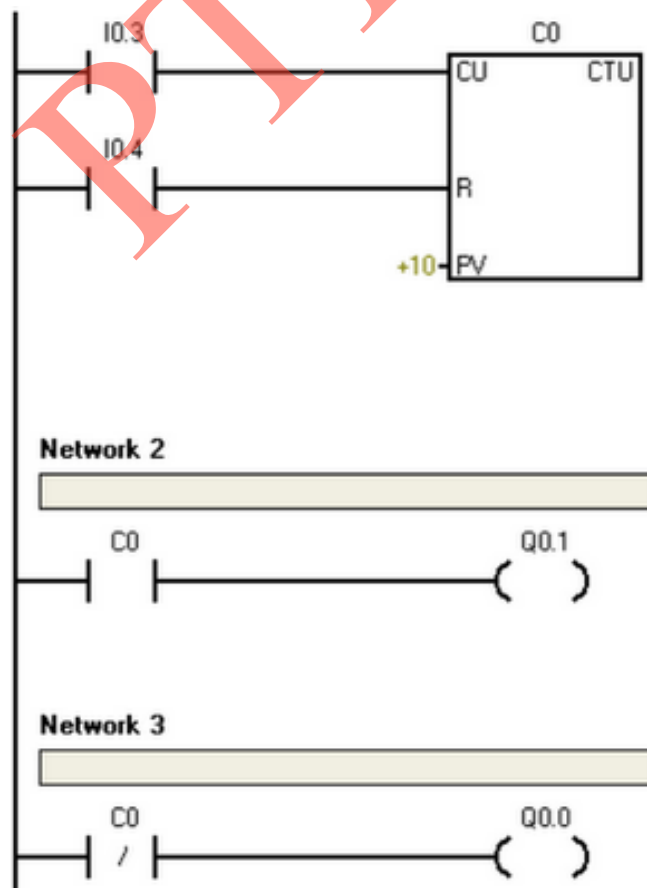
Hình 4.21 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng Counter CU.

Trong ví dụ này ta dùng Counter C0 loại CU (loại đếm lên khi có sườn lên). Khi chân đếm CU là I0.3 có xung kích lên 1, giá trị sẽ được C0 đếm lên 1. Khi kích I0.3 lên 1 tiếp, C0 đếm thêm 1 nữa. Vậy giá trị C0 đang lưu trữ là 2. Tiếp tục cho đến khi giá trị C0 là đã đếm đủ 10, bằng chân PV, thì các tiếp điểm của C0 hoạt động.

Network 2: C0 thường hờ đóng lại và cung cấp nguồn cho Q0.1.

Network 3: C0 thường đóng mở ra làm mất nguồn Q0.0.

Khi muốn Reset C0, ta kích vào chân I0.4. Khi đó các tiếp điểm của C0 tại network 2 và 3 sẽ trở về trạng thái ban đầu, tức làm cho Q0.1 tắt nguồn và Q0.0 có nguồn.



Hình 4.21. Mạch sử dụng Counter CU

* Counter CUD

Counter CUD đếm tiến theo sườn lên của chân CU và đếm lùi theo sườn lên của chân CD.

Khi muốn Reset Counter, kích hoạt chân R (R=1). Nếu tác động chân R lên 1 thì Counter Reset các tiếp điểm để trở về trạng thái ban đầu (tức các tiếp điểm hở sẽ trở về hở và đóng sẽ về đóng).

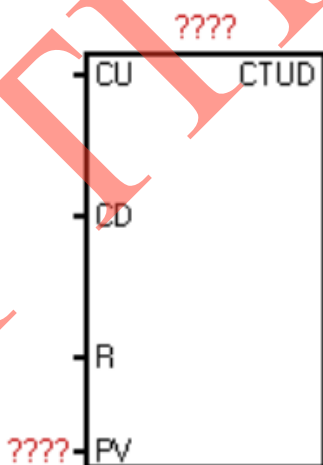
Giá trị đếm của Counter được gán vào chân PV.

Khi giá trị đếm vào chân CU lên bằng (hoặc cao hơn) giá trị đã gán ở chân PV, các tiếp điểm của Counter sẽ tác động.

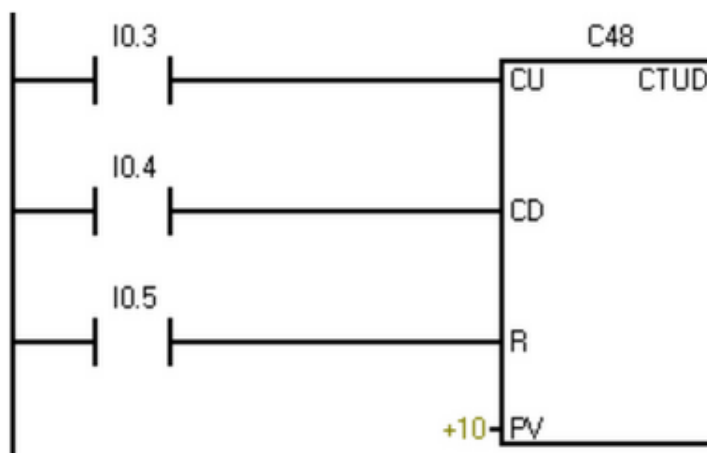
Hình 4.23 là ví dụ mạch điện xây dựng bằng ngôn ngữ LAD sử dụng Counter CUD.

Trong ví dụ này, ta dùng Counter C48 loại CUD. Khi chân đếm CU là I0.3 có xung kích lên 1, giá trị sẽ được C48 đếm lên 1. Nếu I0.3 lại lên 1 nữa, giá trị sẽ được C48 tăng tiếp thêm 1. Vậy giá trị C0 đang lưu trữ là 2. Nhưng khi chân I0.4 lên 1 thì CD sẽ tiếp nhận và hạ mức lưu trữ trong C48 xuống 1. Vậy giá trị lưu trữ của C48 chỉ còn 1. Tiếp tục cho đến khi giá trị C0 là đã đếm đủ 10, bằng chân PV, thì các tiếp điểm của C0 hoạt động. Tức lỗi thường đóng của C48 sẽ mở và lỗi thường mở của C48 sẽ đóng lại.

Khi muốn Reset Counter C48 thì kích vào chân I0.5, các tiếp điểm của C48 sẽ trở về trạng thái ban đầu.



Hình 4.22. Counter CUD



Hình 4.23. Mạch sử dụng Counter CUD

4.3.2.4. So sánh

Trong quá trình lập trình PLC cho hệ thống dù lớn hay nhỏ chúng ta đều cần sử dụng lệnh so sánh (Compare) để đưa ra các điều khiển thông minh nhất và tiện lợi nhất. Trong đó, lệnh so sánh được dùng với rất nhiều giá trị từ Byte, Integer, Double Word, Real.

* So sánh bằng: `--| ==B |--`

* So sánh lớn hơn: `--| >B |--`

* So sánh nhỏ hơn: `--| <B |--`

* So sánh lớn hơn hoặc bằng: `--| >=B |--`

* So sánh nhỏ hơn hoặc bằng: `--| <=B |--`

* So sánh khác: `--| <>B |--`

Chữ B có thể thay bằng chữ I, D, R.

4.4. Ứng dụng

4.4.1. Ứng dụng trong toán học

Vì PLC được xây dựng dựa trên các phép toán logic nên nó được sử dụng để giải quyết các bài toán khá hiệu quả.

Ví dụ 1: Nếu số thực x thỏa mãn điều kiện $x > 3.1416$ thì đảo đèn trạng thái đầu ra.

```
A      Q4.0
L      MD10
L      3.1416 // nạp 3.1416 vào ACCU1, giá trị ở ô nhớ MD10 vào ACCU2.
-R
X      >0
=      Q4.0
```

Ví dụ 2: Nếu số nguyên trong MD10 thỏa mãn $x < -2$ hoặc $x > 3$ thì báo đèn Q4.0 sáng.

```
L      -2
L      MD10
>D      // RLO = 1 nên -2 > MD10
L      3
-R      // nếu MD10 > 3 thì CC1=1, CC0=0 và không thay đổi RLO
O      >0
=      Q4.0
```

Ví dụ 3: Đổi giá trị nguyên 16 bits ($-3278 \div 3277$) đọc từ cổng PIW304 thành một số thực có giá trị đúng bằng mức điện áp tín hiệu tại cổng ($-10V \div 10V$) và cất kết quả vào ô nhớ MD0.

```
L      PIW304 //số đọc được là số nguyên 16 bits
ITD      // chuyển thành số nguyên 32 bits
DTR      // chuyển thành số thực
L      3267.7
```

/R // tính ra đúng giá trị điện áp tại cổng
T MDO

4.4.2 Ứng dụng trong điện tử viễn thông

Mặc dù điện tử - viễn thông không phải là lĩnh vực mà PLC hướng tới nhưng vẫn có khả năng sử dụng nó để xây dựng các hệ thống đơn giản.

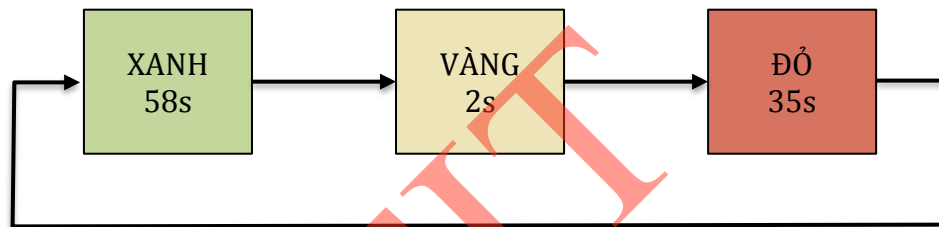
Ví dụ: Xây dựng hệ thống điều khiển đèn tín hiệu giao thông sử dụng PLC S7-300 với các yêu cầu sau:

+ Đèn xanh được giữ trong khoảng thời gian là 58s rồi chuyển sang đèn vàng 2s, sau đó chuyển tiếp sang đèn đỏ 35s, lại quay sang đèn xanh 58s...

+ Chế độ điều khiển tự động này được kích hoạt khi tín hiệu I0.0 có giá trị bằng 1.

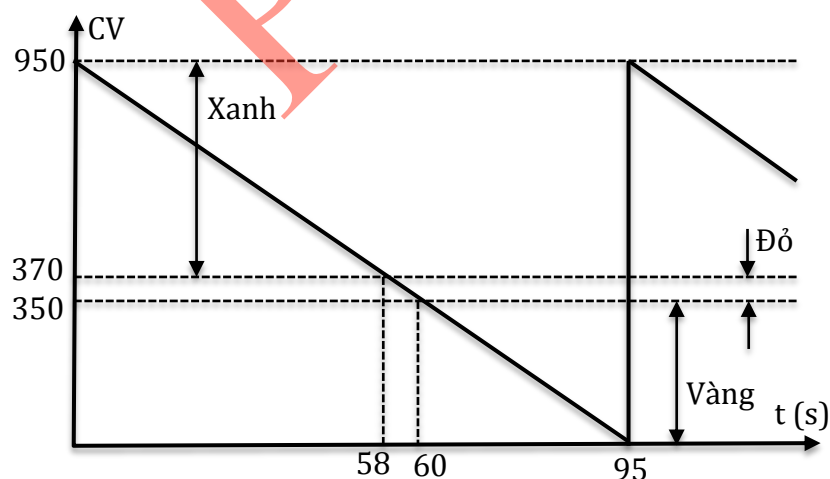
Giải:

* Cách thức hoạt động của hệ thống được minh họa như hình 4.24



Hình 4.24. Cách thức hoạt động của đèn tín hiệu giao thông

* Từ yêu cầu của bài toán, ta tính được giá trị PV nhập vào cho Timer với đèn xanh, đèn đỏ và đèn vàng như hình 4.25.



Hình 4.25. Giá trị PV cho Timer

* Chương trình điều khiển viết bằng ngôn ngữ LAD

```

A    I0.0 //Kích hoạt chế độ điều khiển tự động
A    T1 // Sử dụng Timer T1
L    W#16#1950 //Tạo trễ 95s
SD    T1 //Sử dụng loại Timer trễ sườn lên không nhớ
  
```

L	W#16#1370	//Đèn xanh
LC	T1	//Đọc giá trị đếm tức thời CV
<=I		
JC	Xanh	
L	W#16#1350	//Đèn vàng
>I		
JC	Vang	
S	Q4.0	//Bật đèn đỏ, tắt đèn xanh và vàng
R	Q4.1	
R	Q4.2	
BEU		
Xanh:	S	Q4.2 //Bật đèn xanh, tắt đèn đỏ và vàng
	R	Q4.0
	R	Q4.1
Vang:	S	Q4.1 // Bật đèn vàng, tắt đèn xanh và đỏ
	R	Q4.0
	R	Q4.2

4.4.3. Ứng dụng trong điều khiển

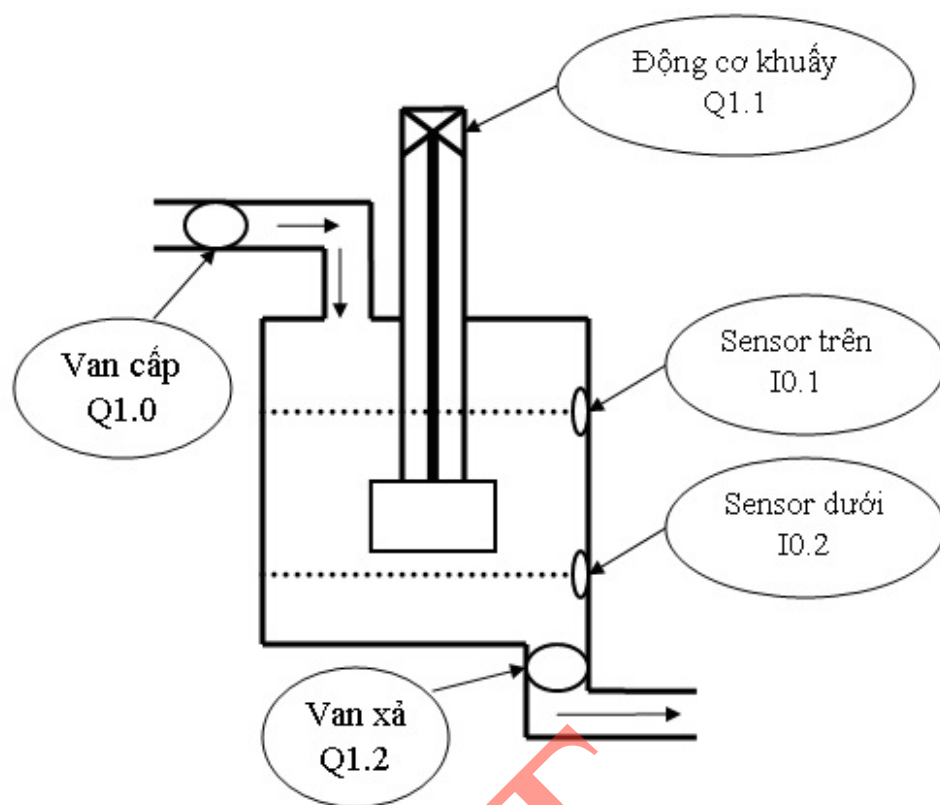
Điều khiển là lĩnh vực PLC được sử dụng rộng rãi nhất. Tùy theo yêu cầu của bài toán mà người sử dụng lựa chọn các PLC có cấu hình phù hợp.

Ví dụ 1: Xây dựng hệ thống tháo/rót nhiên liệu với các yêu cầu sau:

- + Nhấn nút **START**, sau 10 giây thì van cấp mở nhiên liệu vào thùng
- + Sau 2 phút nếu nhiên liệu trong thùng không vượt quá mức dưới thì dừng cấp và thông báo lỗi ra bên ngoài
- + Khi nhiên liệu vượt qua mức dưới thì động cơ khuấy bắt đầu hoạt động
- + Khi nhiên liệu vượt quá mức trên thì đóng van, sau 10 giây dừng động cơ khuấy
- + Nhấn nút **START** lần nữa, van xả mở và tháo nhiên liệu. Khi nhiên liệu xuống dưới mức thấp thì van xả đóng.
- + Thực hiện chu trình trên 4 lần thì đèn **END** báo sáng và kết thúc quá trình.

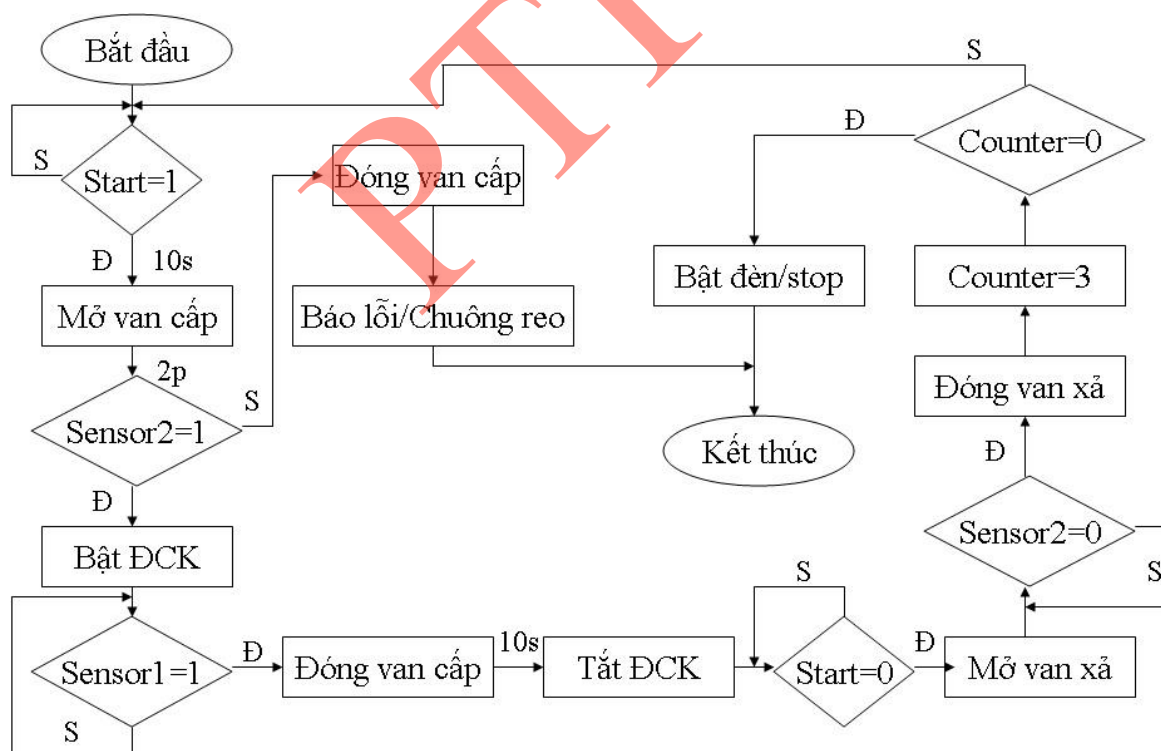
Giải:

Từ yêu cầu của bài toán, ta xây dựng được mô hình hệ thống như hình 4.26



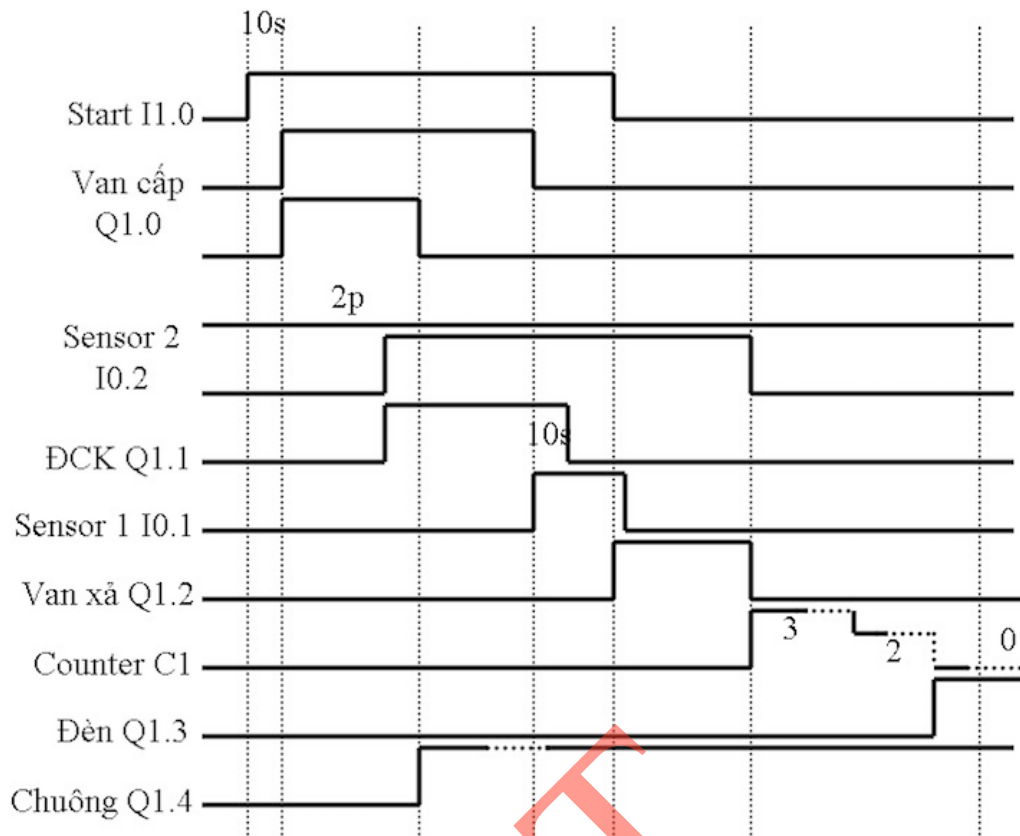
Hình 4.26. Mô hình hệ thống tháo/rót nhiên liệu

* Lưu đồ thuật toán như hình 4.27.



Hình 4.27. Lưu đồ thuật toán

* Giảm đồ thời gian như hình 4.28. Giảm đồ này thể hiện 1 chu trình, các chu trình sau lặp lại tương tự, sau mỗi chu trình thì Counter C1 sẽ đếm lùi từ 3, khi nào Counter C1 đếm tới 0, tức là đã thực hiện được 4 chu trình thì đèn báo END được bật lên và kết thúc quá trình, thoát khỏi hệ thống.



Hình 4.28. Giải đồ thời gian của hệ thống tháo rót nhiên liệu

Trong đó:

Start: I1.0 (ấn Start lần 1 là ở mức 1, ấn Start lần nữa là trở về mức 0).

Sensor trên (sensor1): I0.1.

Sensor dưới (sensor2): I0.2.

Van cấp: Q1.0.

Động cơ khuấy (ĐCK): Q1.1.

Van xả: Q1.2.

Đèn báo: Q1.3.

Chuông báo lỗi: Q1.4

*** Chương trình điều khiển viết bằng ngôn ngữ LAD**

Network1: // Start - Mở van cấp sau khi có sườn lên của I1.0 được 10s

A I1.0

FP M1.0 // Khi có sườn lên của I1.0 (ấn Start)

R Q1.0 // Đóng van cấp

R Q1.1 // Tắt ĐC khuấy

R Q1.2 // Đóng van xả

L S5T#10s // Trễ 10s

SD T20

A T20
FP M2.0
S Q1.0 // Mở van cấp

Network2: // Bật chuông - Báo lỗi thoát khỏi hệ thống

A Q1.0 //Mở van cấp
L S5T#2p //Sau 2 phút
SD T30
A T30
FP M3.0
AN I0.2 //Không có sườn lên của I0.2
R Q1.0 // Đóng van cấp
S Q1.4 //Bật chuông báo lỗi
BEU //Dừng hệ thống

Network3: // Bật ĐC khuấy khi có sườn lên của I0.2

A I0.2
FP M0.2
S Q1.1

Network4: // Đóng van cấp khi có sườn lên của I0.1

A I0.1
FP M0.1 //Khi có sườn lên của I0.1
R Q1.0 //Đóng van cấp

Network5: // Tắt ĐCK sau khi đóng van cấp được 10s

AN Q1.0 //Đóng van cấp
L S5T#10s //Trễ 10s
SD T40
A T40
FP M4.0
R Q1.1 //Tắt ĐCK

Network6: // Mở van xả khi có sườn xuống của I1.0 và Q1.1 tắt
//Tránh được trường hợp ĐCK đang quay mà đã lại ấn
// nút Start lần nữa, khi đó Start vô tác dụng.

A I1.0
FN M1.0

AN Q1.1

S Q1.2

Network7: // Đóng van xả khi có sườn xuống của I0.2. Bật

Counter

A I0.2

FN M0.2

R Q1.2

L W#16#3 //Nạp giá trị 3 vào Counter

FR C1 //Bật Counter

CD C1 //Đếm lùi từ 3

LC //Nạp giá trị tức thời của C1 vào ACCU1

L W#16#0 //Nạp giá trị 0 vào ACCU1, khi đó giá trị tức thời của C1 được đưa sang ACCU2

=I //So sánh ACCU1 với ACCU2, nếu bằng thì nhảy tới nhãn

JC Batden_Stop

Batden_Stop: // Bật đèn END và dừng hệ thống

S Q1.3

BEU

Ví dụ 2: Xây dựng hệ thống đếm sản phẩm với các yêu cầu sau:

- + Khi có tín hiệu START thì hệ thống sẽ hoạt động và STOP thì quá trình sẽ dừng.
- + Khi hệ thống hoạt động, băng tải sản phẩm chạy để đưa sản phẩm đến một vị trí được định sẵn để kiểm tra. Sản phẩm không có nhãn là sản phẩm có lỗi và sẽ được chuyển sang một băng tải khác để đưa ra ngoài.
- + Khi đếm được 500 sản phẩm lỗi hoặc 2000 sản phẩm đúng thì hệ thống sẽ dừng hoàn toàn kể cả khi không có tín hiệu STOP. Khi hoạt động, nếu nút PAUSE được kích hoạt thì tạm dừng toàn bộ và quá trình bắt đầu lại khi nhấn nút PAUSE một lần nữa.

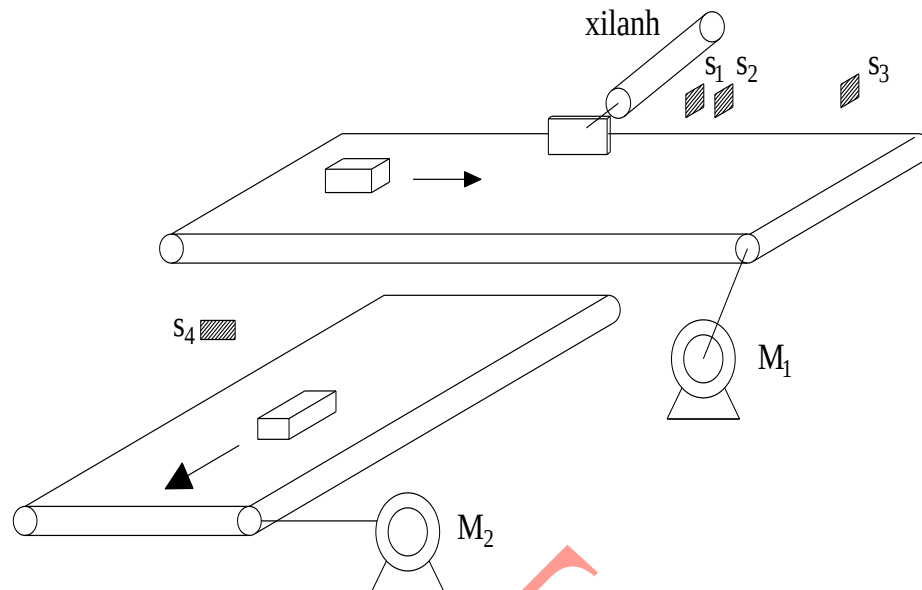
+ Hệ thống được hoạt động trở lại khi có tín hiệu RESET.

Giải:

* Từ yêu cầu của bài toán, ta xây dựng mô hình hệ thống như hình 4.29.

- + M1: Động cơ băng tải 1.
- + M2: Động cơ băng tải 2.
- + S1: Sensor phát hiện vật.
- + S2: Sensor phát hiện vật bị hư.

- + S3: Sensor đếm sản phẩm tốt ở băng tải 1.
- + S4: Sensor đếm sản phẩm hư ở băng tải 2.
- + Xilanh đẩy sản phẩm sang băng chuyền.



Hình 4.29. Mô hình hệ thống đếm sản phẩm

* Chương trình được viết cho S7-300, khối OB1 bằng ngôn ngữ LAD:

- Bảng gán địa chỉ:

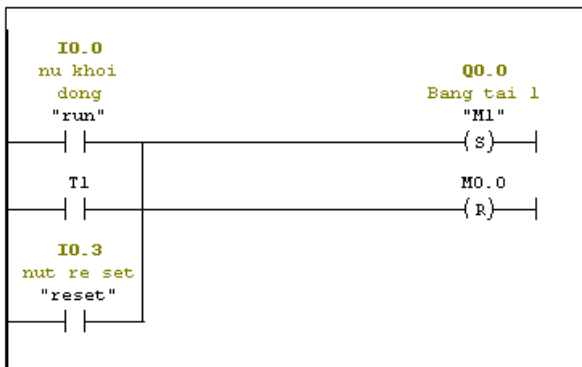
Symbol Editor - [S7 Program(1) (Symbols) -- S7_Pro1\SIMATIC 300 Station\CPU312C(1)]					
Symbol Table Edit Insert View Options Window Help					
All Symbols					
	Status	Symbol	Address	Data type	Comment
1		Cycle Execution	OB 1	OB 1	
2		M1	Q 0.0	BOOL	Băng tải 1
3		M2	Q 0.2	BOOL	Động cơ băng tải 2
4		reset	I 0.3	BOOL	nút re set
5		run	I 0.0	BOOL	nu khởi động
6		s1	I 0.1	BOOL	sensor phát hiện vật
7		s2	I 0.2	BOOL	sensor phát hiện vật bị hư
8		s3	I 0.4	BOOL	sensor đếm sản phẩm con tốt ở băng tải 1
9		s4	I 0.5	BOOL	sensor đếm sản phẩm bị hư ở băng tải 2
10		stop	I 0.6	BOOL	nút dừng
11		Y	Q 0.1	BOOL	điều khiển xilanh đi ra
12					

Hình 4.30. Bảng gán địa chỉ cho các biến

+ Code mô phỏng:

Network 1: Title:

Comment:



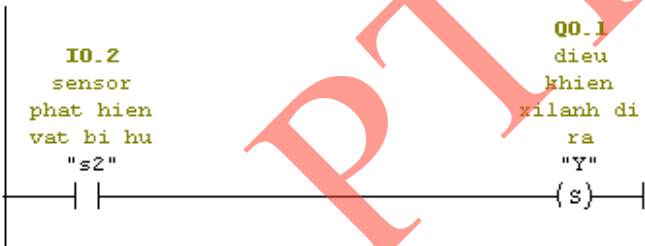
Network 2: Title:

Comment:



Network 3: Title:

Comment:



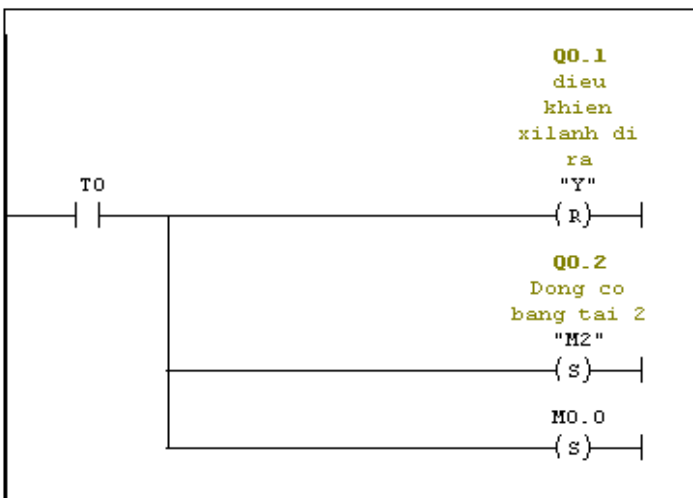
Network 4: Title:

Comment:



Network 5: dieu khien xilanh di ra

Comment:



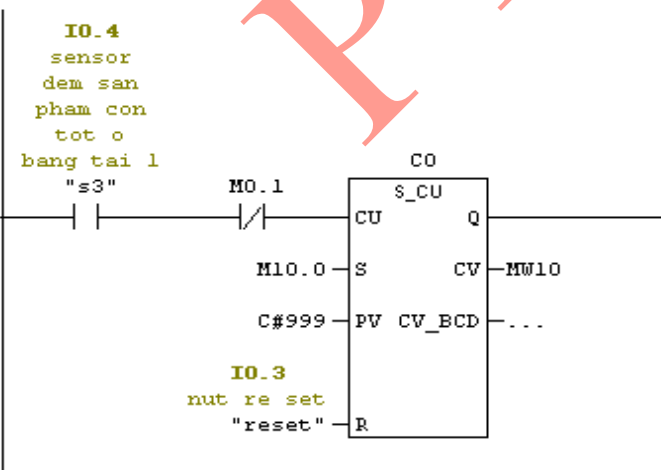
Network 6 : Title:

Comment:



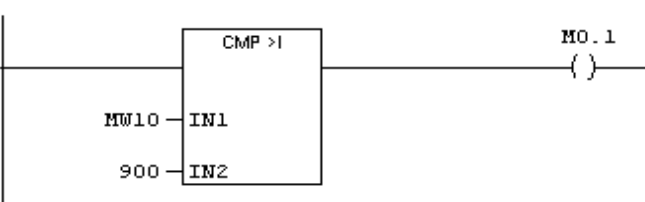
Network 7 : Title:

Comment:



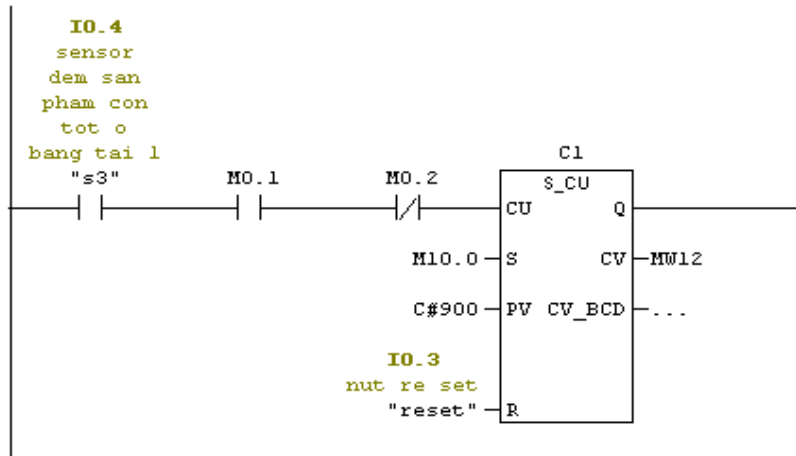
Network 8 : Title:

Comment:



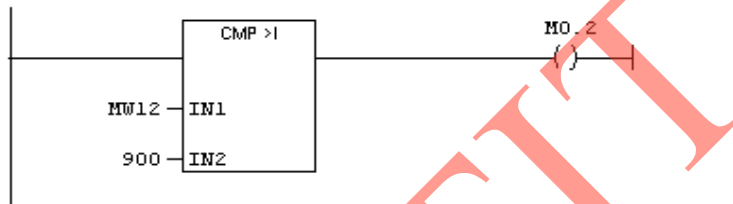
Network 9 : Title:

Comment:



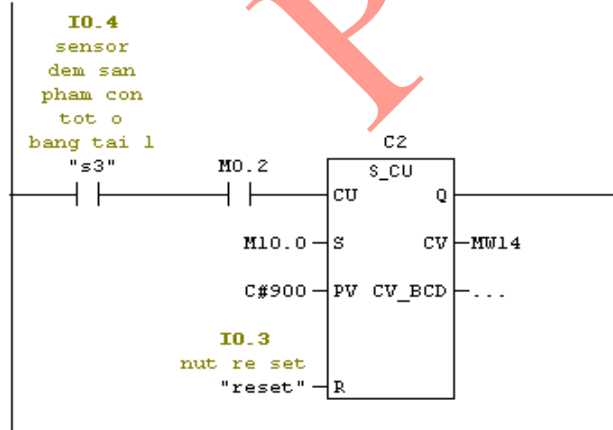
Network 10 : Title:

Comment:



Network 11 : Title:

Comment:



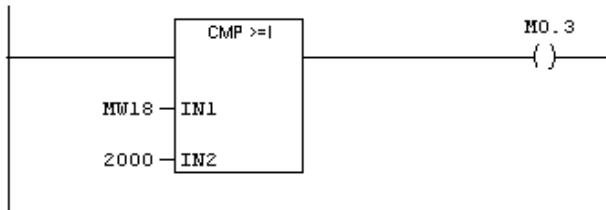
Network 12 : Title:

Comment:



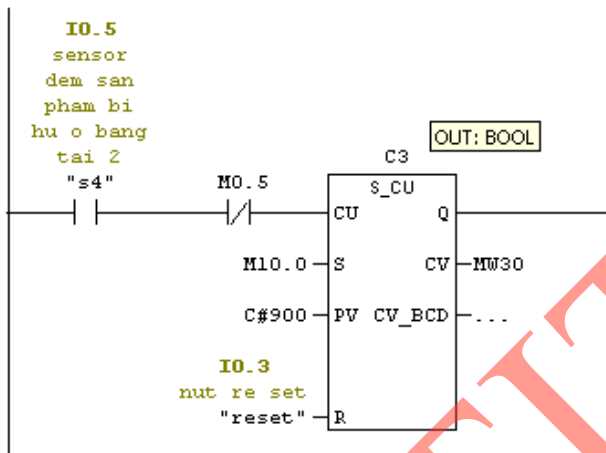
Network 13 : Title:

Comment:



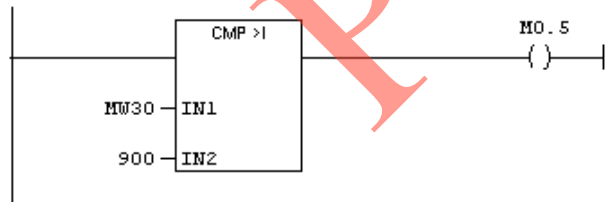
Network 14 : Title:

Comment:



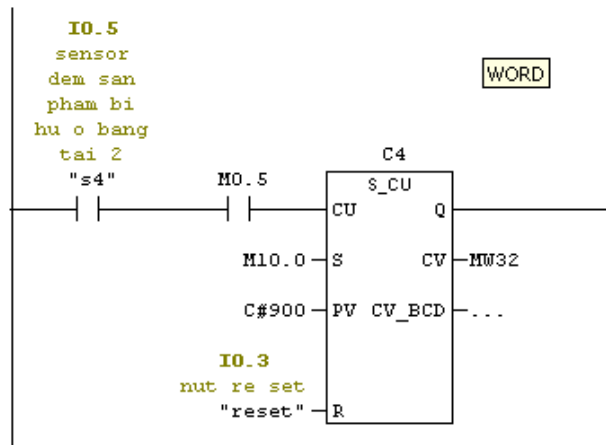
Network 15 : Title:

Comment:



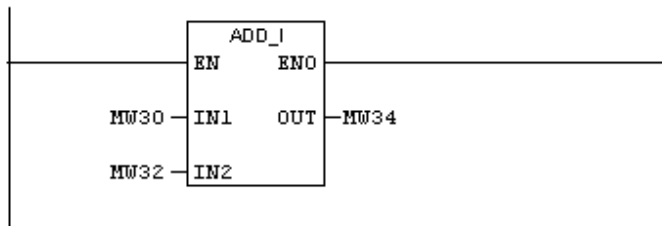
Network 16 : Title:

Comment:



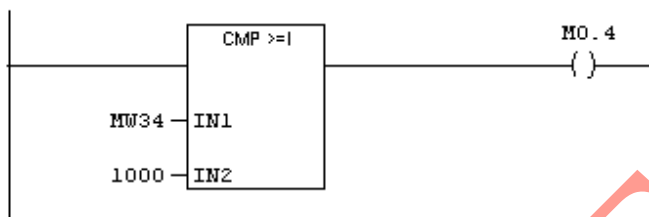
Network 17 : Title:

Comment:



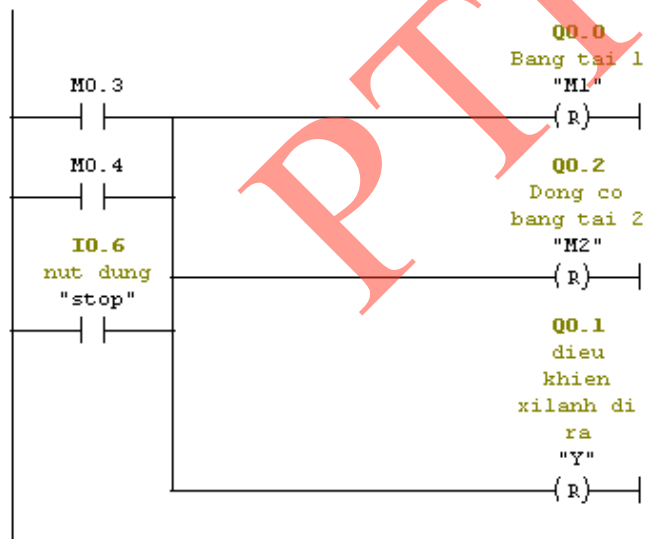
Network 18 : Title:

Comment:



Network 19 : Bang tai 1

Comment:



4.5 Kết luận

Chương 4 đã trình bày khái quát về phần mềm STEP7 được sử dụng để lập trình cho PLC của hãng Siemens. Chương này tập trung giới thiệu ngôn ngữ lập trình STL cho dòng S7-300, là dòng PLC cỡ trung của Siemens bao gồm các thao tác số học với số nguyên 16 bits, số nguyên 32 bits, số thực, các thao tác trên thanh ghi trạng thái. Ngoài ra, chương cũng mô tả cách lập trình bằng ngôn ngữ LAD, một số thao tác với bit để tạo ra các hàm logic cơ bản, cách lập trình cho Timer, Counter và các lệnh so sánh. Kết hợp với kiến thức đã trình bày trong chương 1, 2 và 3, bạn đọc có thể lập trình cho PLC để xây dựng các ứng dụng đơn giản ứng dụng vào một số lĩnh vực như toán học, điện tử viễn thông và đặc biệt là điều khiển tự động.

BÀI TẬP CHƯƠNG 4

Bài tập 4.1

Xây dựng mạch dây dựng hàm NAND bằng ngôn ngữ LAD.

Bài tập 4.2

Xây dựng mạch xây dựng hàm NOR bằng ngôn ngữ LAD.

Bài tập 4.3

Viết đoạn chương trình thực hiện yêu cầu sau:

Nếu số nguyên 16 bits x trong MW10 thỏa mãn $x < -2$ hoặc $x > 3$ thì báo đèn Q1.0 sáng.

Nếu số thực x trong MD10 thỏa mãn $x < 2.3$ thì đảo trạng thái đèn Q3.0.

Bài tập 4.4

Viết đoạn chương trình thực hiện yêu cầu sau:

Nếu số thực x trong MW10 thỏa mãn $-2.2 \leq x < 5.3$ thì báo đèn Q3.0 sáng.

Nếu số thực x trong MD20 thỏa mãn $x < -2.4$ hoặc $x = 3.5$ thì báo đèn Q1.0.

Bài tập 4.5

Viết đoạn chương trình thực hiện yêu cầu sau:

Nếu số nguyên 32 bit x trong MW10 thỏa mãn $x > 3$ đảo trạng thái đèn Q4.0.

Nếu số thực x trong MD10 thỏa mãn $x \neq -2.3$ hoặc $x \geq 2$ thì báo đèn Q2.0 sáng.

Bài tập 4.6

Viết đoạn chương trình thực hiện yêu cầu sau:

Nếu số thực x trong MW20 thỏa mãn $-1.5 < x \leq 4.2$ thì đảo trạng thái đèn Q3.0.

Nếu số nguyên 16 bits x trong MW20 thỏa mãn $x < -1$ hoặc $x > 2$ thì báo đèn Q2.0 sáng.

Bài tập 4.7

Chú thích ý nghĩa từng câu lệnh và cho biết ý nghĩa của đoạn lệnh:

a.

A	Q4.0
L	MW10
L	1.4
-R	
X	> 0

= Q4.0

b.

A Q1.0
L MD10
L -2
-D
X <> 0
= Q1.0

Bài tập 4.8

Chú thích ý nghĩa từng câu lệnh và cho biết ý nghĩa của đoạn lệnh:

a.

L MD10
L 3.1416
/R
T MD20

b.

L IW10
L MW20
*I
T MW25

Bài tập 4.9

Viết chương trình thực hiện:

a. $Q2.0 = I2.0 \cup \overline{I2.1} \cup I2.2$

b. $Q1.0 = (I1.0 \cup \overline{I1.1}) \cup (I1.2 \cup I1.3)$

Bài tập 4.10

Viết chương trình thực hiện:

a. Nếu $\overline{I2.1} \cup I2.2 \neq I2.0$ thì $Q1.0 = 1$

b. Nếu $I1.0 \cup \overline{I1.1} = I1.2$ thì $Q4.0 = 1$

TÀI LIỆU THAM KHẢO

- [1]. *Programmable Controller, Theory and Implementation* , 4th Edition, L.A. Bryan, E.A, Bryan, an Industrial text company Publication, Atlanta, Georgia, USA, 2002
- [2]. *Tự động hoá với Simatic S7-300*, Nguyễn Doãn Phước – Phan Xuân Minh nhà xuất bản khoa học và kỹ thuật năm 2000.
- [3]. *Tự động hoá các quá trình công nghệ*. Trần Doãn Tiến NXB giáo dục 1998.
- [4]. *Điều khiển logic và ứng dụng*, Nguyễn Trọng Thuần – Nhà xuất bản khoa học và kỹ thuật 2001
- [5]. *Kỹ thuật số, máy tính số và ứng dụng*, Lê Mạnh Việt, Đại học Giao thông vận tải - 1997
- [6]. *Automating Manufacturing Systems with PLCs*, Hugh Jack, 2005
- [7]. *PLC Beginner Guide*, Tài liệu hướng dẫn của Omron

PTIT